Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

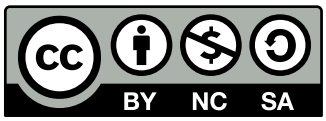
# KEMEDHOC*: A formally verified quantum-safe key exchange protocol

An end-to-end verified workflow from specification to implementation.

---

**Cuong Nguyen**

Master's Thesis
2025



Aalto University
School of Science

**Author** Cuong Nguyen

**Title** KEMEDHOC*: A formally verified quantum-safe key exchange protocol — An end-to-end verified workflow from specification to implementation.

**Degree programme** Computer, Communication and Information Sciences

**Major** Security and Cloud Computing

**Supervisor** Prof. Russell W. F. Lai, Aalto University

**Advisor** Mr. Sampo Sovio (MSc), Huawei Technologies Oy

**Collaborative partner** Huawei Technologies Oy

**Date** 30 October 2025 **Number of pages** 134 **Language** English

**Abstract**

The advent of quantum computing poses significant threats to classical public-key cryptographic systems such as RSA and Elliptic Curve Cryptography, necessitating quantum-safe migration paths for Internet of Things (IoT) communication. The Ephemeral Diffie-Hellman Over COSE (EDHOC) protocol provides lightweight authenticated key exchange for resource-constrained IoT devices but lacks quantum-safe security and formally verified implementations.

This thesis introduces KEMEDHOC, a KEM-based, signature-free, post-quantum variant of EDHOC that achieves mutual authentication through optimized integration of Key Encapsulation Mechanisms (KEMs). Compared to the direct adaptation, KEMEDHOC reduces noticeably message size (by over **33%**) and handshake time (by **15%**), while maintaining comparable security levels.

Formal verification, at design-level, using ProVerif confirms that KEMEDHOC preserve all core EDHOC security properties, including mutual authentication, confidentiality, and forward secrecy. Verified implementations (EDHOC* and KEMEDHOC*) developed in F* and Low* ensure functional correctness, memory safety, and timing constant-time behavior, down to verified C code through KaRaMel compiler.

Overall, this thesis delivers the first end-to-end formally verified post-quantum variant of EDHOC, demonstrating that formal assurance and post-quantum security can be obtained efficiently in lightweight key exchange protocols.

**Keywords** Post-Quantum Cryptography (PQC), Authenticated Key Exchange (AKE), EDHOC, Formal Verification, ProVerif, F*, Low*

# Preface

The successful of this thesis represents a significant undertaking, and it would not have been possible without substantial guidance and support. I am genuinely grateful to everyone who offered their feedback and encouragement throughout this journey.

I am immensely grateful for the opportunity and support provided by Huawei Technologies Oy. Specifically, I extend my profound thanks to my supervisor there, Sampo Sovio. He was exceptionally generous with his time and expertise. I am particularly thankful that he was so welcoming to my initial thesis proposal and offered unwavering support, practical resources, and critical technical perspectives throughout the project.

I wish to express my sincere appreciation to my academic supervisor, Professor Russell W. F. Lai. He provided constructive feedback and insightful suggestions that significantly enhanced the quality and structure of this final document.

I would also like to thank HAIC for generously funding my studies in two consecutive academic years at Aalto University.

Finally, I am deeply thankful to my family and friends for their patience and belief in my work.

All opinions and conclusions presented in this thesis are my own and do not necessarily reflect the views of Huawei Technologies Oy or Aalto University.

Vantaa, 30 October 2025

Cuong Nguyen

# Contents

# 1 Introduction

## 1.1 Key Exchange protocols

Key exchange protocols are a foundational component in securing digital communication. Their primary function is to enable two or more parties to establish a shared cryptographic key over an untrusted network. Once shared, this key can be used to encrypt and authenticate subsequent communications, ensuring confidentiality, integrity, and authenticity.

In traditional computing environments, bootstrapping secure keys between devices is often managed through manual configuration or the use of centralized key distribution services. However, these approaches suffer from significant limitations when scaled to large, dynamic, or resource-constrained environments.

**Static key pre-provisioning.** Pre-shared key (PSK) bootstrapping is one common method for key distribution. However, it becomes unmanageable in systems with many devices. Each device must store a unique key for every peer it may communicate with, leading to scalability and storage issues. PSKs also require secure channels or manual processes for initial key distribution—processes that are not only inconvenient but also prone to human error and misconfiguration. Furthermore, the static nature of PSKs makes them failed to provide *Forward Secrecy*, where the compromise of a pre-shared key can jeopardize all past communications.

**Centralized trusted-third party.** Authenticated key distribution protocols, such as Kerberos [Tec89], mitigate key distribution challenges by relying on a centralized authentication server that distributes session keys, freshly genenrated for each communicating session, to clients. In particular, a client first authenticates itself to the Authentication Server (AS), which then issues a Ticket Granting Ticket (TGT). The client can then use the TGT to request session keys from the Ticket Granting Server (TGS) for communication with other services. After verifying the TGT is valid, the TGS issues a Service Ticket (ST) and session keys to the client. The client can then forward the ST to the Service Server (SS) along with its service request, encrypted using the TGS-issued session key.

While this model works well in closed enterprise environments, it introduces a single point of failure and trust. If the key distribution center (KDC) is compromised or unavailable, the entire system is at risk. Additionally, protocols like Kerberos are not suitable for resource-constrained or battery-powered devices due to their reliance on synchronized clocks and continuous server availability.

**Authenticated Key Exchange (AKE).** Those limitations motivate the use of public-key-based key exchange protocols such as Diffie-Hellman (DH) [DH76] and its authenticated variants. These protocols allow two parties to derive a shared secret without prior knowledge of each other or the need for a trusted third-party.

Authenticated Key Exchange (AKE) protocols strengthen DH with identity verification to prevent Man-in-the-Middle (MitM) attacks.

One of the most widely adopted AKE protocols is Transport Layer Security (TLS) [Res18], which primarily protects communication between web applications and servers over Internet. According to CloudFlare Radar's report, from Oct 2024 to Oct 2025, over 97% of worldwide web traffic is protected by TLS. TLS has undergone several iterations, with TLS 1.3 being the latest version that offers improved security, privacy, and performance. Hence, TLS 1.3 is the de-facto standard for securing web communications today.

## 1.2 Lightweight Key Exchange protocols for IoT

While TLS is effective for securing general-purpose communication, it is not suitable for resource-constrained environments, such as Internet of Things (IoT). IoT devices typically operate under constraints such as limited computing power, energy resources, and memory capacity. Moreover, they frequently operate over low-bandwidth and high-latency networks, such as LoRaWAN [Hax+18] and LPWAN [CZB20]. The overhead of TLS, both in terms of computational complexity and message size, makes it impractical for these settings [RTB20]. Moreover, TLS relies on reliable transport protocols like TCP [Vin81], which are not always available in constrained communication environments.

To address the limitations of TLS, lightweight alternatives have been developed. One notable attempt is Datagram TLS (DTLS) [RTM22], a variant of TLS adapted for unreliable transport layers like UDP. DTLS reduces some overhead but still incurs significant costs in message size, state management, and processing complexity. A typical DTLS handshake involves over 1KB of exchanged data and takes minutes to complete in LoRaWAN environments [Rad+22].

These inefficiencies are particularly problematic for battery-powered devices operating over constrained connections. What is needed are key exchange protocols that minimize bandwidth usage, reduce computational demands, and simplify protocol logic without compromising security.

## 1.3 EDHOC

Ephemeral Diffie-Hellman Over COSE (EDHOC) [SMP24] is a modern lightweight authenticated key exchange protocol tailored for constrained IoT settings. It is designed to establish shared session keys efficiently, using minimal communication and computation. EDHOC achieves compactness by employing Concise Binary Object Representation (CBOR) [BH20] and CBOR Object Signing and Encryption (COSE) [Sch22]. It supports multiple authentication modes and operates over any transport protocol, including those where IP is unavailable, such as Bluetooth Low Energy (BLE) [MLW12].

In EDHOC, the involving peers authenticate each other through signature or static DH keys, resulting in four authentication methods (method 0 to method 3). While method 0 utilizes only signature for authentication, method 3 relies solely on static

DH keys. Method 1 and 2 combines both signature and static DH keys for mutual authentication. This flexibility in authentication materials allows EDHOC to adapt to various security requirements and device capabilities.

Depending on the authentication method and credential type, EDHOC's message size ranges from 101 to 242 bytes [FVW24; Hög+24]. It outperforms DTLS with a 1.44× improvement in key establishment time and a 4× reduction in memory usage [FVW24]. Furthermore, EDHOC serves as a building block for OSCORE [Sel+19], a secure application-layer protocol for IoT.

## 1.4 Research gaps in EDHOC

Despite the upsides of EDHOC in securing constrained environments, there are still unsolved challenges concerning its complete assurance and long-term resistance to quantum adversaries. This thesis project aims to provide solutions by addressing the following two primary **Research Gaps (RG)**:

- **[RG1]:** Post-quantum migration.
- **[RG2]:** Design-to-code formal verification.

The detailed explanations of these two research objectives are elaborated in the subsequent sections. In addition, it is worth noting that other un-tackled challenges, which are out of scope of this thesis, are thoroughly discussed in Section 8.

### 1.4.1 [RG1] Post-quantum migration

Given the foreseeable threat of Cryptographically Relevant Quantum Computer (CRQC) [OPp19], cryptographic schemes based on the hard problem of integer factoring, e.g. RSA [RSA78], or discrete logarithm, e.g. Elliptic Curve Cryptography (ECC) [Mil85; Mil85], are breakable, leading to "Harvest Now, Decrypt Later" threat [ORZ20]. In particular, Shor's algorithm [Sho94; Sho97] can find prime factors of an integer or solve the discrete logarithm problem in polynomial time with a sufficiently large number of qubits. Attackers can now record encrypted communications today and decrypt them in the future when CRQCs become available using Shor's algorithm.

EDHOC, which is based on ECC for both key exchange and authentication, is vulnerable to such quantum attacks. Although EDHOC is designed with post-quantum cryptography (PQC) considerations, there have not been any corresponding specifications, which is stated in the Section 9.4 of RFC 9528 [SMP24]:

> " EDHOC supports all signature algorithms defined by COSE, including PQC signature algorithms such as HSS-LMS. EDHOC is currently only specified for use with key exchange algorithms of type ECDH curves, but any Key Encapsulation Method (KEM), including PQC KEMs, can be used in method 0. While the key exchange in method 0 is specified with the terms of the Diffie-Hellman protocol, the key exchange adheres

to a KEM interface: G_X is then the public key of the Initiator, G_Y is the encapsulation, and G_XY is the shared secret. Use of PQC KEMs to replace static DH authentication would likely require a specification updating EDHOC with new methods. [SMP24] ”

A straighforward approach to post-quantum migration for key exchange protocols is to replace classical DH key exchange with quantum-proof KEM. This substitution has been extensively studied in widely-deployed security protocols and frameworks, such as TLS [KW16; SSW20], WireGuard [Hül+21], Noise [Ang+22], and Split-Key STS [Akm+23], and even EDHOC [Fra+25c]. In the case of signature-based authentication modes (method 0, 1, and 2 in EDHOC), a quantum-safe signature scheme is also required to achieve full post-quantum migration. However, state-of-the-art PQC signature schemes introduce large overhead in both computation and communication costs, which troublesome in low-bandwidth communication environments [Fra+25c; GW22; SKD20]. Furthermore, maintaining multiple codebases for different cryptographic schemes, i.e. KEM and signature, increases the on-device storage cost, which is scarce in constrained devices.

Given the abovementioned issues, we propose a KEM-based, signature-free, authentication method, namely KEMEDHOC, which is inspired by the signature-free spririt of OPTLS [KW16] and the KEM-based approach in KEMTLS [SSW20]. Furthermore, KEMEDHOC provides the equivalent security properties as EDHOC, which is formally proved, while being quantum-resistant and computationally efficient.

### 1.4.2 [RG2] Design-to-code formal verification

Although there are many works on formally verifying EDHOC protocol design, including early drafts and to-be-standardized drafts in both symbolic and computational models [Jac+23; CP22; Fer24; GM23], there is no work, at the time of writing, that investigates the implementation-level security of the protocol. In particular, there are at least three actively maintained implementations, namely UOSCORE-UEDHOC [Hri+21], LAKERS, and CALIFORNIUM, but none of them has functional proofs to guarantee that their executable code corresponds to the design of EDHOC. Protocol implementations include many details which are out of scope of the design-level security analyses, such as, message format, encoding/decoding routines, state machines, and high-level APIs, are error-prone for developers who have no cryptography expertise. Thus, implementation-specific vulnerabilities can be introduced even if the protocol design has been thoroughly verified.

In particular, protocol implementaions have exposed severe bugs, such as HeartBleed [Dat14] and SMACK-TLS [Beu+15], which were not detected even through extensive testing. Additionally, the mentioned implementations do not explicitly establish memory-safety and side-channel resistance. Although LAKERS and CALIFORNIUM are both written in memory-safe languages, Rust and Java respectively, their dependences on underlying C code, the increasingly emergence of microarchitectural-level attacks, like Meltdown [Lip+18] and Spectre [Koc+19], make them still possibly susceptible to certain classes of side-channel vulnerabilities.

To address the abovementioned issues, we utilize the modern, high-level, proof-oriented programming language F* [Swa+16] and its low-level subset Low* [Pro+17].In recent years, several works have utilized these state-of-the-art formal verification tools to construct high-assurance protocol implementations. One of the most remarkable work is miTLS [BFK16] which is a formally verifed implementation of TLS 1.2 in F*. The following-up works targetting other security protocols, such as TLS 1.3 [BBK17], DICE [Tao+21], Noise [Ho+22], and Signal [Pro+19], demonstrate the feasibility and effectiveness of using formal verification to ensure implementation-level security. These verified implementations not only provide functional correctness, which proves that they indeed perform as intended according to their formal specifications, but also guarantee security properties such as memory safety, timing side-channel resistance. In addition, they offer competitive performance compared to unverified counterparts, making them practical for real-world applications.

Given the gap in implementation-level security of EDHOC and the success of the F* ecosystem in building verified protocol implementations, we aim to develop end-to-end verified implementations of EDHOC and KEMEDHOC.

## 1.5 Approach

Firstly, we develop a verified implementation of current EDHOC, refered to as EDHOC*, in F* and Low* for high- and low-level implementation respectively. The implementation ensures functional correctness, memory safety, and resillience to timing side-channel leakages. In addition to serving as a stand-alone verified implementation, EDHOC* forms as a foundation for building the verified implementation of KEMEDHOC.

Subsequently, we design a new authentication method, KEMEDHOC, which is KEM-based, signature-free, and quantum-resistant. Once the preliminary design appears free from naive flaws, we proceed to formally verify the design of KEMEDHOC and implement its verified code, as illustrated in Figure 1.

In Figure 1, the middle rectangle represents the formal specification of KEMEDHOC written in F*, while the right rectangle shows the symbolic model in ProVerif. We thoroughly map the KEMEDHOC design into these F* and ProVerif specifications. The symbolic model is then analyzed by the ProVerif prover to ensure that the protocol design contains no logical flaws and satisfies all claimed security properties, represented by the “Verify Design” rhombus in the figure.

Once the design is formally verified and no attacks are detected, we advance to the machine-level implementation. The left rectangle in Figure 1 illustrates the verified implementation of KEMEDHOC, denoted KEMEDHOC*, developed in F* and Low*. This Implementation guarantees functional correctness with respect to the formal F* specification, constant-time execution, and absence of memory-safety bugs, corresponding to the “Verify Code” rhombus in the figure.

After verification, the Low* code is translated into C using the verified KaRaMel compiler [Pro+17], resulting in the verified C code of KEMEDHOC*. From this verified C code, mainstream C compilers, such as GCC or Clang, can be employed to produce executable binaries for various target platforms.

Regarding cryptographic primitives, we utilize HACL* [Zin+17], a formally verified cryptographic library, to provide crypto backends for both EDHOC* and KEMEDHOC*. HACL* provides both F* specification and Low* implementation of supported cryptographic primitives, so that we can straighforwardly integrate them in our F* and Low* code without concerning about compatibility issues.

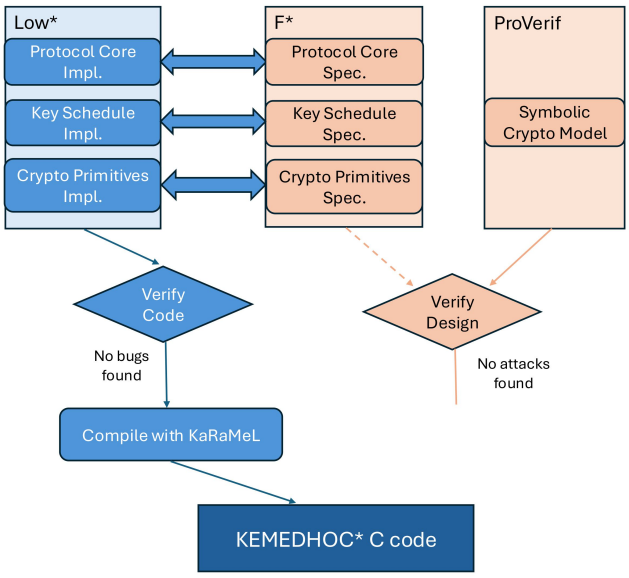



**Figure 1: KEMEDHOC* Architecture**. Orange areas are design-level verification. Blue areas are implementation-level verification.

## 1.6 Contributions

Our main contributions are as follows:

- Based on EDHOC, we design KEMEDHOC, a quantum-safe and signature-free variant of EDHOC. KEMEDHOC utilizes KEM for exchanging ephemeral keys and authenticating parties. Addressing **[RG1]**.
- We formally verify the design of KEMEDHOC using ProVerif [Bla13]. Our analysis shows that KEMEDHOC guarantees security properties required for a lightweight key exchange protocol, specified in this IETF draft [Vuč+20], along with quantum-resistance and DoS mitigation. Addressing **[RG1]** and **[RG2]**.
- We present a verified implementation of EDHOC protocol, called EDHOC*, ensuring the functional correctness, memory safety, and resillience to timing side-channel attacks. To achieve this goal, we utilize the F* ecosystem [Swa+16; Pro+17] to develop and propagate formal guarantees from design level to implementation level. Addressing **[RG2]**.

- On top of EDHOC*, we develop a verified implementation of KEMEDHOC, called KEMEDHOC*, which is also provably secure against memory bugs and timing side-channel leakages. Addressing **[RG2]**.

**Summary.** We present the first verified implementation of EDHOC, called EDHOC*. In addition, we provide the formally verified design and implementation of a quantum-safe variant of EDHOC, called KEMEDHOC and KEMEDHOC* respectively. This work aims to provide a high-assurance, end-to-end formal verification of EDHOC and its quantum-safe, and signature-free, variant, KEMEDHOC, in both design and implementation levels. Illustratively, we aim to tick the three empty boxes in Table 1 at the end of this work. All the code and formal models are publicly accessible at: https://github.com/ancuongnguyen07/kemedhoc.

| | Verified Specification | Verified Implementation |
|---|---|---|
| EDHOC | ✓ [Jac+23; GM23; CP22; Fer24] | □ |
| KEMEDHOC | □ | □ |

✓: verified in the cited works □: not verified

**Table 1:** Verification status of EDHOC and KEMEDHOC before this thesis project.

## 1.7 Limitations

This work still presents several **Limitations (Ls)** that should be acknowledged.

**[L1] Lack of computational security analysis of KEMEDHOC design.** The formal verification of KEMEDHOC design is conducted in the symbolic model using ProVerif [Bla13]. Although the symbolic model is widely used in the analysis of security protocols and has been shown to be effective in identifying vulnerabilities, it abstracts away certain cryptographic details and may not capture all possible attack vectors. A more rigorous computational security analysis would provide stronger assurances about the security of KEMEDHOC design against a broader range of adversarial capabilities, especially quantum threats.

**[L2] Absence of formally verified CBOR encoding/decoding and C.509 certificate parsing routines.** The current implementations of EDHOC* and KEMEDHOC* do not include formally verified CBOR encoding/decoding routines and C.509 certificate parsing. Encoding and parsing components are critical for the correct operation of security protocols, and any vulnerabilities in these areas could compromise the overall security [Ian21; Geo+12]. The current implementations rely on unverified libraries for these functionalities, which may introduce potential security risks.

**[L3] Need for machine-checkable verification between ProVerif model and F* specification.** While both the ProVerif model and the F* specification of KEMEDHOC have been verified independently, there is currently no machine-checkable proof establishing a formal correspondence between the two. Such a proof would ensure that the implementation in F* accurately reflects the security properties analyzed in the ProVerif model. Currently, this correspondence is based on our thoughtful mapping and manual inspection, which may be prone to human error.

**[L4] Insufficient comprehensive benchmarking on resource-constrained settings.** The performance evaluation of EDHOC* and KEMEDHOC* has been conducted primarily on a commercial laptop, which may not accurately represent the performance characteristics on resource-constrained devices, such as microcontrollers and embedded systems. Additionally, the protocols have not been extensively tested in real-world low-bandwidth communication settings. Such environments introduces more challenges, including packet loss, latency, and limited throughput, which could significantly impact the performance and reliability of the protocols.

Despite these limitations, the work presented in this thesis represents a significant step towards high-assurance, quantum-safe key exchange protocols for constrained environments. Furthermore, these limitations provide clear directions for future research and development efforts which are discussed in Section 8.

## 1.8 Related Work

**Post-quantum transition with KEM.** Post-quantum transition of security key agreement protocols has been an emerging research topic in the past few years, especially after NIST PQC standardization [SN17] has gained significant attention. Several works have explored the integration of post-quantum cryptographic primitives into existing protocols. For instance, Thom at el. [SSW20] proposes a KEM-based approach to replace traditional signature-based authentication in TLS. Similarly, efforts have been made to adapt protocols like Noise [Ang+22], WireGuard [Hül+21], and 5G AKA [Dam+22] to post-quantum settings. However, these protocols are not specifically designed for constrained environments, their main use cases is to secure general-purpose communication channels, which undermines the overhead concerns in low-power and low-bandwidth settings.

Regarding to KEM-based lightweight protocols, there is a recent work on post-quantum key establishment which is particularly designed for minimizing overhead in communication between power-limited devices, namely PQuAKE [Blu+25]. PQuAKE can operate on top of existing protocols, such as EAP [Vol+04], and IKEv2 [Ero+10] to provide post-quantum key agreement. Elaborately, PQuAKE attains its minimal overhead by relying on a single signature that is generated out-of-band, rather than during the protocol execution, thereby reducing the number of signatures exchanged between the parties [Blu+25]. The overall architecture of PQuAKE is based on the MQV protocol [Law+03], and the implicit authentication is achieved by utilizing the pre-shared symmetric key in key derivation and certificate encryption. In contrast,

KEMEDHOC does not require a pre-shared key between two parties to provide the implicit authentication. At the time of writing, PQuAKE is at the very first draft and its formal security analysis is not yet available.

**Post-quantum transition with hybrid cryptography.** Besides the KEM-based approach, the hybrid framework [GCR23], in which post-quantum and pre-quantum cryptographic primitives are combined to provide security even in the case of one of the primitives being broken, is also studied and experimented in the context of TLS [SFG25], and Secure Shell (SSH) [KSH25; QC25]. The same approach is also applicable to EDHOC. Yadav et al. [YSB25] introduces a hybrid mode in which ephemeral keys and signatures are the products of both classical and post-quantum cryptographic primitives. This hybrid design introduces large overhead in communication cost since both classical and post-quantum materials need to be transmitted, making it resource-constrained settings. A lightweight version of hybrid EDHOC is proposed with the risk of losing *Forward Secrecy* [YSB25].

**Formal verification at design-level of EDHOC.** Formal verification of a protocol is a process of mathematically proving that the protocol is secure against a set of predefined security properties, such as confidentiality, integrity, and authentication. Before TLS 1.3, this security analysis was conducted sequentially, after the protocol design had been finalized and widely-deployed, causing catastrophic consequences and costly attacks, such as DROWN [Avi+16], Lucky 13 [AP13], and KCI [Hla+15] on predecessors of TLS 1.3.

Nonetheless, with the increasing benefits and recognized importance of formal verification, especially in the landscape of security protocols, formal methods have been integrated into the standardization process from the very beginning, leading to earlier detection of design flaws and efficient resolution of security issues.

EDHOC is one of the protocols that has been extensively verified and analyzed in parallel with its standardization process. At the very first draft (draft 0), EDHOC was formally verified and the issues of injective agreement and unclear identity binding were discovered [NSB20]. In the draft 12, attacks on confidentiality, such transcript collision and final key resues were found and addressed in the draft 14 [Jac+23]. In the draft 14, issues of identity misbinding and HKDF key reuse were brought to light and resolved in the draft 17 [GM23]. Additionally, there was a salt collision attack, leading to the modification in key derivation architecture proposed in the following draft 16 [CP22].

Finally, the analysis of the draft 23, which is the last draft before EDHOC was standardized in RFC 9528 [SMP24], suggested that identity protection can be hardened by using padding before encryption, particularly with external authorization data [Fer24]. To conclude, the formal verification of EDHOC has been conducted in both symbolic and computational models, covering all authentication methods, increasing the assurance of the protocol design even before its adoption in the real-world applications. Nonetheless, the abovementioned works only focus on the design-level security analysis, leaving the implementation-level security verification unexplored.

**Verified protocol implementations.** There have been a long line of works on translating the formal specification of security protocols into verified executable code, ensuring that the implementation is correct with respect to the specification. More specifically, prior works have constructed verified compilers that automatically extract high-level protocol specifications into implemenations in OCaml [CB15; McC+07], and F# [FGR09; Cor+08]. However, these automatic tools do not produce performant code and still rely on unverified cryptographic libraries under the hood. On the other hand, the F* ecosystem, together with KaRaMel compiler, is capable of producing high-performance C code which is formally verified for correctness and memory safety, and leverages the verified cryptographic library HACL*.

In recent years, several works have utilized F* ecosystem to build verified implementations of security protocols, such as TLS [BFK16; BBK17], QUIC [Del+21], DICE [Tao+21], Signal [Pro+19], and Noise [Ho+22]. These studies successfully show that using formal verification ensures implementation-level security, delivering protocol implementations that are trustworthy and performant. However, as the time of writing, there is no verified implementation of a quantum-safe security protocol, leaving a critical gap in the landscape of high-assurance cryptographic software.

## 1.9 Concurrent and Independent Work

**Signature-free post-quantum EDHOC.** During the course of this thesis project, Fraile et al. [Fra+25b; Fra+25a] publish two quantum-safe variants of EDHOC that are based on KEM for both key exchange and authentication. We temporarily designate the two drafts as KEM-5 EDHOC [Fra+25a] and KEM-3 EDHOC [Fra+25b] respectively in this thesis since the authors do not provide specific protocol names.

The KEM-5 EDHOC [Fra+25a] introduces a KEM-based authentication method with 5 mandatory messages in the scenario where both parties do not know the credentials of each other beforehand. Briefly, the protocol requires parties' credentials, e.g. certificates, to be exchanged in Message 2 and 3, allowing each party to verify the communicating peer's identity and retrieve the public key for KEM encapsulation. The Responder sends its credential in Message 2 even before verifying the Initiator's identity, leaking identity of Responder to an *active* adversary. On the other hand, KEMEDHOC, which targets the scenario where both parties know each other's credentials beforehand through pre-provisioning, requires only 3 mandatory messages. In KEMEDHOC, the Responder and Initiator send their credentials in Message 1 and 2 respectively, after verifying the peer's identity, thus protecting the identity of both parties from an *active* attacker.

The second draft, KEM-3 EDHOC [Fra+25b], introduces a KEM-based authentication method with 3 mandatory messages, targeting the same scenario as KEMEDHOC. Note that the scenario where both parties know each other's credentials beforehand is the typical use case of EDHOC since resources-constrained devices, such as IoT sensors, usually are pre-provisioned at on-boarding time or manufacturing time. The protocol flow of KEM-3 EDHOC is the same as KEMEDHOC. However, KEM-3 EDHOC employs the binary keystream XOR operation to encrypt plaintext 1, while KEMEDHOC utilizes AEAD encryption for both confidentiality and integrity. By

using AEAD scheme, KEMEDHOC ensures that any tampering with the encrypted plaintext 1 will be detected during decryption, while KEM-3 EDHOC needs to check the $\texttt{MAC}_2$ in plaintext 2 to verify the cumulative transcript, which is less efficient.

Both drafts do major changes to the key derivation architecture of EDHOC and do not provide formal verification of their designs, in symbolic or computational models. Hence, their security properties are not yet guaranteed. On the other hand, KEMEDHOC follows the key derivation architecture of EDHOC as much as possible, thus inheriting the security properties analyzed in prior works [CP22; Fer24; GM23]. In addition, KEMEDHOC is formally verified in the symbolic model using ProVerif [Bla13], ensuring that the design is indeed secure as claimed. Furthermore, we provide a reference implementation of KEMEDHOC that is formally verified at machine-byte level, ensuring the functional correctness, memory safety, and timing side-channel resistance.

**PQ migration with quantum-safe signature.** In the signature-based authentication modes of EDHOC (method 0), the transition to PQ cryptography can be conducted simply by replacing the DH-based key exchange with a quantum-safe KEM and the legacy signature scheme with a quantum-safe signature scheme. PQ-EDHOC [Fra+25c], which is an instance of this approach, demonstrates that ML-KEM-512 and FALCON1 are the most suitable candidates for KEM and signature scheme respectively due to their small sizes and reasonable performance. However, the considerable overhead of FALCON1 signature and code footprint is still a concern in constrained settings. In contrast, KEMEDHOC eliminates the need for signatures and achieves implicit authentication using KEM, resulting in a more efficient protocol in interms of both message size and codebase footprint. A detailed comparison between KEMEDHOC and PQ-EDHOC is presented in Section 7.

# 2 Preliminaries

**Notations.** The notation $\mathtt{m} \xleftarrow{\$} \mathcal{M}$ means that $\mathtt{x}$ is sampled uniformly at random over the set $\mathcal{M}$. $\mathtt{y} \xleftarrow{\$} \mathcal{A}(\mathtt{x})$ indicates that $\mathtt{y}$ is the output of a probabilistic algorithm $\mathcal{A}$, given input $\mathtt{x}$ and uniformly random coins. $\mathtt{y} \leftarrow \mathcal{A}(\mathtt{x})$ denotes that $\mathtt{y}$ is the output of a deterministic $\mathcal{A}$ with input $\mathtt{x}$.

Let $(\mathbb{G}, +)$ be a cyclic group of prime order $\mathtt{q}$, generated by the generator $\mathtt{G}$. The group $\mathbb{G}$ is defined as the set $\{\mathtt{xG} : \mathtt{x} \in \mathbb{Z}_\mathtt{q}\}$. Within this structure, the shared secret $\mathtt{S} \leftarrow \mathtt{xY}$ is computed through the a scalar $\mathtt{x} \in \mathbb{Z}_\mathtt{q}$ and a group element $\mathtt{Y} \in \mathbb{G}$ .

Let $\mathtt{H}$ be a collision-resistant hash function, denoted as $\mathtt{H} : \{\mathtt{0},\mathtt{1}\}^* \rightarrow \{\mathtt{0},\mathtt{1}\}^{256}$. The Hash-based Key Derivation Function (HKDF) [Kra10] has two functions, namely `Extract` and `Expand`, which are used to derive pseudorandom keys. The function `Extract(salt, IKM)` extracts a pseudorandom key from the Input Key Material (`IKM`) using a `salt` value. The function `Expand(PRK, info, L)` expands the pseudorandom key `PRK` into a key of `L` bytes, using `info` as additional context information. $|\mathtt{B}|$ indicates the length of a byte string `B` in bytes, while $|\mathtt{Hash}|$ denotes the length of a hash output in bytes, e.g., $|\mathtt{Hash}| = 32$ for SHA-256 hash function. $\varepsilon$ represents an empty string.

## 2.1 EDHOC

EDHOC is a lightweight authenticated key exchange protocol that establishes a shared key for secure communication between low-power devices, typically microcontrollers operating with 10's of kB of random-access memory (RAM) and 100's of kB of code memory, and at 10's of MHz [Hög+24]. Its compact messages and minimal processing overhead are ideal for the Internet-of-Things (IoT) landscape in which low-bandwidth environments, such as LoRaWAN [Hax+18], LPWAN [CZB20], interoperate and exchange packets. Although EDHOC is mainly designed for establishing a security context for the Object Security for Constrained RESTful Environments (OSCORE) [Sel+19], an application-layer security protocol for IoT devices, it is interoperable with other application-layer protocols and applications, such as HTTP/HTTPS. The OSCORE protocol is especially designed to protect the Constrained Application Protocol (CoAP) [SHB14] and CoAP-mappable HTTP packets, ensuring confidentiality, integrity, and authentication for messages exchanged between devices in a lightweight and efficient manner [Sel+19].

In order to have a more precise use case of EDHOC and complementary lightweight security protocols, we describe an industrial scenario to illustrate their roles and advantages. We consider the scenario of a smart greenhouse at a rural area. In a large-scale automated greenhouse, thousands of plants are grown in a controlled environment to optimize yield and quality. The greenhouse is equipped with a variety of IoT sensors and actuators to monitor and regulate environmental conditions. These devices must operate efficiently while ensuring secure communication to prevent cyber threats that could impact productivity and plant health. Moreover, the greenhouse is divided into 50 sections, each dedicated to various types of crops, and each contains:

- Environmental Sensors: meaure temperature, humidity, $CO_2$ intensity, soil moisture, and light intensity.
- Irrigation Actuators: control water flow and nutrient dosing based on sensors' data, minimizing waste and ensuring optimal plant growth.
- Ventilation and Shading Controllers: adjust fans, vents, and shading panels to regulate weather.
- Wireless Transceivers: publish sensor data and receive commands from a central hub. These transceivers serve as the logistic backbone for the IoT network within the smart greenhouse.

All devices are battery-powered and connect wirelessly with a local gateway, which communicates to a central cloud-hosted management system. The control software processes data, fine-tune climate parameters, and optimizes irrigation schedules for each crop. Moreover, through the central management system, farmers can monitor real-time data, receive alerts, and remotely control greenhouse operations via a web interface or mobile app.

One of the main challenges of this setup is the avaiability. In the rural area, the Internet connectivity to the cloud can be intermittent. Therefore, the gateway should implement a cache mechanism so that the connection could be continued once the Internet is down. Furthermore, security issues is another critical concern. Wireless communication could be exposed to eavesdropping, DoS attacks, and unauthorized control commands, possibly resulting in severe consequences, such as, overwatering or underwatering crops, $CO_2$ levels manipulation leading to damaged plants. Additionally, since all IoT devices are low-powered, they require an energy-efficient protocol for secure communication to extend battery life while maintaining reliability and responsiveness.

The above scenario illustrates the need for lightweight, secure communication protocols like EDHOC and OSCORE. In particular, EDHOC can be used to establish secure keys between IoT devices, such as sensors and actuators, and the gateway, ensuring that data exchanged is encrypted and authenticated. OSCORE can then be used to protect CoAP messages exchanged between these devices and the gateway, using the established keys from EDHOC. This combination of protocols provides a robust security framework, an end-to-end security protocol, that is well-suited for the resource-constrained environment.

### 2.1.1 Protocol Overview

EDHOC consists of three mandatory messages and an additional optional message for explicit key confirmation. It is based on well-established protocol frameworks, such as Extract-and-Expand [KE10], the Noise XX pattern [Per18], and SIGMA [Kra03].

EDHOC defines four authentication methods, as presented in Table 2, which vary based on whether the `Initiator` and `Responder` perform authentication using signatures (SIG) or static Diffie-Hellman keys (STAT) [SMP24]. In addition, the

derivation of pseudorandom intermediate keys, $\texttt{PRK}_{\texttt{3e2m}}$ and $\texttt{PRK}_{\texttt{4e3m}}$, varies according to the authentication method.

**Credentials.** In order to save bandwidth, parties exchange credential identifiers `kid`, referencing to credentials stored on the other peer, instead of the credential payloads — C.509 [Mat+25], X.509 [Boe+08] certificate, or raw public key. Note that these identifiers does not need to be unique in the scope of the storing device:

> "Applications MUST NOT assume that 'kid' values are unique and several keys associated with a 'kid' may need to be checked before the correct one is found. Applications might use additional information such as 'kid context' or lower layers to determine which key to try first. Applications should strive to make ID_CRED_x as unique as possible, since the recipient may otherwise have to try several keys. [SMP24]"

Moreover, in typical settings, it is assumed that involving parties had already stored locally peers' credentials even before intitiating an EDHOC protocol session. For example, credentials could be pre-provisioned during the manufacturing process or exchanged through a secure out-of-band channel. On the other hand, EDHOC also supports credential delivery on-the-fly through protocol messages, but it increases message size significantly. Hence, it is recommended to use pre-provisioned credentials whenever possible to achieve the most compact message size.

| Method | `Initiator` Authentication Key | `Responder` Authentication Key |
|---|---|---|
| 0 | Signature Key (SIG) | Signature Key (SIG) |
| 1 | Signature Key (SIG) | Static DH Key (STAT) |
| 2 | Static DH Key (STAT) | Signature Key (SIG) |
| 3 | Static DH Key (STAT) | Static DH Key (STAT) |

**Table 2:** Authentication Keys for EDHOC Method Types.

### 2.1.2 Message Flow

EDHOC includes three mandatory messages ($\texttt{Msg}_1, \texttt{Msg}_2$, and $\texttt{Msg}_3$), an optional message for providing addtional key confirmation ($\texttt{Msg}_4$), and an error message if applicable [SMP24]. If there is no error occured, the shared session key is established for application data protection after the $\texttt{Msg}_3$. In general, EDHOC protocol is described as the following steps, also illustrated in Figure 2.

#### 2.1.2.1 Message 1

**Initiator composing Message 1.** Initiator sets up connection ID ($C_I$), authentication method METHOD (integer 0,1,2, or 3) and supported cipher suites. Then, Initiator randomly generates a private ephemeral DH key x and computes the public DH share $G_x \leftarrow xG$. Setup information and the public DH share $G_x$ is sent along with the optional external authorization data $EAD_1$.

**Responder processing Message 1.** Upon receiving the Message 1 ($Msg_1$), the Responder checks whether the cipher suite chosen by the Initiator, $C_I$, is supported. If not, it sends an error message *'Wrong selected cipher suite'* signaling that the Initiator has selected an unsupported cipher suite and it needs to re-run the protocol with a supported cipher suite.

#### 2.1.2.2 Message 2

**Responder composing Message 2.** Responder randomly produces an ephemeral DH secret y, forms the ephemeral DH public share $G_y \leftarrow yG$ and the shared secret $G_{xy} \leftarrow yG_x$. From the computed shared secret $G_{xy}$, Responder computes the transcript hash $TH_2 \leftarrow H(G_Y, H(Msg_1))$ and derives pseudorandom intermediate keys, $PRK_{2e}$ and $PRK_{3e2m}$. $PRK_{2e}$, an output of HKDF Extract function [Kra10], is the core secret value for the further keys and salt values to be derived.

To prove the authenticity, Responder computes $MAC_2$ using the derived key $PRK_{3e2m}$ regardless of the chosen authentication method. Then, it commits its authenticity through either $MAC_2$ or the signature $\sigma_2$ depending on the chosen authentication method. In particular, if METHOD is either 2 or 3, which are static DH-based, the computed $MAC_2$ is employed. If METHOD is 0 or 1, which are signature-based, then Responder produces the signature $\sigma_2$ over $MAC_2$ using the private key $sk_R$. Through the use of either $MAC_2$ or $\sigma_2$, the integrity of values ($C_R$, $ID\_CRED_R$, $TH_2$, $CRED_R$, $EAD_2$) is preserved.

The plaintext $PTX_2 := (C_R, ID\_CRED_R, EAD_2, [MAC_2 \text{ OR } \sigma_2])$, where $ID\_CRED_R$ is the credential identifier of Responder stored locally in Initiator and $EAD_2$ is an optional external authorization data, is encrypted by the binary additive stream cipher with the key stream $K_2$. The resulting ciphertext and the DH share $G_y$ are sent to Initiator as the Message 2 ($Msg_2$).

**Initiator processing Message 2.** Initiator computes the transcript hash $TH_2$ and derives the keystream $K_2$ as Responder does. Then, Initiator decrypts the received ciphertext $CTX_2$ to obtain the plaintext $PTX_2$. From the decrypted plaintext, Initiator obtains the credential identifier $ID\_CRED_R$ and identify the credential stored locally in Initiator that matches $ID\_CRED_R$. If Initiator cannot find a valid credential, then it returns an error message *'Unknown Credential Referenced'* and aborts the protocol session. If a matched credential is found, it continues verifying $\sigma_2$ or $MAC_2$. The verifying key for $\sigma_2$ is the public key $pk_R$ of Responder which is pre-provisioned in Initiator or obtained through a trusted third party. If either $\sigma_2$ or $MAC_2$ is invalid, then Initiator aborts the protocol.

#### 2.1.2.3 Message 3

**`Initiator` composing Message 3.** `Initiator` then derives pseudorandom intermediate keys $\texttt{PRK}_{\texttt{4e3m}}$ and $\texttt{PRK}_{\texttt{out}}$ and the symmetric encryption key $\texttt{K}_3$, given the transcript hash $\texttt{TH}_3 \leftarrow \texttt{H}(\texttt{TH}_2, \texttt{PTX}_2, \texttt{CRED}_\texttt{R})$. `Initiator` computes $\texttt{MAC}_3$ using the derived key $\texttt{PRK}_{\texttt{4e3m}}$ regardless of the chosen authentication method. Next, depending on the chosen authentication mode, `Initiator` commits its authenticity through either $\texttt{MAC}_3$ or the signature $\sigma_3$. Specifically, if `METHOD` is either 1 or 3, which are static DH-based, the computed $\texttt{MAC}_3$ is employed. If `METHOD` is 0 or 2, which are signature-based, then `Initiator` produces the signature $\sigma_3$ over $\texttt{MAC}_3$ using the private key $\texttt{sk}_\texttt{I}$. The array of values $(\texttt{ID\_CRED}_\texttt{I}, \texttt{TH}_3, \texttt{CRED}_\texttt{I}, \texttt{EAD}_3)$ is protected by either $\texttt{MAC}_3$ or $\sigma_3$ to guarantee its integrity.

The plaintext $\texttt{PTX}_3 := (\texttt{ID\_CRED}_\texttt{I}, \texttt{EAD}_3, [\texttt{MAC}_3 \texttt{ OR } \sigma_3])$, where $\texttt{ID\_CRED}_\texttt{I}$ is the credential identifier of `Initiator` stored locally in `Responder` and $\texttt{EAD}_3$ is an optional external authorization data, is encrypted using the AEAD encryption [Rog02] with the key $\texttt{K}_3$ and initial vector $\texttt{IV}_3 \leftarrow \texttt{KDF}\ (\texttt{PRK}_{\texttt{3e2m}}, 4, \texttt{TH}_3, \texttt{IV\_len})$ . The resulting ciphertext $\texttt{CTX}_3$ is sent to `Responder` as the Message 3 $\texttt{Msg}_3$. After sending $\texttt{Msg}_3$, the `Initiator` derives the session key $\texttt{PRK}_{\texttt{out}}$ and derives the application key $\texttt{PRK}_{\texttt{exporter}}$. The `Initiator` should not use the $\texttt{PRK}_{\texttt{out}}$ or $\texttt{PRK}_{\texttt{exporter}}$ until the `Responder` has been confirmed with the same key material, either through the optional Message 4 ($\texttt{Msg}_4$) or a message protected with $\texttt{PRK}_{\texttt{exporter}}$.

**`Responder` processing Message 3.** `Responder` derives the encryption key $\texttt{K}_3$ as the `Initiator` does. Then, `Responder` decrypts the authenticated ciphertext $\texttt{CTX}_3$ to obtain the plaintext $\texttt{PTX}_3$. If the AEAD tag for $\texttt{CTX}_3$ is invalid, then `Responder` returns an error message and aborts the protocol session. If the decryption is successful, then `Responder` obtains the credential identifier $\texttt{ID\_CRED}_\texttt{I}$ and identify the credential stored locally in `Responder` that matches $\texttt{ID\_CRED}_\texttt{I}$. If `Responder` cannot find a valid credential, then it returns an error message *'Unknown Credential Referenced'* and aborts the protocol session. If a valid credential is indentified, `Responder` verifies the signature $\sigma_3$ or $\texttt{MAC}_3$.

The verifying key for $\sigma_3$ is the public key $\texttt{pk}_\texttt{I}$ of `Initiator` which is pre-provisioned to `Responder`. If either $\sigma_3$ or $\texttt{MAC}_3$ is invalid, then `Responder` aborts the protocol session. If the authentication material is valid, then the `Responder` computes the session key $\texttt{PRK}_{\texttt{out}}$ and the application key $\texttt{PRK}_{\texttt{exporter}}$ as the `Initiator` does.

At this point, both parties should have the same session key $\texttt{PRK}_{\texttt{out}}$ and application key $\texttt{PRK}_{\texttt{exporter}}$ which can be used for further key derivation and application data protection. `Responder` should not use the $\texttt{PRK}_{\texttt{out}}$ or $\texttt{PRK}_{\texttt{exporter}}$ until `Initiator` has been confirmed with the same key material, either through the optional Message 4 or a message protected with $\texttt{PRK}_{\texttt{exporter}}$.

**Session key.** At the end of EDHOC protocol, both parties have established a shared session key $\texttt{PRK}_{\texttt{out}} \leftarrow \texttt{Expand}(\texttt{PRK}_{\texttt{4e3m}}, (7, \texttt{TH}_4, |\texttt{Hash}|), |\texttt{Hash}|)$ which is used for deriving further keys for application data protection. Application keys are derived using

the interface $\texttt{EDHOC}_{\texttt{exporter}}(\texttt{exporter\_label}, \texttt{context}, \texttt{L})$ and pseudorandom key $\texttt{PRK}_{\texttt{exporter}} \leftarrow \texttt{Expand}(\texttt{PRK}_{\texttt{out}}, (\texttt{10}, \varepsilon, |\texttt{Hash}|), |\texttt{Hash}|)$, where $\varepsilon$ denotes an empty string. The tuple (exporter_label, context) must be unique for each use case, and L is the desired length of the key (in bytes) defined by the application.

#### 2.1.2.4 Message 4 (optional)

**Responder composing Message 4.** This optional message serves for the explicit key confirmation goal. Responder derives AEAD encryption input materials, $\texttt{K}_4$ and $\texttt{IV}_4$, then encrypts an external authorization data $\texttt{PTX}_4 := \texttt{EAD}_4$ and sends the authenticated ciphertext to Initiator.

**Initiator processing Message 4.** Initiator derives $\texttt{K}_4$ and $\texttt{IV}_4$ in the same manner as Responder. Then, Initiator decrypts the received ciphertext $\texttt{CTX}_4$ to obtain the plaintext $\texttt{PTX}_4$. If the AEAD tag for $\texttt{CTX}_4$ is invalid, then Initiator returns an error message and aborts the protocol session. If the decryption is successful, Initiator is able to assure that the Responder has the same pseudorandom key $\texttt{PRK}_{\texttt{4e3m}}$ and transcript hash $\texttt{TH}_4$, which are expanded into $\texttt{K}_4$, as it has computed. This leads to the explicit confirmation of the shared session key.

### 2.1.3 Key Schedule

HKDF Extract and Expand functions are at the heart of EDHOC key schedule deriving uniformly pseudorandom keys (PRKs), and other keying materials. In particular, the Extract function is utilized to generate the fixed-length pseudorandom intermediate keys, e.g., $\texttt{PRK}_{\texttt{2e}}$, and Expand function is used to generate other variant-length keys: encryption keys, e.g., $\texttt{K}_2$, initial vectors, e.g., $\texttt{IV}_2$, and MAC keys, e.g., $\texttt{MAC}_2$. EDHOC key schedule derives a distinct key for each intended purpose, with each derivation utilizing as much as possible information available at the time of derivation. If the static-ephemeral DH shared secret is available at each party, then the Extract function uses it as the IKM and previously derived PRK, if available, as the salt value to derive the next pseudorandom key.

In constrast, if the static-ephemeral DH shared secret is not available, especially in the case of SIG authentication method, then the previously derived PRKs are reused for the subsequent derivation stage. These PRKs then is input into the Expand function as the IKM along with transcript hash as the context information to generate the next set of keying material, including: a stream-cipher key ($\texttt{K}_2$) for encrypting $\texttt{PTX}_2$, MAC tags ($\texttt{MAC}_2, \texttt{MAC}_3$), AEAD encryption keys and initial vectors ($\texttt{K}_3/\texttt{IV}_3, \texttt{K}_4/\texttt{IV}_4$). The detail key schedule is illustrated in Figure 3.

**Lightweight Key Update.** In order to provides forward secrecy without the need to re-run EDHOC protocol, EDHOC supports an optional function called EDHOC_KeyUpdate(context) which allows the parties to update the session key

material based on the current $\texttt{PRK}_{\texttt{out}}$ and $\texttt{PRK}_{\texttt{exporter}}$. New $\texttt{PRK}'_{\texttt{out}}$ and a newly derived $\texttt{PRK}'_{\texttt{exporter}}$ are computed as follows:

$$\texttt{PRK}'_{\texttt{out}} \leftarrow \texttt{Expand}(\texttt{PRK}_{\texttt{out}}, (11, \texttt{context}, |\texttt{Hash}|), |\texttt{Hash}|),$$
$$\texttt{PRK}'_{\texttt{exporter}} \leftarrow \texttt{Expand}(\texttt{PRK}'_{\texttt{out}}, (10, \varepsilon, |\texttt{Hash}|), |\texttt{Hash}|).$$

The `context` is a unique identifier for the key update procedure and may be represented, for example, by a counter, a random value, or a timestamp. The old $\texttt{PRK}_{\texttt{out}}$ and keys derived from it which are no longer needed must be securely deleted to ensure forward secrecy. Although this key update mechanism provides forward secrecy in an efficient way, it does not offer security properties as robust as those achived by re-executing EDHOC protocol with fresh ephemeral keys. Note that compromising the $\texttt{PRK}_{\texttt{out}}$ results in the compromise of all keys derived from it since the last invocation of `EDHOC_KeyUpdate(context)` .

**Transcript Hashes.** Transcript hashes ($\texttt{TH}_2, \texttt{TH}_3, \texttt{TH}_4$) used throughout EDHOC protocol are summarized in the following table Table 3.

| Transcript Hash | Definition |
|---|---|
| $\texttt{TH}_2$ | $\texttt{H}(\texttt{G}_\texttt{Y}, \texttt{H}(\texttt{Msg}_1))$ |
| $\texttt{TH}_3$ | $\texttt{H}(\texttt{TH}_2, \texttt{PTX}_2, \texttt{CRED}_\texttt{R})$ |
| $\texttt{TH}_4$ | $\texttt{H}(\texttt{TH}_3, \texttt{PTX}_3, \texttt{CRED}_\texttt{I})$ |

**Table 3:** EDHOC Transcript Hashes.

**Pseudorandom Keys.** Pseudorandom keys and salts used throughout EDHOC protocol are summarized in the following table Table 4.

**MAC Tags.** MAC tags used throughout EDHOC protocol are summarized in the following table Table 5.

**Initial Vectors (IVs).** Initial vectors used throughout EDHOC protocol are summarized in the following table Table 6.

**Encryption Keys.** Encryption keys used throughout EDHOC protocol are summarized in the following table Table 7.

| **Key/Salt** | **SIG** | **STAT** |
|---|---|---|
| $\mathtt{PRK}_{2e}$ | | $\mathtt{Extract}(\mathtt{TH}_2, \mathtt{G}_{xy})$ |
| $\mathtt{SALT}_{3e2m}$ | N/A | $\mathtt{Expand}(\mathtt{PRK}_{2e}, (1, \mathtt{TH}_2, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{3e2m}$ | $\mathtt{PRK}_{2e}$ | $\mathtt{Extract}(\mathtt{SALT}_{3e2m}, \mathtt{G}_{iy})$ |
| $\mathtt{SALT}_{4e3m}$ | N/A | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (5, \mathtt{TH}_3, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{4e3m}$ | $\mathtt{PRK}_{3e2m}$ | $\mathtt{Extract}(\mathtt{SALT}_{4e3m}, \mathtt{G}_{rx})$ |
| $\mathtt{PRK}_{out}$ | | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (7, \mathtt{TH}_4, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{exporter}$ | | $\mathtt{Expand}(\mathtt{PRK}_{out}, (10, \varepsilon, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |

**Table 4:** EDHOC pseudorandom keys and salts in signature-based (SIG) and static-DH-based (STAT) modes. N/A indicates the parameter is not used in that mode. $\mathtt{PRK}_{2e}$, $\mathtt{PRK}_{out}$, and $\mathtt{PRK}_{exporter}$ are derived the same way regardless of the authentication method.

| MAC Tag | Definition |
|---|---|
| $\mathtt{MAC}_2$ | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (2, (\mathtt{C}_R, \mathtt{ID\_CRED}_R, \mathtt{TH}_2, \mathtt{CRED}_R, \mathtt{EAD}_2), \lvert\mathtt{MAC}_2\rvert), \lvert\mathtt{MAC}_2\rvert)$ |
| $\mathtt{MAC}_3$ | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (6, (\mathtt{ID\_CRED}_I, \mathtt{TH}_3, \mathtt{CRED}_I, \mathtt{EAD}_3), \lvert\mathtt{MAC}_3\rvert), \lvert\mathtt{MAC}_3\rvert)$ |

**Table 5:** EDHOC MAC Tags.

| IV | Definition |
|---|---|
| $\mathtt{IV}_3$ | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (4, \mathtt{TH}_3, \lvert\mathtt{IV}_3\rvert), \lvert\mathtt{IV}_3\rvert)$ |
| $\mathtt{IV}_4$ | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (9, \mathtt{TH}_4, \lvert\mathtt{IV}_4\rvert), \lvert\mathtt{IV}_4\rvert)$ |

**Table 6:** EDHOC Initial Vectors.

| Key | Definition |
|---|---|
| $\mathtt{K}_2$ | $\mathtt{Expand}(\mathtt{PRK}_{2e}, (0, \mathtt{TH}_2, \lvert\mathtt{PTX}_2\rvert), \lvert\mathtt{PTX}_2\rvert)$ |
| $\mathtt{K}_3$ | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (3, \mathtt{TH}_3, \lvert\mathtt{K}_3\rvert), \lvert\mathtt{K}_3\rvert)$ |
| $\mathtt{K}_4$ | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (8, \mathtt{TH}_4, \lvert\mathtt{K}_4\rvert), \lvert\mathtt{K}_4\rvert)$ |

**Table 7:** EDHOC Encryption Keys.

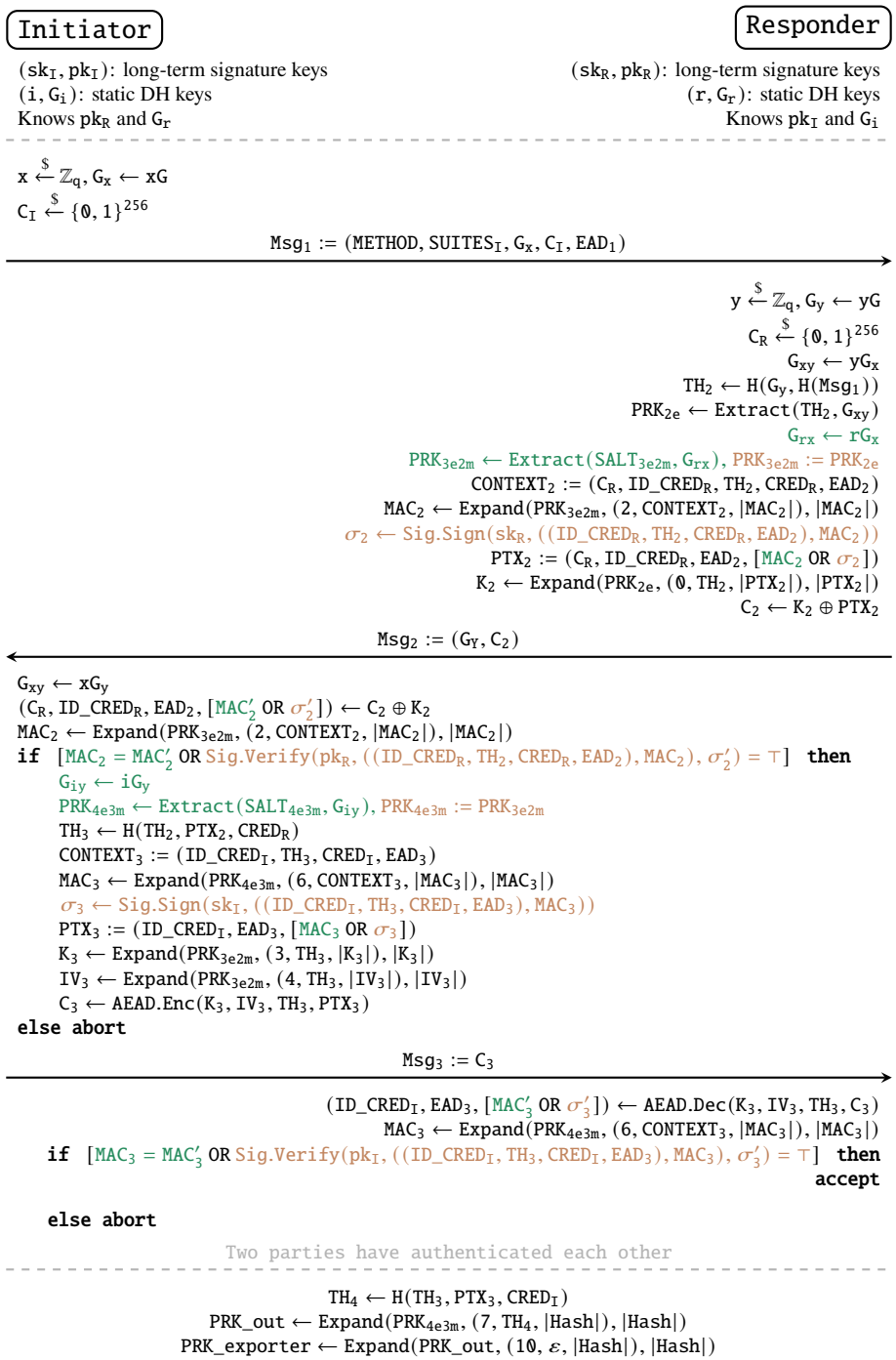



**Figure 2:** EDHOC protocol in four authentication methods. The optional Message 4 is omitted for simplicity. The green text represents steps that are performed only if the party uses static DH key (STAT), while the brown text indicates steps that are executed only if the party uses signature-based authentication (SIG). $\mathtt{MAC}_2/\mathtt{MAC}_3$ may appear both in black when computed in a common steps, and in green when used specifically for STAT methods.

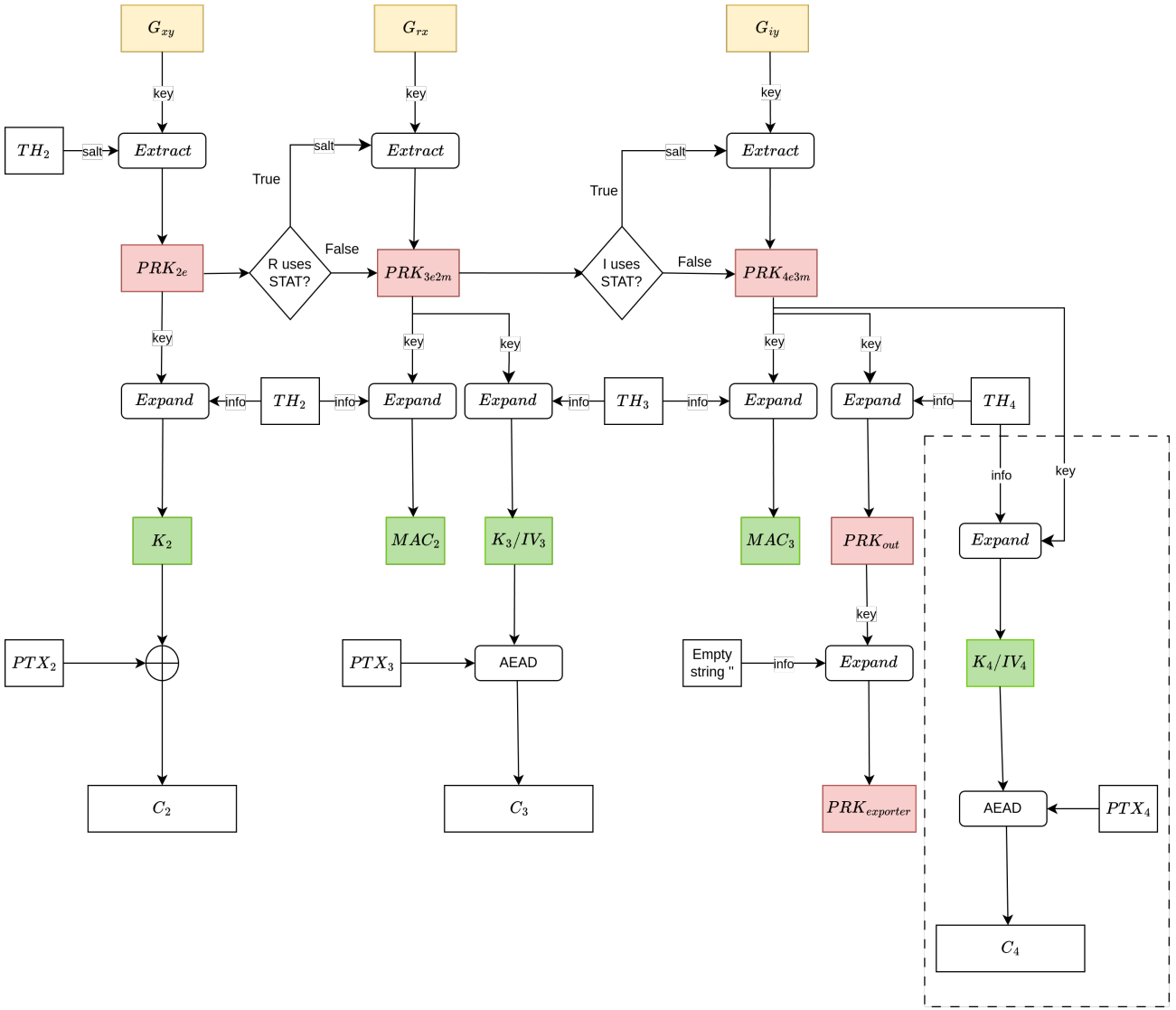



**Figure 3:** EDHOC key schedule. The yellow boxes denote DH-shared secrets ($\texttt{G}_{\texttt{xy}}, \texttt{G}_{\texttt{rx}}, \texttt{G}_{\texttt{iy}}$). The red boxes denote the intermediate keys ($\texttt{PRK}_{\texttt{2e}}, \texttt{PRK}_{\texttt{3e2m}}, \texttt{PRK}_{\texttt{4e3m}}, \texttt{PRK}_{\texttt{out}}, \texttt{PRK}_{\texttt{exporter}}$) and the green boxes denote MAC, encryption keys, and initial vectors ($\texttt{MAC}_2, \texttt{MAC}_3, \texttt{K}_2, \texttt{K}_3/\texttt{IV}_3, \texttt{K}_4/\texttt{IV}_4$). The operations which are within the dashed area are optional and only executed if the Message 4 ($\texttt{Msg}_4$) is sent to `Initiator`. Adapted from [Pér+24].

### 2.1.4 Security Properties

Given that EDHOC is designed to operate on top of OSCORE, it is required to adhere to the **Security Requirements (SRs)** defined in the Internet Draft "Requirements for a Lightweight AKE for OSCORE" [Vuč+20]. These security requirements are summarized as follows:

- **[SR1] Mutual Authentication:** Upon completion of the EDHOC session, both parties must successfully authenticate one another, ensuring that the shared key material is solely established between the intended and verified parties. Notably, both peers must reach agreement on a fresh session identifier, their respective roles, and associated credentials. In addition, the protocol should be *at least* resistant to the following attacks:
  - *Key Compromise Impersonation (KCI)*: an adversary not only impersonates a party that has compromised its long-term secret key but also impersonates the other uncompromised parties to the compromised party.
  - *Unknown key share (UKS)*: an adversary acts as a third party tricking the initiator into believing that it has shared a secret key with the responder, while the responder has completed the protocol session with the adversary instead. This attack causes the initiator and responder to hold mismatched perception of each other's identities, thereby violating the fundamental property of an authenticated key exchange protcol that ensures both parties agree on their peer's identity [SPA19].
  - *Selfie*, or *Reflection attack*: an attacker tricks a party into initiating a protocol session with itself, using the same credentials for both roles. This attack exploits the symmetry of the protocol, causing the party to believe it has established a secure session with another entity, while in reality, it is communicating with itself.
- **[SR2] Confidentiality:** The shared secrect key generated upon completion of the protocol must remain exclusively known to the two authenticated peers involved in the session. In particular, EDHOC satisfies the following security objectives:
  - *Session key secrecy*: the shared session key must be protected against eavesdropping, ensuring that an adversary cannot learn the key material even if it can observe the messages exchanged during the protocol run and compromise the long-term keys of a party.
  - *Session key independence*: if a shared key of a session is compromised, an attacker cannot learn shared keys of other sessions based on that already-compromised key.
  - *Session key uniqueness*: the shared session key must be unique, never repeated, across protocol sessions.

  - *Forward secrecy*: even if the ephemeral private key of a party is compromised, the confidentiality of the shared session keys in previous sessions must still be preserved.

- **[SR3] Identity Protection:** Identity, which is identifiable information that is used to authenticate and recognize a party, such as public keys, MAC addresses, or cryptographic certificates, must be protected against active attackers for one peer and passive attackers for the other peer. In particular, SIGMA-I [Kra03], a pattern that EDHOC is based on, safeguards the initiator's identity from active attackers and the responder's identity from passive attackers. In more detail, the `Initiator` sends its identity after it verified the identity of `Responder` in Message 3. On the other hand, the `Responder` sends its identity without any assurance of the `Initiator`'s identity in Message 2. As a result, the credential identifier of the `Responder` can be exposed to an active attacker who eavesdrops the destination address embedded in a legit Message 1 and sends its own Message 1 to the same destination.

- **[SR4] Cryptographic strength:** The protocol should employ cryptographic keys that provide a minimum security level of 128 bits, implying a brute-force search space of at least $2^{128}$. This level of security must be maintained across all aspects of the protocol, including authentication mechanisms, the agreed session key, and protection of cryptographic parameter negotiation.

- **[SR5] Protection of external security data:** The protocol should efficiently integrates external security applications by using External Authorization Data (EAD) message fields. Critically, these EAD fields must be protected at the same level as the protocol messages carrying them. In other words, if a message has an encrypted field, then the EAD value must be also encrypted with the same cryptographic strength. Otherwise, if a message is plain, then EAD values are publicly known.

- **[SR6] Downgrade protection:** In order to withstand long deployment lifetime, cryptographic agility is essential, so the protocol must support modularity and ability to negotiate preferred cryptographic algorithms. Downgrade protection is essential to prevent attackers from coercing a system into using weaker security parameters. Typical examples of such threats include cipher suite downgrade attacks, where an adversary manipulate the negotiation to use less secure choices, and key material downgrade attacks, in which the attacker intercepts key derivation to inject weaker or compromised keys. This protection is guaranteed in EDHOC by embedding ciphersuite selection into authenticated transcript hash chained up during the protocol session.

- **[SR7] Non-repudiation:** If a signature is used to authenticate a party, then the other party can prove the authenticated party has participated in the protocol session by showing the signature and signed data to a judge, a trusted third party. On the other hand, if static DH keys are used for authentication, then a party can deny that it has participated in the protocol session.

The abovementioned security objectives are not specific to EDHOC; they are derived from the general security requirements applicable to lightweight authenticated key exchange protocols designed to work with OSCORE.

## 2.2 KEM

Key Encapsulation Mechanism (KEM) is a cryptographic primitive that allows two parties to share a secret key over an insecure channel, as an alternative to Diffie-Hellman key exchange. According to [CS03; Den03], KEM is an asymmetric encryption scheme that produces a shared secret $\mathtt{K}$ in a key space $\mathcal{K}$. The KEM scheme is defined as a tuple consisting of the following three algorithms:

- **Key generation (probabilistic algorithm)** $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KEM.KeyGen}\left(1^{\lambda}\right)$: a probabilistic algorithm that takes a security paramter $\lambda$ and outputs a public/secret key pair $(\mathtt{pk}, \mathtt{sk})$ .
- **Encapsulation (probabilistic algorithm)** $(\mathtt{ct}, \mathtt{K}) \xleftarrow{\$} \mathtt{KEM.Encaps(pk)}$: a probabilistic algorithm that takes a public key $\mathtt{pk}$ and outputs a ciphertext $\mathtt{ct}$ and a shared secret $\mathtt{K} \in \mathcal{K}$.
- **Decapsulation (deterministic algorithm)** $\mathtt{K}/\perp \leftarrow \mathtt{KEM.Decaps(sk, ct)}$: a deterministic algorithm that takes a secret key $\mathtt{sk}$, a ciphertext $\mathtt{ct}$ and outputs a shared secret $\mathtt{K} \in \mathcal{K}$ or *rejection* $\perp \notin \mathcal{K}$ if the decapsulation fails.

**KEM Correctness [CS03].** Given every key pair $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KEM.KeyGen}(1^{\lambda})$ and every encapsulation $(\mathtt{ct}, \mathtt{K}) \xleftarrow{\$} \mathtt{KEM.Encaps(pk)}$, a KEM is $(1-\delta)$ *correct*, in the $\delta$-correct scheme, if it satisfies the following condition:

$$\mathtt{Pr}\left[\mathtt{K}' = \mathtt{K} | \mathtt{K}' \leftarrow \mathtt{KEM.Decaps(sk, ct)}\right] \geq 1 - \delta.$$

The condition means that the probability that the decapsulated key $\mathtt{K}'$ is equal to the encapsulated key $\mathtt{K}$ is at least $(1-\delta)$ for a negligible $\delta$. It is inferred that a KEM is *perfectly correct* if $\delta = \mathtt{0}$.

**Unauthenticated key exchange protocol.** A key exchange protocol can be built upon a KEM scheme $(\mathtt{KEM.KeyGen}, \mathtt{KEM.Encaps}, \mathtt{KEM.Decaps})$ to establish a shared secret between two communicating parties, as shown in Figure 4. Two involving parties are denoted as `Initiator (I)` and `Responder (R)` respectively. Firstly, `Initiator` generates a public/secret key pair $(\mathtt{pk}, \mathtt{sk})$ and sends the public key, or encapsulation key, $\mathtt{pk}$, to `Responder`. Then `Responder` encapsulates using $\mathtt{pk}$ to produce a ciphertext $\mathtt{ct}$ and a shared secret $\mathtt{K}$. Received $\mathtt{ct}$ from `Responder`, `Initiator` decapsulates using its secret key $\mathtt{sk}$ to reproduce the shared secret $\mathtt{K}$. If the protocol proceeds correctly, then at the end of the protocol session, parties have successfully established a shared secret key for further communication.

To explicitly confirm the integrity of the shared key, `Initiator` can send to `Responder` an additional message carrying the hash of the *decapsulated* `K` so that `Responder` is able to compare the received hash with the computed hash of the *encapsulated* `K`. Note that the KEM-based key exchange protocol described here is unauthenticated, meaning that it does not provide any authenticity of the parties involved in the protocol. Hence this schoolbook KEM-based key exchange is not secure from *active* adversaries, particularly is vulnerable to MitM attacks.

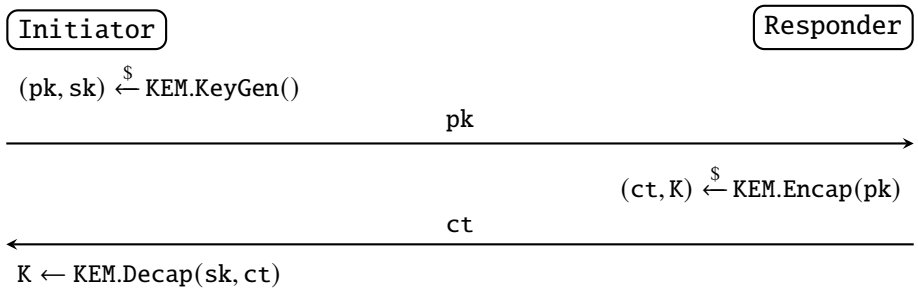



**Figure 4:** The unauthenticated key exchange protocol using KEM scheme.

**Authenticated key exchange protocol.** To address the security issues of unauthenticated key exchange protocol, an authenticated key exchange protocol uses both ephemeral and long-term KEM keys to derive a shared secret key. Figure 5 illustrates the the mutually authenticated key establishment protocol in which both involving parties, namely `Initiator` and `Responder`, know each other's static public keys, $\mathtt{pk}_\mathtt{I}$ and $\mathtt{pk}_\mathtt{R}$ respectively, before starting a communication session.

In addition to the ephemeral key $\mathtt{K}_\mathtt{e}$, in this authenticated settings, both parties also generate shared keys against the long-term authentication key pairs $(\mathtt{pk}_\mathtt{I}, \mathtt{sk}_\mathtt{I})$ and $(\mathtt{pk}_\mathtt{R}, \mathtt{sk}_\mathtt{R})$, resulting in the implicit, party-specific authentication keys $\mathtt{K\_auth}_\mathtt{t}$ and associated ciphertexts $\mathtt{ct\_auth}_\mathtt{t}$, where $\mathtt{t} \in \{\mathtt{I}, \mathtt{R}\}$.

At the end, the shared session key `SharedKey` is derived from not only the ephemeral key $\mathtt{K}_\mathtt{e}$ but also the long-term authentication keys $\mathtt{K\_auth}_\mathtt{I}$ and $\mathtt{K\_auth}_\mathtt{R}$ . In particular, a secure Key Derivation Function (KDF) takes the three key materials as input to derive the shared session key. HKDF, which produces cryptographically strong keys with variable length, is a suitable candidate for the KDF function. The correctness of the shared session key is guaranteed if and only if the three key materials are equal, or in other words, the cryptographic hash of them are equal. The result session key, derived from the shared ephemeral-static and ephemeral-ephemeral shared secrets, guarantees not only the confidentiality of data exchange, encrypted with the established session key, but also the authenticity of both involved parties through the long-term authentication keys.

$$\mathtt{SharedKey} \leftarrow \mathtt{KDF}(\mathtt{K}_\mathtt{e}, \mathtt{K\_auth}_\mathtt{I}, \mathtt{K\_auth}_\mathtt{R}).$$

The KEM-based authenticated key exchange protocol achieves the weak forward secrecy according to the Canetti-Krawczyk security model [CK01]. The forward

secrecy is defined as the property that the compromise of long-term keys does not compromise the security of past session keys. In other words, even if the static authentication keys are compromised at some point in the future from either party, the session keys established in the past are still secure.

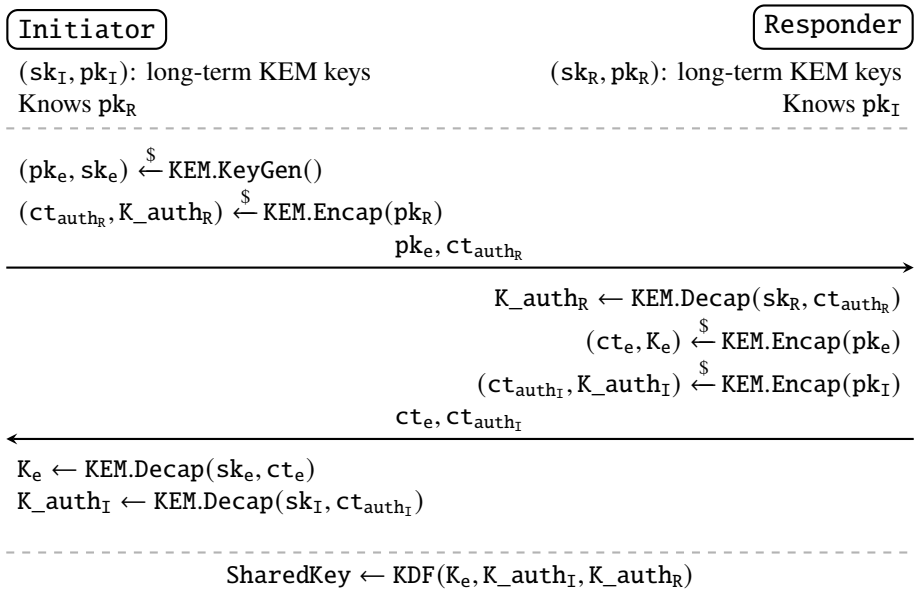



**Figure 5:** The mutually authenticated key exchange protocol using KEM scheme.

# 3 Automated Formal Verification for Cryptographic Protocols

This section provides theoretical background on applying formal methods to obtain formally proved security properties of a cryptographic protocol. In addition, the motivation behind the tools selected in this thesis is also discussed.

Formal verification involves mathematically proving that a system's formal model adheres to a set of properties described by a formal specification. This methodology is generally categorized into two distinct domains: specification verification and implementation verification. Specification (high-level) verification concerns abstract properties of the formal model, whereas implementation (low-level) verification focuses on the implementation's correctness with respect to the specification and various implementation-dependent properties.

In the context of cryptographic protocols, at the abstract level, formal verification is used to ensure that the protocols meet security properties such as integrity, confidentiality, and authentication. In addition, for a protocol implementation, it is essential to guarantee that the implementation correctly matches its high-level specification, and is secure against various of side-channel attacks.

**Cryptographic protocols are challenging to design and implement securely.** Since cryptographic protocols are inherently complex, even a subtle error in their design or implementation can lead to serious security vulnerabilities, including flaws in widely-deployed systems. For example, TLS, and its predecessor SSL, are widely used protocols for secure communication over the Internet, and have been subject to numerous vulnerabilities over the years, some of which were due to design flaws, while others were due to implementation bugs, which many of them expose to side-channel attacks [Avi+16; AP13; Hla+15; MDK14; BB03]. There are many reasons why crytographic protocols are challenging to be constructed correctly, including the complexity of the protocols, the difficulty of reasoning about security properties, and the priority of performance over security in many cases.

**Pen-and-paper proofs are error-prone and complicated for design-level verification.** Traditional formal analysis of cryptographic protocols, often done by pen-and-paper proofs, involves cryptographers and security experts constructing mathematical proofs and arguments manually. Although this approach has been the standard practice for establising security guarantees for new cryptographic protocols, manual arguments, which are error-prone and time-consuming, can lead to subtle flaws that are difficult to be detected. Furthermore, the manual proofs often depends on overly simplified models, "cryptographic cores", to make the analysis tractable, which can lead to the false sense of security and incomplete models where actual attacks lie outside the defined scope. Despite using methodologies designed to minimize the risk of human error, such as code-based-game-playing [BR06] and universal composability framework [Can01], these approaches are themselves complicated and error-prone in practice [Bar+21]. Due to the aforementioned challenges, there was a formal call for

more automated, scalable, and machine-checkable approaches to formal verification of cryptographic protocols among the security community [Hal05].

**Automated testing and code reviews are insufficient for implementation-level verification.** Regarding the implementation of cryptographic protocols, traditional approaches, such as code reviewing and testing, are insufficient for catching all subtle bugs and vulnerabilities, especially those that are related to side-channel attacks even if the constant-time principles are followed at the source-code level. The traditional landscape of cryptographic protocol verification, which heavily relies on manual proofs carefully crafted by experts and basic automation testing, simply cannot keep up with the increasing complexity of cryptographic protocols and the scale of their deployment in real-world systems. To overcome these difficulties in formal verification, computer-aided verification tools have been developed to automate the process for cryptographic protocols. This machine-checkable approach is more rigorous and scalable than traditional pen-and-paper proofs, helping to minimize human mistakes and accelerate the the verification process.

In order to drive the formal verification process towards more machine-checkable and less error-prone, a number of automated verification tools have been developed and applied into real-world security protocols, such as WireGuard [LBB19], Signal [KBB17], and Noise [KNB19]. The following sections introduce and discuss the core principles of two classes of verification, namely design-level and implementation-level, and state-of-the-art verification tools employed in this project.

## 3.1 Design-level Verification

Design-level security verification comprises the process of establising security guarantees for cryptographic mechanisms at the design phase. This process requires composing sophisticated building blocks, where abstract constructions form primitives, pritimives build protocols, and protocols construct systems. These established security proofs are a standard practice and essential for any newly proposed cryptographic design across all levels: primitives, protocols, and systems. There are two disctinct lines of models, which are symbolic models and computational models, reflecting the preferences of two research communities in formal method and cryptography respectively [Bar+21].

### 3.1.1 Symbolic Model

Symbolic model is an abstract model of cryptographic protocols that focuses on the message flows and the symbolic representation of cryptographic operations, such as encryption, decryption, and signing. In this model, cryptographic primitives are treated as idealized functions, perfect black-boxes, and their security properties are expressed in terms of symbolic relations between messages. In a formal language, messages containing values like keys or signatures are defined as atomic terms. These terms cannot be decomposed into smaller components like bitstrings [Bar+21]. Cryptographic computation over messages, or terms, is represented by constructors

and destructors. Constructors are used to create new terms, such as encryption and signing, while destructors are used to extract information from terms, such as decryption and signature verification. The relationship between constructors and destructors, representing properties of cryptographic primitives, is expressed by a set of mathematical equations known as equational theory, which defines how the terms can be manipulated and transformed [Bar+21]. For example, the equation for symmetric encryption and decryption is defined as follows:

$$\texttt{Decrypt}(\texttt{Encrypt}(\texttt{m}, \texttt{k}), \texttt{k}) = \texttt{m},$$

where m is a message, and k is a key. In other words, the above equation states that decrypting an encrypted message with the same key k returns the original message m.

Another example is the equation for deterministic signature verification:

$$\texttt{Verify}(\texttt{Sign}(\texttt{m}, \texttt{sk}), \texttt{m}, \texttt{pk}(\texttt{sk})) = \texttt{True},$$

where sk is a secret key, pk() is a symbolic function deriving a corresponding public key from the given secret key, and m is a message. In this equation, a signature of a message m with a secret key sk is valid if and only if the signature is verified with the corresponding public key pk(sk) derived from the secret key sk and the original message m.

The equational theory plays a crucial role in symbolic models, since it defines the properties of cryptographic primitives and abilities of the adversary in intervening the communication between honest parties. An adversary is typically modelled as a Dolev-Yao attacker [DY81], who is able to freely intercept, modify, and replay messages, yet unable to compromise the underlying cryptographic primitives. An attacker is only allowed to manipulate the terms, or enrich the knowledge of the terms, using specified primitives and equational theories.

Hence, the power of advesary, defined by how it can manipulate the terms through the equational theories, is overlooked if the equational theories are not properly defined. In the above example, where m, k, and sk are atomic terms, an attacker can only decrypt an encrypted message or generate a valid signature if it has access to the corresponding decryption key and signing key respectively, meaning that these key terms are available in the attacker's knowledge base. Due to the abstract nature of a symbolic model, which hides the computational complexity of underlying cryptographic primitives, and the ability to handle unbounded number of sessions, it is well-suited for reasoning about logical security properties and for the automated detection of logical vulnerabilities in cryptographic protocols.

Security properties in symbolic models are built upon two main concepts: trace properties and equivalence properties. Trace properties are concerned with the sequences of malicious events that never occur in any execution trace of the protocol, such as secrecy of a decryption key remains out of the attacker's knowledge base, or the integrity of a message is preserved during the protocol run.

Equivalence properties asserts that two protocols are indistinguishable from the adversary's viewpoint, meaning that the adversary cannot distinguish between the them, nor between their executions and an ideal security specification, based on the

messages exchanged during protocol execution. For instance, when reasoning about indistinguishablity-based secrecy, the equivalence property asserts that the adversary cannot differentiate between an execution trace containing the genuine secret and one containing a random value. This implies that the adversary gains no information about the secret from the messages exchanged during the protocol run [Bar+21].

The limitation of symbolic models is that they do not capture the computational aspects of cryptographic primitives, such as the hardness of breaking the cryptographic primitives, and the adversary's computational power. As a result, symbolic models are not suitable for reasoning about security properties that require computational assumptions, which are addressed by computational models.

Although the symbolic model is highly abstract and does not closely reflect the real-world attack scenarios, its abstraction facilitates the development of automated verification tools. Several such extensively-used tools exist, such as ProVerif [Bla13], Tamarin [BDS15], and DEEPSEC [CKR18].

### 3.1.2 Computational Model

Computational models are more realistic and closer to the real-world protocol implementations. In this model, messages are represented as bitstrings, and cryptographic primitives are modeled as probabilistic functions that operate on these bitstrings. For example, symmetric encryption and decryption are modeled as the triple algorithm (`KeyGen`, `Encrypt`, `Decrypt`), where `KeyGen` is a probabilistic algorithm that generates a key, `Encrypt` is encryption algorithm that takes a message and a key as input and returns a ciphertext, and `Decrypt` is a decryption algorithm that takes a ciphertext and a key as input and returns the original message. To reason about the correctness of the symmetric encryption scheme, the following equation must hold for all messages `m` and keys `k` generated by `KeyGen`:

$$\texttt{Decrypt}(\texttt{Encrypt}(\texttt{m}, \texttt{k}), \texttt{k}) = \texttt{m}.$$

Since the key is a bitstrings, the computational power required to break the encryption scheme is decreased if a bit of the key is leaked to the adversary, reflecting the real-world security property where the secure degree of a symmetric encryption scheme is determined by the length of the key. In computational models, adversaries are probabilistic Turing machines [Hal05].

There are two main frameworks for reasoning about security properties in computational models: the game-based framework [Sho04; BR06] and simulation-based framework [Bar+21].

In the game-based approach, security properties are modeled as interactive games between an adversary and an challenger. In this framework, the adversary's objective is to win the game by violating a specified security property, such as breaking the confidentiality of a secret key or forging a signature. The adversary's success probability is then used to define the security of the protocol. For instance, a symmetric encryption scheme is secure if the adversary's winning probability in the game of distinguishing between the encryption of a secret key and a random value is negligible, not exceeding a certain threshold.

The formal proofs of game-based security properties are powered by the game-hopping technique [Sho04], which allows to transform the game into a sequence of component games, each of which is easier to compute the success probability of the adversary, and the final game is the original game with the desired security property where the winning probability of the adversary can be unknown.

In other words, given a specific cryptographic security game in which the success probability of the adversary is unknown, the goal of game-hopping is to determine or bound that probability by step-by-step transforming the original game into a chain of intermediate games, where each game is easier to analyze, until the adversary's winning probability is known or bounded. Bounding the changes in success probability from game transformation is done by reducing to an assumed, well-studied, hard problem such as discreate logarithm problem or factoring problem [Bar+21].

The simulation-based framework, on the other hand, characterizes security through indistinguishability between real and ideal worlds. In this setting, the real world corresponds to the execution of the protocol in the presence of an adversary, while the ideal world represents an execution managed by a trusted party that provides the same functionality as the protocol. Informally, a protocol is considered secure if there exists a simulator capable of translating any real-world attack into an equivalent attack in the ideal world, which is, by definition, secure [Bar+21].

Simulation-based security proofs are more sophisticated than game-based proofs; however, they offer modularity enabling the analysis of security properties of complex protocols in a compositional manner.

While computational modeling is often labor-intensive, there has been a growing body of work developing automated tools, ranging from semi-automated to highly automated systems, to support the formal analysis of security protocols by designers and implementers. Among the widely used tools in the security community are CryptoVerif [Bla23] (highly automated), and EasyCrypt [Bar+11] (semi-automated).

In this thesis project, we verify KEMEDHOC protocol using the symbolic model rather than the computational model since the former model is more suitable for reasoning about logical security properties which are the main focus in the design phase. Regarding the gaps that symbolic models do not cover, such as computational assumptions, they are addressed partly by the implementation-level verification, which is discussed in the following section.

## 3.2 ProVerif

ProVerif [Bla13] is an automated symbolic protocol verifier based on the Dolev-Yao model [DY81], in which the attacker is presumed to possess complete control of the network—able to read, intercept, and modify messages during protocol execution. ProVerif, alongside Tamarin, ranks among the most widely-used tools for cryptographic protocol verification, demonstrated by its extensive application in both academic research and industrial projects, including verification of TLS 1.3 [BBK17], Noise [KNB19], and Signal [KBB17].

**Verification process overview.** The overall verification flow of ProVerif is illustrated in Figure 6. Initially, ProVerif inputs the protocol model and security properties to be analyzed. The protocol model is represented by pi calculus [AF01], constructors (destructors) and/or equational theories. Pi calculus is used to model the message flows, how processes interact with each other, and how the adversary intervenes the communication. Constructors and destructors are employed to abstract and model cryptographic primitives and their associated properties, which are represented through equational theories. Moreover, ProVerif takes security properties to be analyzed—such as secrecy, authenticity, and anonymity—which are expressed as trace properties.

Given those inputs, ProVerif backend translates the protocol model and trace properties into internal representations—specifically, Horn clauses [Hor51] and corresponding derivability queries based on those clauses. ProVerif then applies a resolution algorithm to determine whether a fact can be derived from the set of clauses. If such a fact is derivable, meaning it is available in the attacker's knowledge, the corresponding security property is violated. Otherwise, the analyzed property is provably true. The derivability of a fact links to a corresponding attack, but the discovered attack is potentially false due to the abstraction of Horn clauses.

When a false attack is discovered, it is not reliably clear that whether the analyzed protocol truly satisfies the queried security properties. Consequently, additional manual analysis is required. Owing to its abstraction based on Horn clauses, ProVerif is capable of handling an unbounded number of sessions, effectively reflecting the behaviour of real-world security protocols [Bla13].

**Pi calculus.** Pi calculus [AF01] is a process calculus that provides a formal framework for modelling and reasoning about concurrent systems in which the state of communication channels evolves and change during computation. In ProVerif, pi calculus is used to model message flows though input and output channels, protocoll sessions, local definitions (local terms or local variables), conditional branches with automatic error-handling. For example, `P|Q` models two processes, which can be two protocol runs or two stages of key derivation, running concurrently. `!P` denotes an infinite number of concurrent instances of the process `P` executing in parallel.

**Constructor and Destructor.** Constructor and destructor are the fundamental concepts within the process calculus used to model data structures and cryptographic primitives. Constructor is used to build terms, represented as patterns in Horn clauses. In ProVerif, terms refer to variables, atomic data (e.g., keys, nonces), and constructor applications [Bla13]. For instance, `senc(m,k)` refers to a symmetric encryption scheme that encrypts message `m` using the secret key `k`. By constrast, destructor is used to unwrap and manipulate terms. Destructors are partial functions applied to terms and defined by a set of rewrite rules. For example, `sdec(senc(m,k),k)` = m defines a symmetric decryption which decrypts the ciphertext generated by `senc(m,k)` using the secret key `k` to reproduce the original message `m`.

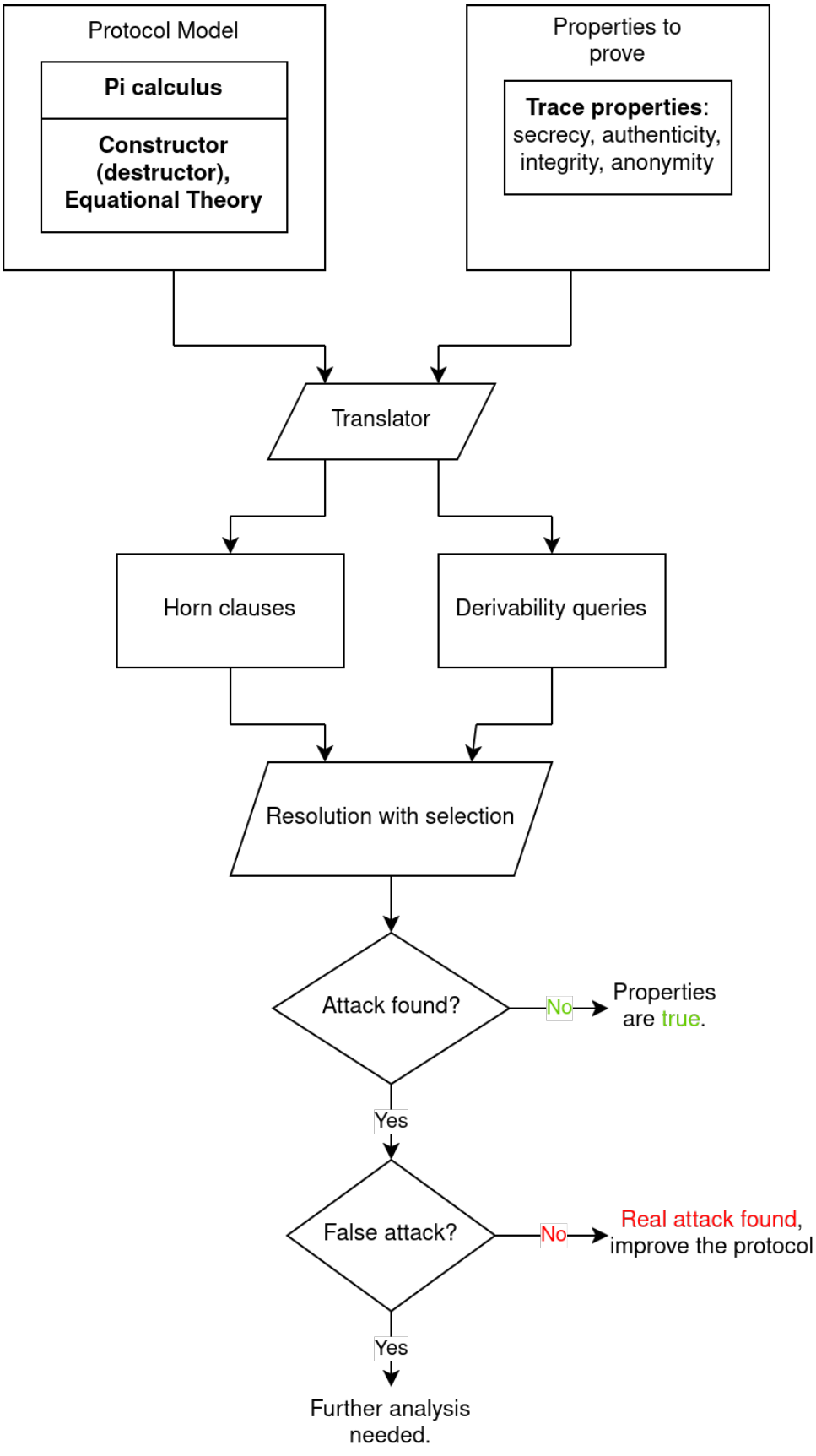



**Figure 6:** ProVerif verification flow, adapted from [Bla13].

**Trace property.** Trace property refers that a undesirable event, defined by the trace property, never arises in any execution traces. A execution trace is a set of computations or interactions conducted by an attacker during a particular protocol run. In ProVerif, trace properties are used to check the soundness of a security property. ProVerif employs the built-in predicate `attacker()` to reason about the secrecy of a particular term. For example, the secrecy of the symmetric key `k` is represented as `attacker(k)`, which means that `k` cannot be derivable from the collection of Horn clauses exposed to the attacker. If a real attack is found by ProVerif, meaning that the considered security property is violated, the execution trace, internally represented as a sequence of applied Horn clauses, leading to the attack will be reproduced and displayed to the user for further analysis. In addition to secrecy, authentication and other security properties can be modeled through correpondence and indistinguishability assertions.

**Horn clause [Hor51] and derivability queries.** Horn clause is the core internal representations of protocol model and adversary's capabilities in ProVerif. The primary unit of a Horn clause is the pattern which represents messages or terms. For example, a name is encoded as $\texttt{a}[\texttt{p1}, \ldots, \texttt{pn}]$, or, a constructor application is represented as $\texttt{f}(\texttt{p1}, \ldots, \texttt{pn})$. A fact which is used to assert a property about patterns is of the form $\texttt{pred}(\texttt{p1}, \ldots, \texttt{pn})$. A Horn clause is a construction of facts and logical rules. For instance, $\texttt{F1} \vee \cdots \vee \texttt{Fn} \Leftrightarrow \texttt{F}$ states that if there is exist a fact `Fi`, where `i` is ranging from 1 to `n`, is true, then the fact `F` is also true and vice versa. Regarding the adversary's capabilities, Horn clauses are used to specify how the adversary can derive terms by utlizing the defined constructors/destructors, equational theories, and terms exposed in public channels. For instance, the clause $\texttt{attacker}(\texttt{k}) \wedge \texttt{attacker}(\texttt{senc}(\texttt{m}, \texttt{k})) \Rightarrow \texttt{attacker}(\texttt{m})$ states that if an attacker posesses a secret key `k` and the ciphertext generated by $\texttt{senc}(\texttt{m}, \texttt{k})$, then the attacker can derive the original message `m` by utilizing the destructor $\texttt{sdec}(\texttt{senc}(\texttt{m}, \texttt{k}), \texttt{k}) = \text{m}$.

Internally, derivability queires are a mechanism asking the prover to prove a particular fact, e.g., $\texttt{attacker}(\texttt{m})$, not derivable from the established collection of Horn clauses. The internal process of translation into Horn clauses encompasses approximation, particularly the precise number of process repetitions is disregarded; therefore, Horn clauses are applicable unlimited number of times [Bla13]. Although this approximation is reasonable, which means the Horn clause model is also valid in a more precise model, such as applied pi calculus model, as long as it is correct in the approximated model, it can cause false attacks. These fault outputs are derived traces exist in Horn clause model but do not correspond to an execution trace of the original protocol model.

**Resolution algorithm.** Resolution algorithm comprises a set of steps utilized by ProVerif to determine whether a fact is logically derivable from a set of Horn clauses. Specifically, when two clauses, `R` and $\texttt{R}'$, are such that `R` unifies with a hypothesis of $\texttt{R}'$, a new clause can be inferred by the successive application of `R` and $\texttt{R}'$. As described in [Bla13], the formal definition of the resolution algorithm is presented as follows.

**Definition 1.** Let $R$ and $R'$ be two clauses such that $R = H \Rightarrow C$, and $R' = H' \Rightarrow C'$, with $H$ and $H'$ representing hypotheses, and $C$ and $C'$ representing conclusions. Suppose there exists a fact $F_\emptyset \in H'$ such that $C$ and $F_\emptyset$ are unifiable, and let $\sigma$ be the most general unifier of $C$ and $F_\emptyset$. In addition, $R \circ_{F_\emptyset} R' = \sigma(H \cup (H' \setminus \{F_\emptyset\})) \Rightarrow \sigma C'$. The clause $R \circ_{F_\emptyset} R'$ is obtained by resolving $R'$ with $R$ upon $F_\emptyset$; that is, it can be derived from $R$ and $R'$ as follows:

$$\frac{R = H \Rightarrow C \qquad R' = H' \Rightarrow C'}{R \circ_{F_\emptyset} R' = \sigma(H \cup (H' \setminus \{F_\emptyset\})) \Rightarrow \sigma C'}$$

Although the problem of checking derivability is exactly addressed by Prolog systems, such systems are not applicable to ProVerif's use cases since such systems do not guarantee the termination for unbounded number of complex terms introduced by the unlimited amount of protocol sessions [Bla13].

### 3.2.1 The selection of ProVerif for design-level verification

ProVerif and Tamarin are both widely used tools for formal verification of cryptographic protocols, but ProVerif is more suitable for this project due to its simplicity and fast verification process. ProVerif, based on Horn clauses, provides a higher degree of automation in reasoning many security properties, such as secrecy, and authentication, compared to Tamarin, which requires more user-defined lemmas. In prior works on formal verification of EDHOC [Jac+23], ProVerif is significantly faster than Tamarin, taking only a few minutes at most compared to several hours for Tamarin, to verify the same set of security properties.

On the down side, ProVerif does not support associative and commutative axioms, leading to the inability to model cryptrographic primitives such as XOR or Diffie-Hellman with their full algebraic properties as effectively as Tamarin does. However, Diffie-Hellman and XOR computation are not used in KEMEDHOC, eliminating the need for such algebraic properties in the verification process.

Another disadvantage of ProVerif is that it potentially produces false attacks, but they can be easily identified and removed by manually analyzing the resulting attack traces. Despite these limitations, ProVerif is still a powerful tool for formal verification of security protocols at the design level, where the focus is on reasoning about logical security properties rather than computational assumptions. Therefore, ProVerif is selected as the primary tool for the design-level verification of KEMEDHOC.

### 3.2.2 ProVerif Toy example

In order to help readers gain familarity with ProVerif's fundamental concepts and syntax, in this section, we present Toy key exchange protocol based on the example from ProVerif documentation [Bla+23] and discuss its formal model.

**Toy protocol.** The Toy key exchange protocol, illustrated in Figure 7, involves two parties, `Client` and `Server`, establising a shared secret which is initialized by `Client`. The Toy protocol consists of the following cryptographic primitives:

- $\sigma \xleftarrow{\$} \texttt{Sign}(\texttt{ssk}, \texttt{m})$: a digital signature scheme that takes signing key ssk and message m as inputs and generates a signature.
- $\texttt{m} \leftarrow \texttt{CheckSign}(\sigma, \texttt{spk})$: a signature verification scheme that takes signature $\sigma$, and verification key spk(ssk) as inputs and returns the message m if the signature is valid.
- $\texttt{c} \leftarrow \texttt{Aenc}(\texttt{pk}, \texttt{m})$: an asymmetric encryption scheme that takes public key pk and message m as inputs and produce ciphertext c
- $\texttt{m} \leftarrow \texttt{Adec}(\texttt{sk}, \texttt{Aenc}(\texttt{pk}, \texttt{m}))$: an asymmetic decryption scheme that takes secret key sk and ciphertext generated by Aenc as inputs and then reproduce the original message m.
- $\texttt{c} \leftarrow \texttt{Senc}(\texttt{k}, \texttt{m})$: a symmetric encryption scheme that takes secret key k and message m as inputs and generate ciphertext c.
- $\texttt{m} \leftarrow \texttt{Sdec}(\texttt{k}, \texttt{Senc}(\texttt{k}, \texttt{m}))$: a symmetric decryption scheme that takes secret key k and ciphertext generated by Senc as inputs and then reproduce the original message m.

The Toy protocol is supposed to provide the secrecy of the shared secret and mutual authentication between two involving parties. In other words, the secret s is only kwown to Client and the party Client intentionally communicates, Server. Moreover, a party can guarantee that the other communicating party is the honest party that is supposed to be involved in the protocol session.

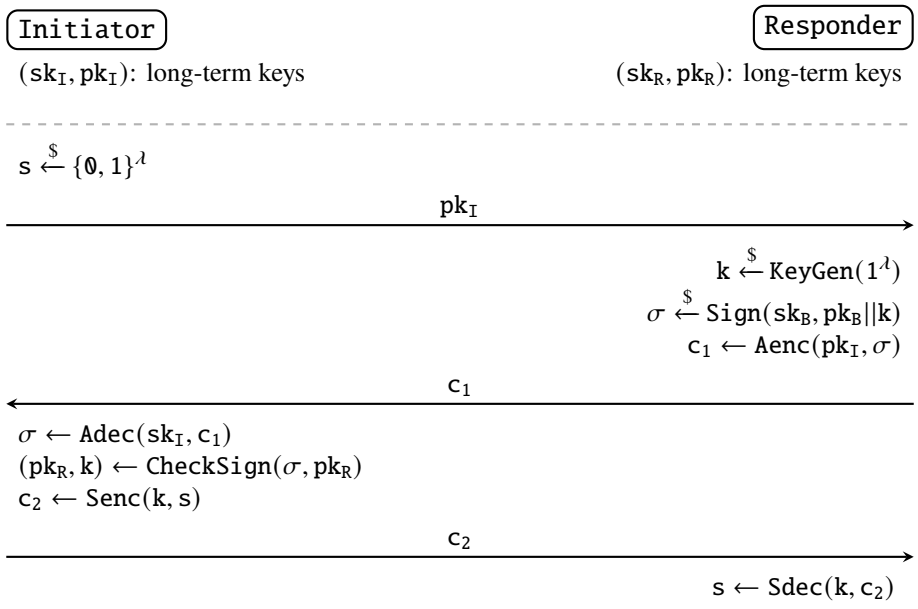



**Figure 7:** Toy key exchange protocol.

**ProVerif model of cryptographic primitives.** In the symbolic model, cryptographic primitives are perfect black-boxes, which means that their computational assumptions are ignored. Their correctness are defined by the constructor/destructor mechanism. For example, in order to model an asymmetric encryption scheme, we defines the following constructors and destructors:

- `pk(skey)`: a constructor deriving a corresponding public key `pkey` from the given `skey` type.
- `aenc(pkey, bitstrings)`: a constructor encrypting the given message of type `bitstring` using the public key `pkey`. The output is a ciphertext of type `bitstring`, where `bitstring` is an abstract type of machine bytes.
- `adec( sk , aenc(pk(skey), m) ) = m`: a destructor reconstructing the original message `m` encrypted by the constructor `aenc`.

The reduction rule, shown in Listing 1, for the asymmetric encryption scheme states: if anyone, including an attacker, has the secret key `sk` and ciphertext generated by `aenc`, where the public key is derived by `pk(sk)`, then he/she is able to recover the original message `m`. Precisely defining reduction rules is instrumental in modeling cryptographic primitives and characterizing attacker capabilities. All type and constructor definitions, along with the reduction rules, are detailed in Listing 2.

```
reduc forall m:bitstring, sk:skey; adec(sk, aenc(pk(sk), m)) = m.
```

**Listing 1:** Reduction rule for asymmetric encryption scheme [Bla+23].

```
(*Asymmetric encryption*)
type skey.
type pkey.
fun pk(skey):pkey.
fun aenc(pkey, bitstring):bitstring.

(*Symmetric encryption*)
type key.
fun senc(key, bitstring):bitstring.

(*Digital signature*)
type sskey.
type spkey.
fun spk(sskey):spkey.
fun sign(sskey, bitstring):bitstring.

(*Rewrite rules*)
reduc forall m:bitstring, sk:skey; adec(sk, aenc(pk(sk), m)) = m.
reduc forall m:bitstring, k:key; sdec(k, senc(k, m)) = m.
reduc forall m:bitstring, ssk:sskey; getmess(sign(ssk, m)) = m.
reduc forall m:bitstring, ssk:sskey; checksign(sign(ssk, m), spk(ssk)) = m.
```

**Listing 2:** ProVerif model of cryptographic primitives [Bla+23].

**Events and queries.** Events and queries are key concepts used to guide ProVerif verify security properties. ProVerif has the built-in secrecy-checking query, denoted as `query attacker()`, to verify if the given value is derivable by the attacker. For example, as shown in Listing 5, `query attacker(s)` asks the ProVerif to check if the secret `s`, which is not exposed to the attacker at the beginning, is derivable by the attacker. To verify the mutual authentication, we use a list of events (`acceptsServer(key, pkey)`, `acceptsClient(key)`, `termServer(key)`,

`termClient(key, pkey)`) and correspondence queries. For instance, the query shown in Listing 3 states: if then event `termClient(x, y)` has been invoked, then the event `acceptsServer(x, y)` had been triggered *before* with the same values of `x` and `y`. While `event` allows several occurences, **`inj-event`** only accepts one occurence, ensuring the one-to-one relationship between number of protocol runs conducted by each party [Bla+23]. The injective correspondence query shown in Listing 4 states that if the event `termServer(x)` is invoked, then the event `acceptsClient(x)` had been initiated *once before* with the same value `x`.

```
query x:key, y:pkey; event(termClient(x, y)) ==> event(acceptsServer(x, y)).
```

**Listing 3:** Correspondence query to verify the authenticity [Bla+23].

```
query x:key; inj-event(termServer(x)) ==> inj-event(acceptsClient(x)).
```

**Listing 4:** Injective query to verify the authenticity [Bla+23].

```
(*Events*)
event acceptsClient(key).
event acceptsServer(key, pkey).
event termClient(key, pkey).
event termServer(key).

free s:bitstring [private]. (*private shared secret, not publicly known*)
query attacker(s). (*Query to check secrecy*)

(*Correspondence queries to check authentication*)
query x:key, y:pkey; event(termClient(x, y)) ==> event(acceptsServer(x, y)).
query x:key; inj-event(termServer(x)) ==> inj-event(acceptsClient(x)).
```

**Listing 5:** Events and queries for secrecy and mutual authentication properties [Bla+23].

**ProVerif model of protocol processes.** This paragraph focuses on explaining the process of `Client`, named `clientA` in the model, and the main process, as illustrated in Listing 6. During the process `clientA`, the event `acceptsClient(k)` is prompted after the `Client` ensures that the received signature is valid and the decrypted public key is equal to the `Server`'s public key `pkB`. This event marks the receipt of symmetric key `k` from the client's side. At the end of the process `clientA`, the event `termClient(k, pkA)` is triggered to record that the client, who posesses the private key corresponding to the public key `pkA`, has finished the protocol run resulting in the symmetric key `k`. In the main process, public keys of `Client` and `Server` are exposed publicly as supposed, done by `out(internet, pkA)` and `out(internet, pkB)`. Furthermore, the parallel processes of both parties are simulated in unbounded number of sessions by utilizing the exclamation mark `!` prepended to the process name, e.g. `! clientA`.

```
(*Client process*)
let clientA(pkA:pkey, skA:skey, pkB:spkey)
    = out(internet, pkA); (*send 'pkA' to the public channel 'c'*)
    (*receive ciphertext1 'c1' as the respond*)
    in(internet, c1:bitstring);
    let signature_prime = adec(skA, c1) in (*decrypt 'c1'*)
    (*verify if the signature decrypted from 'c1' is valid,
    if YES then get 'pkB'' and the symmetric key 'k', if
    the parsed 'pkB'' is equal 'pkB' then continue otherwise
    terminate the current session.*)
    let (=pkB, k:key) = checksign(signature_prime, pkB) in
    (*Event: client has accepted the symmetric key 'k'*)
    event acceptsClient(k);
    (*encrypt the shared secret 's' and send to server*)
    out(internet, senc(k, s));
    (*Event: client has terminated/finished a protocol run
    resulting in the symmetric key 'k'. 'pkA' for authentication
    to the server *)
    event termClient(k, pkA).

(*Main process*)
process
    (*Generate secret keys for client and server*)
    new skA:skey;
    new skB:sskey;

    (*Generate public key for client and then
    make it public known*)
    let pkA = pk(skA) in
    out(internet, pkA);

    (*Generate public key for server and then
    make it public known*)
    let pkB = spk(skB) in
    out(internet, pkB);

    (*Run unbounded number of client and server processes
    in parallel*)
    ( (!clientA(pkA, skA, pkB)) | (!serverB(pkB, skB, pkA)) )
```

**Listing 6:** ProVerif model of the Toy protocol run [Bla+23].

**Verification results.** As shown in Listing 7, the third query is `true`, which means that the `Client` is able to authenticate itself to the `Server`. However, the second query is `false`, which means that the `Server` is not able to authenticate itself to the `Client`, not satisfying the property of mutual authentication. Regarding the secrecy property, the first query is `false`, which means that an attacker can derive the shared secret `s` from the defined constructors, destructors, and running processes.

In conclusion, the Toy protocol does not guarantee the secrecy and mutual authentication properties. Improving the security of Toy protocol is left as an exercise for readers. The full model of the Toy exchange protocol is available in Listing 6.

```
Query not attacker(s[]) is false.
Query event(termClient(x,y)) ==> event(acceptsServer(x,y)) is false.
Query inj-event(termServer(x)) ==> inj-event(acceptsClient(x)) is true.
```

**Listing 7:** Verification results of the Toy protocol.

## 3.3 Implementation-level Verification

Implementation-level verification advances security assurance by applying formal methods directly to the analysis and validation of cryptographic code. Even with a provably secure design, there is no guarantee that the corresponding implementation adheres to its specification or exhibits resistance to implementation flaws, such as memory vulnerabilities and side-channel leakages. These implementation-level

vulnerabilities are not addressed at the high-level specification stage, nor are they solely attributable to implementation errors or aggressive optimization. Consequently, attackers possess a broader attack surface at the implementation level compared to the design level. State-of-the-art automated tools, which supports low-level verification, is able to generate not only provably secure but also performant implementations.

At machine level, three core properties are typically verified: functional correctness, memory safety, and side-channel resillience. In order to prove the abovementioned fundamental properties, a Satisfiability Modulo Theories (SMT) [BT18] solver is often leveraged to effortlessly discharge verification conditions. Since implementation-level verification offers high-assurance security for verified code, it has been widely applied to real-world protocol implementations like TLS 1.3 [BBK17], Signal [Pro+19], Noise [Ho+22], and DICE [Tao+21].

Moreover, many formally verified implementations of cryptographic primitives have been deployed and integrated into worldwide services. For instance, a verified elliptic curve implementation from the Fiat Cryptography library [Erb+19; Goo17] is used in Google's BoringSSL, and HACL* [Zin+17], a verified cryptographic library, is part of Mozilla Firefox's NSS security engine [Cor17]. More recently, Apple has integrated the formally verified implementation of ML-KEM into CryptoKit cryptographic library, building a solid foundation for securing application data produced by milions users [Inc25].

### 3.3.1 Properties to prove

**Functional Correctness.** Functional correctness refers to the accuracy of translatation from high-level specification into low-level implementation. In other words, it ensures that the implementation accurately behaves as defined in its design specification, having the equivalent input/output behaviours. In general, there are two approaches to prove the functional correctness of a particular implementation: proving the equivalence between the reference implementation and the considered implementation, or demonstrating the equivalence between the functional specification and the implementation. The latter approach, proving through a functional specification, provides a higher assurance level and reduced base of trusted computing since it relies on a mathematically rigorous definition of behaviour rather than operational semantics of another executable code. Functional specification is represented as pairs of pre-conditions and post-conditions, where a pre-condition refers to the requirements of input and a post-condition specifies the expected states of output.

**Memory Safety.** Memory safety property refers to the resillience against the common memory-related vulnerabilities, such as buffer overflow, null pointer dereference, and use-after-free. The approach to model memory safety is specific to which level the verification is placed at, source code level or machine code level. For instance, Low* [Pro+17], which is a built-in subset of F* [Swa+16], introduces a memory model called `HyperStack` reasoning both stack and heap regions regarding their liveness, disjointness, and size.

**Side-channel Resistance.** Side-channel attacks are vulnerabilities that allow an attacker infer secret information based on the observed variation of physical behaviours of a system, such as execution time, power consumption, and eletromagnetic emanations. Among these side-effects, timing variations has showed many catastrophic exploits, even breaking widely-deployed softwares, such as remote timing attacks on OpenSSL library [BB03; BT11], or an attack on Advanced Encryption Standard (AES) implementation [Bog+10]. Formal reasoning about side-channel vulnerabilities relies on leakage models, which define the classes of information observable to an attacker during computation. For example, the program counter policy reveals control flow during execution, while the branch-operation model exposes values associated with branch conditions.

Once the leakage model is properly defined, an implementation is provably secure against side-channel attacks if the aggregated leakages during execution are uncorrelated with secret data values. Formally, this is known as observational non-interference: changes to secret inputs will not result in any observable changes in the program's output [BGL18]. According to [Bar+21], there are three widely used techniques to prove the independence of secret inputs:

- **Static analysis:** Uses type systems or data-flow analysis to monitor correlations between secret inputs and sensitive operations.
- **Quantitative analysis:** Builds formal models of hardware features (e.g., cache memory) to estimate upper bounds on leaked information.
- **Deductive verification:** Proves that leakage traces are indistinguishable when public inputs are the same, implying independence from secret inputs.

While static analysis allows for full automation, deductive verification typically requires manual assistance in complex scenarios. On the other hand, static analysis may lead to false positives due to its inability to ensure proof completeness, whereas deductive verification can achieve complete proofs with user's guidance.

**Translation from source-code level to machine-code level.** Formal verification at source-code level does not guarantee that the compiled executable remains secure due to the aggressive optimization applied by mainstream compilers. For instance, a time-variant binary can be compiled from the constant-time code as side-channel defense mechanisms are eliminated by the optimazation of mainstream compilers (GCC or Clang) [Kau+16]. To address this issue, formal analysis should be also performed at machine-code level. Tools such as Vale [Bon+17] and Jasmin [Alm+17] enable formal reasoning directly over assembly code. An alternative approach to obtaining verified machine code, without the complexity of writing low-level code manually, is to use a verified compiler. Such a compiler ensures that the generated machine code is provably correct with respect to the formally verified source code. CompCert [Ler09] is a prominent example that compiles a substantial fragment of the C language in a functionally correct and provably secure manner.

**Challenges and limitations.** Despite using a verified compiler, e.g., CompCert, can preserve security properties proved at the source-code level, the compiled assembly code is less efficient and adaptable to less platforms than those generated by mainstream compilers. The current logical model of constant-time does not cover microarchitectural features in modern hardwares, e.g. out-of-order execution, which have been exploited to conduct devastating attacks, such as Meltdown [Lip+18], and Spectre [Koc+19].

### 3.3.2 Satisfiability Modulo Theories (SMT)

Satisfiability Modulo Theories (SMT) is a decision problem that determines whether a first-order logic formula, which extends propositional logic by enabling quantifiers over non-boolean variables and relationship between them, is satisfiable under a given theory [BT18]. Background theories can be arithmetic, bit-vectors, or uninterpreted functions, which are used to generalize boolean satisfiability problem (SAT).

SMT provides a reasonable trade-off between expressiveness and decidability, allowing to reason about complex mathematical structures through first-order logic while maintaining the efficiency of propositional logic. SMT solvers are applied to a wide range of model-checking applications, including bounded, predicate-abstraction-based model checking, and interpolation-based, as back-end engines to discharge verification conditions [BT18].

To decide if a formula restricted by a theory is satisfiable, SMT solvers can, in principle, convert the formula to Disjuntive Normal Form (DNF) [Pos21] and check each disjunct for satisfiability. However, this approach is computationally expensive and impractical for large formulas. Instead, modern SMT solvers incrementally refine and check DNF in a divide-and-conquer approach. In general, there are two main approaches to SMT solving:

- **Lazy approach:** This is a predominant approach used by SMT solvers, such as Z3 [MB08]. In this approach, a combination of SAT engine and theory solvers, which are specialized constraint satisfiability procedures, is used to solve the the satisfiability problem. The SAT engine is responsible for handling the propositional relations and constructing a propositional model. The theory solvers are responsible for checking the satisfiability of conjunctions of theory literals. In other words, the SAT engine build a propositional model, and the theory solvers check if the theory literals that corresponds to the propositional model are satisfiable under the background theory. Advanced integration between SAT engine and theory solvers allows SMT solvers to obtain features as follows:
  - **Incrementality:** Theory solvers can incrementally check the satisfiability of a formula by adding new constraints to the existing model. This allows SMT solvers to efficiently handle large formulas and complex theories.
  - **Backtracking:** Theory solvers can backtrack to previous states when a formula is unsatisfiable, maintaining the same state as the SAT engine.

  - **Conflict set generation:** When a formula is found to be unsatisfiable, theory solvers can generate a conflict set, which is a set of constraints that cannot be satisfied simultaneously. This conflict set can be learned by the SAT engine and used to prune the search space in future iterations.
  - **Literal deduction:** Theory solvers can find literals included in its current set of literals that are implied by the current model. This information can be transmitted to the SAT engine to avoid guessing the value of these literals in future iterations.
  - **Explaination generation:** Theory solvers can have the ability to generate explanations for why a literal is deduced or why a conflict occurred. This is useful for SAT engines to do backtracking and conflict resolution.

  The currently most prominent example of lazy-approach SMT solver is DPLL(T) architecture [NOT06].

- **Eager approach:** This approach aims to fully reduce a SMT problem to a SAT problem through encoding. Given a first-order logic formula, an eager SMT solver encodes the formula into a propositional logic formula that is satisfiable if and only if the original formula is satisfiable under the background theory. Then, the encoded propositional logic formula is solved by a SAT solver.

The lazy approach is more efficient than the eager approach for most SMT problems, as it empowers theory-specific reasoning to specialized theory solvers, which can handle complex theories more efficiently than a general-purpose SAT solver.

### 3.3.3 Floyd-Hoare logic and weakest precondition logic

**Notations.** `P` denotes a pre-condition (requirements on an input), `Q` is a program, and `R` denotes a post-condition (description of an output).

**Floyd-Hoare logic.** Floyd-Hoare logic is a logical basis for formally reasoning properties of a program [Hoa69]. The central concept of Floyd-Hoare logic is the notation, also known as a Hoare triples:

$$\texttt{P}\,\{\texttt{Q}\}\,\texttt{R}.$$

Informally, `P` {`Q`} `R` states that if the pre-condition `P` is satisfied, or assertion `P` is true, before the program `Q` is initialized, then the post-condition `R` is ensured, or assertion `R` is true, at the completion of program `Q`. This logical basis allows using properly defined axioms and rules of inference to prove properties about programs.

**Weakest pre-condition logic.** Developed upon the Floyd-Hoare logic, the weakest pre-condition, denoted as `wp`, logic demonstrates a calculus for the formal derivation of programs. Unlike the Floyd-Hoare logic, which demonstrates that a program will not produce undesirable results given the sufficient pre-conditions, the weakest

pre-condition logic demonstrates that a program will generate desiable results given necessary and sufficient pre-conditions [Dij75]. Hence, wp introduces less restrictive rules than the Floyd-Hoare logic. More specifically, the weakest pre-condition logic is formally represented as:

$$\mathtt{wp}(\mathtt{Q}, \mathtt{R}).$$

Logically, we can prove properties, or post-conditions, of a program through the following transformation:

$$\mathtt{P}\,\{\mathtt{Q}\}\,\mathtt{R} \rightarrow (\mathtt{wp}(\mathtt{Q}, \mathtt{R}))\ \{\mathtt{Q}\}\,\mathtt{R}, \tag{1}$$

and the Equation 1 holds if and only if $\mathtt{P} \Rightarrow \mathtt{wp}(\mathtt{Q}, \mathtt{R})$ holds.

The Floyd-Hoare logic and the weakest pre-condition logic are core principles leveraged by the F* ecosystem [Swa+16] to solve satisfiability problems.

## 3.4 F*

F* [Swa+16] is a single, proof-oriented language that integrates the functions of three tools: a proof assistant, an effectful programming language, and a SMT-backed semi-automated verifier. In other words, F* aims to synthesize the strengths of different proof systems—including proof assistants (Coq, Agda), general-purpose languages (OCaml, Haskell), and semi-automated verifiers (Dafny, WhyML)—into a single entry point. This approach ensures that simple verification is fully automated and complex verification is terminated with the support of user's guidance. F* code, once verified, can be extracted for execution in OCaml, F#, and C.

**Primitive effects.** In order to play different roles in the program verification ecosystem, F* employs an extensible lattice of monadic effects. By default, F* provides the following built-in effects:

- `PURE`: a pure, or recursive, function that does not have any observable side-effects.
- `DIV`: a function that have branches of computation, resulting in potentially divergent code.
- `GHOST`: a function that is purely for verification, containing only specificational computations, and is erased if the program is extracted for execution.
- `STATE`: a function that has stateful computations.
- `EXN`: a function that possibly raises an exception.
- `ALL`: a function that has all primitive effects.

Although these effects are primitives, they can be granularly refined by a user through predicate transformers. The finely-grained, user-specified effects, have to satisfy the following strict requirements: `PURE` monad egregates side-effect-free computations from other effects; `GHOST` effect covers proof-only computations; `ALL` provides a

semantic to a collection of all primitive effects. In addition, while `PURE` and `GHOST` are totally correct effects, termination and correctness are guaranteed, `DIV`, `STATE`, and `EXN` are partially correct, with the assumption of termination, due to the possiblity of divergence.

**Predicate Transformers as Semantics.** Developed using weakest precondition semantics, F* utilizes its dependent type system to elevate the precondition transformers into a monad. This monad is defined over the types themselves, rather than being restricted to pure computations [Swa+16]. In F*, each effect has its own monadic and predicate transformer semantics. More precisely, for every type `a`, there is a corresponding type `M.WP a`, where `M` is a monad and `WP` denotes a weakest pre-condition, which models predicate transformers for functions that returns a value of type `a`.

**Refinement type (intrinsic proof).** The automation capability of F* in SMT solving is powered by the strongly-typed dependence mechanism. In this system, specifications are expressed using dependent types, which uses type-level computations to define the equivalence between types. Refinement type, a key component in this mechanism, is a type of the form `x : t{`$\phi$`}`, which restricts a type (`t`) to only include those values (`e`) that satisfy a specific logical property ($\phi$) [Swa+16]. For example, the positive natural numbers, the type `pos_nat`, is defined as `pos_nat:int{x > 0}`, where `int` is a type for integers. Through the mechanism of subtyping, refinement types inherently enable proof irrelevance and establish modularity in proofs. In addition, this approach enables automated reasoning since the SMT solver can implicity refine the values of base type into the subtype when the formula is satisfiable.

**Lemma (extrinsic proof).** Besides refinement types, F* supports another approach for representing specifications, namely **`Lemma`**, as program properties, in some complex cases, cannot be technically represented in refinement types. The **`Lemma`** construct acts as a Hoare-logic-based refinement mechanism that is applied to pure, side-effect-free computations. This approach allows a user to prove post-conditions of a pure function seperately from its definition. These lemmas are erased during extraction process. Furthermore, after the lemmas corresponding to a particular function are proved, subtyping relations can be applied to reason the returning value a more precise type. Specifically, **`Lemma`** is beneficial in cases that manual proofs are required for conditions cannot be automatically proved by F*'s subtyping mechanism.

**Verification principle.** Based on a program and its specification, whether defined by refinement types and lemmas, F* automatically infers its corresponding type, effect, and predicate transformer [Swa+16]. Subsequently, it generates proof obligations to verify the consistency between the inferred predicate transformer and the user's specification. These obligations are then resolved through a two-step process: first, SMT solving, and then, if necessary, manual proofs.

**Syntactic sugar.** In order to ease the way of writing specifications, F* provides syntactic sugars for frequently-used terms. For instance, pre- and post-conditions with `PURE` effect are abbreviated as **`requires`** `p` and **`ensures`** `p`, where `p` denotes a predicate. On the other hand, the function that has unconditional `PURE` effect is aliased as **`Tot`**. These syntactic sugars are frequently used in examples and proof snippets illustrated in this thesis.

### 3.4.1 F* example

In this section, we show a toy example of ChaCha20 [Ber08], a stream cipher, specfication that utilizes both intrinsic and extrinsic proofs. The F* code snippet Listing 8 show many essential parts of F*'s syntax. Note that the code snippet just illustrates partly of the ChaCha20 specification and does not include the full implementation of ChaCha20.

In F*, a function implementation is defined by **`let`** keyword, wheareas a function signature is optionally declared by **`val`** keyword. Arrow types, encoded as →, are used in F* to specify input arguments and an output type for a function signature. The output computation type is as form of `E T`, where `E` is the computation effect and `T` is the return value type. For example, **`Tot`** `bytes` states that the function has `PURE` effect and returns a sequence of bytes.

Refinement types are illustrated in `chacha20_encrypt_bytes` function. `cipher` computation type has an enriched type based on the base type `bytes`. The refinement type `bytes{length cipher == length msg}`, ensures that the length of `cipher` must be equal to the length of `msg`. This is an example of intrinsic proof in which enriched type helps SMT solver to automatically unfold terms and reach termination.

In addition to instrinsic proof, Listing 8 also illustrates an example of extrinsic proof through **`Lemma`**. `lemma_chacha20_decrypt_encrypt_equiv` is a lemma reasoning about the correctness of ChaCha20 specification. The lemma states that the original message `msg` is reproduced if the `ciphertext`, generated by `chacha20_encrypt_bytes` , is decrypted with the same `key`, `nonce`, and `counter`. **`Lemma`** is a special function which always return `unit ()` type and would be erased during extraction. Hence, **`Lemma`** is used only for logical proofs, not for program execution.

```
let size_key = 32 let size_nonce = 12 let size_block = 64

val chacha20_encrypt_bytes:
    k: lbytes size_key -> nonce: lbytes size_nonce -> counter: size_nat
    -> msg: bytes{length msg / size_block <= max_size_t}
    -> Tot cipher: bytes{length cipher == length msg}
let chacha20_encrypt_bytes key nonce ctr0 msg =
  let st0 = chacha20_init key nonce ctr0 in
  chacha20_update st0 msg

val chacha20_decrypt_bytes:
    k: key -> nonce: lbytes size_nonce -> counter: size_nat
    -> cipher: bytes{length cipher / size_block <= max_size_t}
    -> Tot msg: bytes{length cipher == length msg}
let chacha20_decrypt_bytes key nonce ctr0 cipher =
    let st0 = chacha20_init key nonce ctr0 in
    chacha20_update st0 cipher

(*Lemma for extrinsic proof: The lemma below ensures correctness of ChaCha20 specification.*)
let lemma_chacha20_decrypt_encrypt_equiv
    (k: lbytes size_key) (nonce: lbytes size_nonce) (counter: size_nat) (msg: bytes)
    : Lemma (ensures (
        let ciphertext = chacha20_encrypt_bytes key nonce counter msg in
        msg == (chacha20_decrypt_bytes k nonce counter ciphertext))) = ()
```

**Listing 8:** ChaCha20 F* code snippet, adapted from HACL* library [Zin+17].

## 3.5 Low*

Low* [Pro+17] is a small subset of F* which is specified for low-level programming, modelling the memory structure and control-flow of executable code. Unlike typical ML languages that rely on a garbage collector or implicit heap allocation, Low* introduces a memory structure that is similar to the one used in CompCert [Ler09], a verified compiler. This design provides the user the explicit control necessary for writing efficient, security-critical low-level code. From the user perspective, Low* has syntactically similar to F* and is equipped with additional low-level views of memory layout and access pattern.

**`ST` computation type.** `ST` is the core computation type for stateful functions that read/write data in a particular memory region. `ST` is as of the following form:

$$\texttt{ST (a : Type) (pre : s} \rightarrow \texttt{Type) (post : s} \rightarrow \texttt{a} \rightarrow \texttt{s} \rightarrow \texttt{Type)},$$

where `a` is a value type, `s` denotes a memory type (stack region, or heap region), and `pre`, and `pos`, are pre and pos-condition respectively [Pro+17]. This `ST` signature refers that a computation of type `ST`, which operates in the initial memory `m0:s` satisfying the pre-condition `pre m0`, generates an output `o:a` and satisfies the post-condition `post m0 o m1` within the final memory `m1:s`. While the value type `a` can be any built-in primitives in F* or user-defined, built on top primitives, types, the memory type `s` has to be instantiated by `HyperStack` memory model.

**`HyperStack` memory model.** `HyperStack` is a memory model that covers both stack and heap. Technically, `HyperStack` is generilzation of the `HyperHeap` memory model introduced in F* [Swa+16]. `HyperStack` organizes memory into distinct regions, each identified by a unique id (`rid`). These regions are categorized as either stack region or heap region using predicate `is_stack_region`. Given the `ST` monad and the `HyperStack` memory model, a stateful function can be modeled for initializing new heap regions, allocating, reading, writing, and freeing references in those regions [Pro+17]. For example, Listing 9 describes the the syntax of freeing a reference in a heap region. In particular, the pre-condition requires that the reference `x` is live in the initial memory `m` and the memory region is a heap region. The post-condition ensures that the memory remains unchanged and the dereferenced value `y` is equal to the value stored in the reference `x`.

```
val deref:
    x:ref a
    -> ST a
    (requires (fun m -> mem m x /\ not (is_stack_region m)))
    (ensures (fun m0 y m1 -> m0 = m1 /\ y = m1[x]))
```

**Listing 9:** Low* function signature of dereferencing a reference in a heap region, adapted from [Pro+17].

**Stack region.** Stack region, also called stack frame, provides an efficient C-like stack memory management along with the ability to reason about the stack region's

liveness, disjointness, and size. To model this, a `tip` region, which is a region that is on top of a stack, is introduced to represent the currently active memory space. In addition, primitive operations, such as `GHOST`-effected `pop` and `push`, along with their effectful variants `pop_frame` and `push_frame`, are provided to manipulate the stack region. To allocate a new stack region initialized with the given value, a programmer can use `salloc : intit:a`. The KaRaMel compiler guarantees that the abovementioned operations are well-scoped, meaning that the stack region is always well-formed as long as the post-condition of each operation is guaranteed.

**`Stack` effect.** The `Stack` effect is a specific instance of `ST` effect ensuring that the stack tip is unchanged, `tip : m0 = tip : m1`, and the initial and final memory regions are equivalent, after the computation. In other words, it guarantees that any temporary stack frames created during the computation are properly popped, and all heap regions are allocated within the function scope are freed before the function returns. This guarantee is crucial for preventing memory leaks and ensuring that the `HyperStack.mem` remains well-formed throughout the execution of the program.

**Side-channel mitigation.** Low* and its formal foundation offer a robust framework for proving resistance to side-channel attacks, particularly those exploiting control flow and memory access patterns. First, Low* supports a programming style that leverages type abstraction to protect secret-dependent state, such as cryptographic keys or intermediate values. By using abstract types, it ensures that no information about secrets can be leaked through their type structure. Second, operations on secret data are implemented using these abstract types in a way that avoids leakage through outputs, branches, or memory access patterns. For instance, the `eq_mask` operation securely tests equality by returning a mask rather than a boolean, unlike a leaking alternative such as `eq_leak`. Secrets are only "ghostly revealed" in ghost code. Third, the formal semantics of $\lambda$ow*, a formal model of Low*, are instrumented to record execution events such as branching points (`brT`, `brF`) and memory accesses (`read`, `write`), enabling formal reasoning about possible side channels. Finally, it is proved that Low* programs written against a secret interface, where secrets are abstract and operations are secret-independent, generate identical execution traces regardless of secret values [Pro+17]. This trace independence provides strong guarantees against side-channel leakage via control flow or memory access patterns.

### 3.5.1 Low* example

Following the F* example in Subsection 3.4.1, we show a Low* code snippet of ChaCha20 [Ber08] specification in Listing 10. Unlike the F* code snippet, this Low* code aims to establish formal guarantees at the machine-code level. The code snippet illustrates the use of `Stack` effect and `lbuffer` type, which is a Low* type for a mutable buffer, to model the ChaCha20 specification.

The `chacha20_encrypt` function is a stateful function that takes the length of the output (`len`), a mutable buffer for the output (`out`), a plaintext (`text`), a key (`key`), a nonce (`n`), and a counter (`ctr`) as input arguments.

The computation type of the function is `Stack : unit`, which indicates that the function has `Stack` effect and returns a unit value. The pre-condition of the function requires that the output, plaintext, key, and nonce buffers are live in the initial memory. In addition, the function requires that buffers representing the plaintext and output are either disjoint or fully overlapping, ensuring that in-place encryption is safely handled.

The post-condition ensures that only the output buffer is modified in the final memory, and its content must be equal to the high-level specification of ChaCha20, which is `chacha20_encrypt_bytes` function shown in the Listing 8. The `as_seq : mem : buffer` is a Low* function that converts a mutable buffer to a sequence, used for comparing the content of the output buffer with the expected result.

Inside the function body, `push_frame` and `pop_frame` are used to manage the stack region, ensuring that the stack frame is properly allocated and deallocated during the execution of the function.

The Low* code snippet can be compiled to C code, as shown in Listing 11, by the KaRaMel compiler. The C code snippet illustrates the translation from the Low* code to C, where `Hacl_Chacha20_chacha20_encrypt` function is implemented as a C function with the same input arguments and a similar output type. The `unit` type in Low* is translated to **`void`** in C, indicating that the function does not return any value.

```
module Spec = Spec.Chacha20

val chacha20_encrypt:
    len:size_t
  -> out:lbuffer uint8 len -> text:lbuffer uint8 len
  -> key:lbuffer uint8 32ul -> n:lbuffer uint8 12ul -> ctr:size_t
  -> Stack unit
  (requires fun h ->
    live h key /\ live h n /\ live h text /\ live h out /\ eq_or_disjoint text out)
  (ensures  fun h0 _ h1 -> modifies (loc out) h0 h1 /\
    as_seq h1 out == Spec.chacha20_encrypt_bytes (as_seq h0 key) (as_seq h0 n) (v ctr) (as_seq h0 text))

let chacha20_encrypt len out text key n ctr =
  push_frame();
  let ctx = create (size 16) (u32 0) in
  chacha20_init ctx key n ctr;
  chacha20_update ctx len out text;
  pop_frame()
```

**Listing 10:** ChaCha20 Low* code snippet, adapted from HACL* library [Zin+17].

```
void Hacl_Chacha20_chacha20_encrypt(
    uint32_t len, uint8_t *out, uint8_t *text,
    uint8_t *key, uint8_t *n, uint32_t ctr) {
        uint32_t ctx[16U] = { 0U };
        Hacl_Impl_Chacha20_chacha20_init(ctx, key, n, ctr);
        Hacl_Impl_Chacha20_chacha20_update(ctx, len, out, text);
    }
```

**Listing 11:** ChaCha20 C code snippet, compiled by KaRaMel [Zin+17].

## 3.6 KaRaMel

KaRaMel is compiler that translate Low* to well-formatted C code. The resulting C code then can be compiled to machine code using various C compilers, both mainstream and verified ones, including Clang, GCC, and CompCert. Overally, the toolchain translates Low* programs into C code through a series of intermediate stages as illustrated in Figure 8.

**Low* to λow*.** Firstly, an ML-like Abstract Syntax Tree (AST) for Low* terms is obtained by leveraging the existing extraction features in F*, similar to features in Coq [Pie08]. Then, KaRaMel transforms this AST into the λow*subset, which is a formalized translation. The resulting subset at the end of this stage, λow*, is a formal model of Low* after irrelevant specifications and ghost code are removed. Furthermore, λow*supports step-by-step execution rules, producing traces for memory access and control branching [Pro+17]. This tracing mechanism is vital for reasoning side-channel leakages.

**λow* to C*.** C* is an intermediate language that retains the explicit scoping structure of λow*, but offers calling conventions and syntax more similar to C by utilizing an explicit call-stack of continuations isolated from memory regions [Pro+17]. This translation preserves execution traces, formally proved by bisimulation, which is beneficial for reasoning side-channel leakages based on trace equivalence.

**C* to Clight.** Clight [Bar+20; Ler09] is a deterministic subset of C utilized by the verified compiler CompCert. By employing CompCert's verified backend, the formal guarantees proved at source-code level can provably go down to machine-code level. This transformation needs to address several discrepancies between C* and Clight. While Clight only supports non-compound values directly, C* manages constant local structures as first-class values [Pro+17].

**Clight to C.** KaRaMel aims to generate pretty-printed, readable C code for manual review and maintainability at the end. Hence, the toolchain relies heavily on an abstract C AST and the C standard coding style. For instance, `enum` and `switch` are used whenever possible rather than magic constants and deeply-nested `if – else` clauses. This issue is solved by partitioning C* structures into non-compound fields which then are translated to local variables in Clight [Pro+17]. Another issue is to bridge the gap between an expression language, Low* or F*, to a statement language, C. Specifically, in C code, a buffer allocated under a conditional branch, like `if – then`, is freed at the end of the branch, but there is no such block-scope mechanism in Low*, which has an explicit predicate `with_frame` just for constructing proof. To resolve the conflict between C99 block scope and Low* `with_frame`, buffers allocated inside a branch must be hoisted to the closest `push_frame` to avoid misaligned lifetime post-transition [Pro+17].

**C to machine code:** Given the C code, a user can compile it with any C compilers, such as GCC, Clang, or Microsoft's C compiler. If the programmer aims to have the highest level of assurance, through formal guarantees, for non-performance-critical code, it is highly recommended to use CompCert [Ler09], a verified compiler.

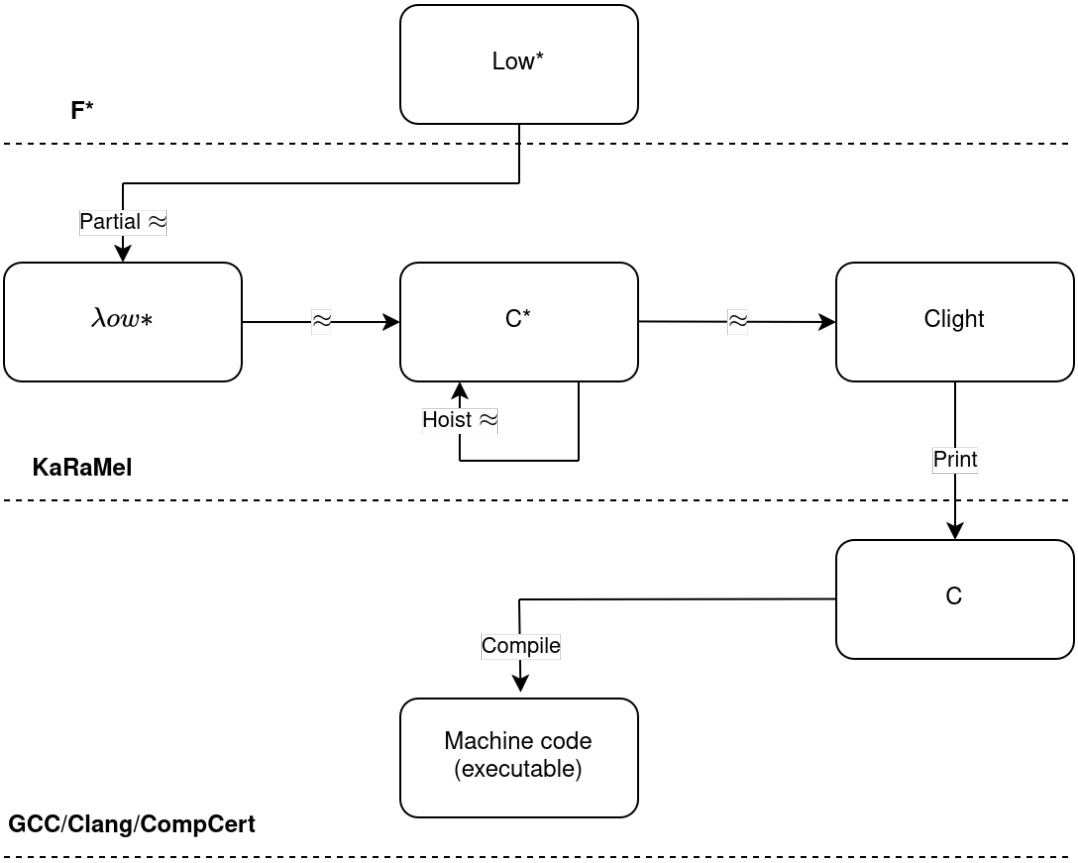



**Figure 8:** The toolchain for translating Low* to machine code, adapted from [Pro+17].

# 4 KEMEDHOC Design

This section describes the design of the post-quantum variant of EDHOC, called KEMEDHOC, which replaces DH with KEM. The following sections describe the protocol overview, the key schedule, and security properties.

The quantum-proof design still maintains three mandatory messages and one optional message, similar to EDHOC, for explicit key confirmation. Moreover, new fields are introduced in Message 1 and Message 2 either as the drop-in replacement or the add-on to the existing EDHOC message fields.

**Credentials.** KEMEDHOC still relies on the X.509/C.509 certificate, and raw public key for retrieving peers' credentials. However, these certificates should reserve an additional buffer for embedding the KEM authentication key, which is noticeably larger than pre-quantum alternatives. This memory overhead can cause fragmentation, in which a transmitted certificate exceeds the maximum size that an IP protocol handle in one shot. The requirement for fragmentation management is beyond the scope of protocol design at the application layer. Fortunately, CoAP [SHB14] protocol, the underneath application protocol, has a built-in handler to resolve this fragmentation issue. In a typical setting, those credentials should be provisioned out-of-band in a secure channel, beyond the scope of the key exchange session, then only their identifiers, which are more compact, are transmitted during the protocol run.

**Authentication method and cipher suite.** KEMEDHOC introduces a new authentication method, which is based on long-term KEM keys, leading to the contruction for the new value for `METHOD`, decoded as an integer of 5. Similarly, new slots for post-quantum cipher suites need to be defined for COSE encoding to serialize the cipher suite field in Message 1.

**Migrating DH to KEM.** In [Fuj+12; Alm+24], the authors describe how to construct a generic authenticated key exchange protocol using only KEM. We mainly follow this approach to build the quantum-safe variant of EDHOC. For this, we use a KEM which is *Indistinguishability under Chosen Ciphertext Attack* (IND-CCA). In general, projecting to the DH-based key exchange, the KEM public key and encapsulating ciphertext serve as the replacement to DH public share, and the shared KEM key plays the role of DH shared secret. In terms of authentication, the involving parties have their own long-term KEM public/private key pairs, serving the same purpose, but not the same mechanism, as static DH key pairs. The public key of these long-term pairs, namely $\texttt{pk}_\texttt{I}$ and $\texttt{pk}_\texttt{R}$, are embedded in the corresponding credential (X509 or C509 certificate) or encoded as a stand-alone raw public key. In a typical setting, these long-term public keys are known to the peers before the protocol run.

Note that KEM cannot replace DH in a non-interactive setting, in which the `Initiator` has a guarantee of implicit authentication even after the first message due to the ephemeral-static DH shares. Instead, the `Initiator` needs to wait for the `Responder`'s response including the ciphertext from an encapsulation, bounded to the

Initiator's KEM public key, to assure a sort of authentication. The full message flow is illustrated in Figure 9.

## 4.1 Message Flow.

In general, KEMEDHOC protocol proceeds the following phases.

### 4.1.1 Message 1

**Initiator composing Message 1.** The Intitator selects the cipher suite that supports KEM primitives and method 5. Then, it sets up a connection ID ($C_I$), which can be randomly generated bytes or an increasing counter depending on the application. The Initiator generates a KEM private/public key pair ($sk_e, pk_e$) for ephemeral key exchange. In order to authenticate the Responder, the Initiator encapsulates Responder's public static KEM key $pk_R$ to obtain a ciphertext and a shared key for authentication, denoted $ct\_auth_R$ and $K\_auth_R$ respectively.

From the computed authentication shared key $K\_auth_R$, the Initiator derives the pseudorandom key $PRK_{1e} \leftarrow Extract(TH_1, K\_auth_R)$, where $TH_1 \leftarrow H(pk_e, ct\_auth_R)$. Then, the encryption key $K_1$ and initial vector $IV_1$ are further expanded from $PRK_{1e}$ through the HKDF Expand function. The plaintext $PTX_1 := (C_I, ID\_CRED_I, EAD_1)$ is encrypted by the AEAD encryption using the derived $K_1$ and $IV_1$. The resulting ciphertext $C_1$, along with the chosen method (Method), cipher suite (CipherSuite), public ephemeral KEM key $pk_e$, authentication ciphertext $ct\_auth_R$, and connection ID ($C_I$) are sent to the Responder as the Message 1 (Msg1).

**Responder processing Message 1.** Upon receiving Message 1, Responder verifies that if it supports the authentication method and cipher suite proposed by the Initiator. If does not support, then it returns a error message *'Wrong selected cipher suite'* to ask the Initiator to re-initiate the session with another cipher suite. If Responder supports the selected cipher suite, then it starts composing Message 2.

### 4.1.2 Message 2

**Responder authenticating Initiator.** The Responder decapsulates the authentication ciphertext $ct\_auth_R$, using its private static KEM key $sk_R$, to obtain the authentication key $K\_auth_R$. From the computed $K\_auth_R$, the Responder derives the encryption key $K_1$ and $IV_1$ as the Initiator does. Then, it decrypts the ciphertext retrieved from $Msg_1$ to obtain $ID\_CRED_I$. If the decryption fails due to invalidity of AEAD authentication tag, the Responder terminates the session and returns an error message. If the decryption is successful, the Responder then looks up to identify the locally-stored credential that matches $ID\_CRED_I$. If there is no matching credential, the Responder returns an error message *'Unknown Credential Referenced'*. Otherwise, Responder continues proceeding the handshake session.

**Responder composing Message 2.** Responder does encapsulation, given the public ephemeral KEM key $\mathtt{pk_e}$ as input, to obtain a pair of ciphertext and shared key, $\mathtt{ct_Y}$ and $\mathtt{K_e}$, for ephemeral key exchange. To construct authentication materials to the Initiator, Responder encapsulates the public static KEM key of Initiator, $\mathtt{pk_I}$, and attains a pair of ciphertext and shared key, $\mathtt{ct\_auth_I}$ and $\mathtt{K\_auth_I}$ respectively. The pseudorandom key $\mathtt{PRK_{2e}}$ is extracted from the hash of $\mathtt{PRK_{1e}}$, transcript hash $\mathtt{TH_2}$, and the ephemeral shared key $\mathtt{K_e}$. In particular, $\mathtt{PRK_{2e}} \leftarrow \mathtt{Extract}(\mathtt{H}(\mathtt{PRK_{1e}}, \mathtt{TH_2}), \mathtt{K_e})$, where $\mathtt{TH_2} \leftarrow \mathtt{H}(\mathtt{ct_Y}, \mathtt{ct\_auth_I}, \mathtt{H}(\mathtt{Msg_1}))$.

Then, the Responder further derives the pseudorandom key $\mathtt{PRK_{3e2m}}$ from the authentication key $\mathtt{K\_auth_R}$ and $\mathtt{SALT_{3e2m}}$, $\mathtt{PRK_{3e2m}} \leftarrow \mathtt{Extract}(\mathtt{SALT_{3e2m}}, \mathtt{K\_auth_R})$. The encryption key $\mathtt{K_2}$ and MAC tag $\mathtt{MAC_2}$ are further expanded from $\mathtt{PRK_{3e2m}}$. Then the plaintext 2, $\mathtt{PTX_2} := (\mathtt{C_R}, \mathtt{ID\_CRED_R}, \mathtt{EAD_2}, \mathtt{MAC_2})$, is encrypted by the AEAD encryption scheme using the encryption key $\mathtt{K_2}$, generating the ciphertext $\mathtt{C_2}$. The Message 2 comprises the ciphertext $\mathtt{C_2}$ and the ephemeral KEM ciphertext $\mathtt{ct_Y}$.

**Initiator processing Message 2.** Upon receiving the Message 2 $\mathtt{Msg_2}$, the Initiator decapsulates the transmitted KEM ciphertext $\mathtt{ct_Y}$ using the private ephemeral KEM key $\mathtt{sk_e}$ to obtain the ephemeral shared key $\mathtt{K_e}$. To achieve the authentication key $\mathtt{K\_auth_I}$, the Initiator decapsulates the authentication KEM ciphertext $\mathtt{ct\_auth_I}$ using the private static KEM key $\mathtt{sk_I}$. Then, the Responder derives pseudorandom keys $\mathtt{PRK_{2e}}$, $\mathtt{PRK_{3e2m}}$, and the encryption key $\mathtt{K_2}$ as the Intitator does. Given the key $\mathtt{K_2}$, the Responder is able to decrypt the ciphertext retrieved from $\mathtt{Msg_2}$ and obtain the credential identifier $\mathtt{ID\_CRED_R}$ and MAC tag $\mathtt{MAC_2}$.

From the extracted $\mathtt{ID\_CRED_R}$, the Initiator queries its local storage to find the matching credential $\mathtt{CRED_R}$. If there is no such that matching credential, the Initiator returns an error message *'Unknown Credential Referenced'*. If a corresponding credential is found, then the Initiator constructs $\mathtt{MAC'_2}$ and then compare to the retrieved $\mathtt{MAC_2}$. If the two MAC tags are not equal, then the Initiator returns an error code and terminates the session. Otherwise, the Initiator proceeds the protocol session and composes the Message 3.

### 4.1.3 Message 3

**Initiator composing Message 3.** The Initiator derives encryption key $\mathtt{K_3}$ and initial vector $\mathtt{IV_3}$ from the pseudorandom key $\mathtt{PRK_{3e2m}}$ and transcript hash $\mathtt{TH_3}$, where $\mathtt{TH_3} \leftarrow \mathtt{H}(\mathtt{TH_2}, \mathtt{PTX_2}, \mathtt{CRED_R})$. The pseudorandom key $\mathtt{PRK_{4e3m}}$ is extracted from the authentication key $\mathtt{K\_auth_I}$ and the expanded salt $\mathtt{SALT_{4e3m}}$. In particular, $\mathtt{PRK_{4e3m}} \leftarrow \mathtt{Extract}(\mathtt{SALT_{4e3m}}, \mathtt{K\_auth_I})$.

The Reponder expands the pseudorandom key $\mathtt{PRK_{4e3m}}$ to obtain MAC tag $\mathtt{MAC_3}$ and the shared session key $\mathtt{PRK_{out}}$. The plaintext $\mathtt{PTX_3} := (\mathtt{ID\_CRED_I}, \mathtt{EAD_3}, \mathtt{MAC_3})$ is encrypted by AEAD encryption using the encryption key $\mathtt{K_3}$ and intial vector $\mathtt{IV_3}$, resulting in the authenticated ciphertext 3 $\mathtt{C_3}$. The Message 3 ($\mathtt{Msg_3}$) consists of only the authenticated ciphertext $\mathtt{C_3}$.

**Responder processing Message 3.** Upon receiving the Message 3 $\texttt{Msg}_3$, the Responder derives $\texttt{K}_3$ and $\texttt{IV}_3$ as the Initiator does. Given these keying materials, the Responder decrypts the ciphertext 3, also known as the Message 3, to obtain the plaintext 3 $\texttt{PTX}_3$ and authentication AEAD tag. If the AEAD tag is not valid, or the decryption fails, the Responder returns an error code and terminates the session. If the decryption is successful, the AEAD tag is valid, the Responder retrieves $\texttt{ID\_CRED}_\texttt{I}$ and the MAC tag $\texttt{MAC}_3$.

The Responder then verifies that if the decrypted $\texttt{ID\_CRED}_\texttt{I}$ matches the credential identifier the Responder has received in the Message 1. If the two identifiers are not matching to each other, then the Responder returns an error code and terminates the session. Otherwise, computes the pseudorandom key $\texttt{PRK}_{\texttt{4e3m}}$ and MAC tag $\texttt{MAC}'_3$ as the Initiator does.

If the computed MAC tag $\texttt{MAC}'_3$ and the retrieved MAC tag $\texttt{MAC}_3$ are not equal, the Responder returns an error code. If the two MAC tags are equal, the Responder derives the session key $\texttt{PRK}_{\texttt{out}}$.

At this point, if the two honest parties should share the same session key $\texttt{PRK}_{\texttt{out}}$, which can be further derived to other application key through the exporter interface.

### 4.1.4 Message 4 (optional)

**Responder composing Message 4.** This optional message, similarly in EDHOC, serves for the goal of explicit key confirmation. The Responder derives AEAD keying materials, encryption key $\texttt{K}_4$ and intial vector $\texttt{IV}_4$, and then encrypts the external authorization data $\texttt{EAD}_4$, obtaining the ciphertext 4. This authenticated ciphertext 4 serves as the Message 4 $\texttt{Msg}_4$.

**Initiator processing Message 4.** Upon receiving the Message 4, the Initiator derives the encryption key $\texttt{K}_4$ and initial vector $\texttt{IV}_4$ as the Responder does. Then the Initiator decrypts the Message 4 to obtain the $\texttt{EAD}_4$. If the decryption fails, generating an invalid AEAD tag, it returns an error code and aborts the protocol session. Otherwise, the Initiator exposes the $\texttt{EAD}_4$ value to the application layer. At this point, the Initiator assures that it and the other party , the Responder, have shared the same session key.

## 4.2 Key Schedule

KEMEDHOC's key schedule scheme is based on the original EDHOC's key schedule with the replacement of ephemeral-static DH shares ,$\texttt{G}_{\texttt{rx}}$ and $\texttt{G}_{\texttt{iy}}$, by authentication KEM keys, $\texttt{K\_auth}_\texttt{R}$ and $\texttt{K\_auth}_\texttt{I}$. The full detail of KEMEDHOC's key schedule is illustrated at Figure 10.

**Transcript Hashes.** Transcript hashes ($\texttt{TH}_1$, $\texttt{TH}_2$, $\texttt{TH}_3$, $\texttt{TH}_4$) used throughout KEMEDHOC protocol are summarized in the following table Table 8.

| TH | Definition |
|---|---|
| $\mathtt{TH}_1$ | $\mathtt{H}(\mathtt{pk}_\mathtt{e}, \mathtt{ct\_auth}_\mathtt{R})$ |
| $\mathtt{TH}_2$ | $\mathtt{H}(\mathtt{ct}_\mathtt{Y}, \mathtt{ct\_auth}_\mathtt{I}, \mathtt{H}(\mathtt{Msg}_1))$ |
| $\mathtt{TH}_3$ | $\mathtt{H}(\mathtt{TH}_2, \mathtt{PTXT}_2, \mathtt{CRED}_\mathtt{R})$ |
| $\mathtt{TH}_4$ | $\mathtt{H}(\mathtt{TH}_3, \mathtt{PTXT}_3, \mathtt{CRED}_\mathtt{I})$ |

**Table 8:** KEMEDHOC Transcript Hashes.

**Pseudorandom Keys.** Pseudorandom keys and salts used throughout KEMEDHOC protocol are summarized in the following table Table 9.

| Key/Salt | Definition |
|---|---|
| $\mathtt{PRK}_{1e}$ | $\mathtt{Extract}(\mathtt{TH}_1, \mathtt{K\_auth}_\mathtt{R})$ |
| $\mathtt{PRK}_{2e}$ | $\mathtt{Extract}(\mathtt{H}(\mathtt{PRK}_{1e}, \mathtt{TH}_2), \mathtt{K}^{\mathtt{XY}})$ |
| $\mathtt{SALT}_{3e2m}$ | $\mathtt{Expand}(\mathtt{PRK}_{2e}, (4, \mathtt{TH}_2, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{3e2m}$ | $\mathtt{Extract}(\mathtt{SALT}_{3e2m}, \mathtt{K\_auth}_\mathtt{R})$ |
| $\mathtt{SALT}_{4e3m}$ | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (8, \mathtt{TH}_3, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{4e3m}$ | $\mathtt{Extract}(\mathtt{SALT}_{4e3m}, \mathtt{K\_auth}_\mathtt{I})$ |
| $\mathtt{PRK}_{out}$ | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (10, \mathtt{TH}_4, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |
| $\mathtt{PRK}_{exporter}$ | $\mathtt{Expand}(\mathtt{PRK}_{out}, (13, \varepsilon, \lvert\mathtt{Hash}\rvert), \lvert\mathtt{Hash}\rvert)$ |

**Table 9:** KEMEDHOC Pseudorandom Keys.

**MAC Tags.** MAC tags used throughout KEMEDHOC protocol are summarized in the following table Table 10.

| MAC Tag | Definition |
|---|---|
| $\mathtt{MAC}_2$ | $\mathtt{Expand}(\mathtt{PRK}_{3e2m}, (5, (\mathtt{C}_\mathtt{R}, \mathtt{ID\_CRED}_\mathtt{R}, \mathtt{TH}_2, \mathtt{CRED}_\mathtt{R}, \mathtt{EAD}_2), \lvert\mathtt{MAC}_2\rvert), \lvert\mathtt{MAC}_2\rvert)$ |
| $\mathtt{MAC}_3$ | $\mathtt{Expand}(\mathtt{PRK}_{4e3m}, (9, (\mathtt{ID\_CRED}_\mathtt{I}, \mathtt{TH}_3, \mathtt{CRED}_\mathtt{I}, \mathtt{EAD}_3), \lvert\mathtt{MAC}_3\rvert), \lvert\mathtt{MAC}_3\rvert)$ |

**Table 10:** KEMEDHOC MAC Tags.

**Initial Vectors (IVs).** Initial vectors used throughout KEMEDHOC protocol are summarized in the following table Table 11.

**Encryption Keys.** Encryption keys used throughout KEMEDHOC protocol are summarized in the following table Table 12.

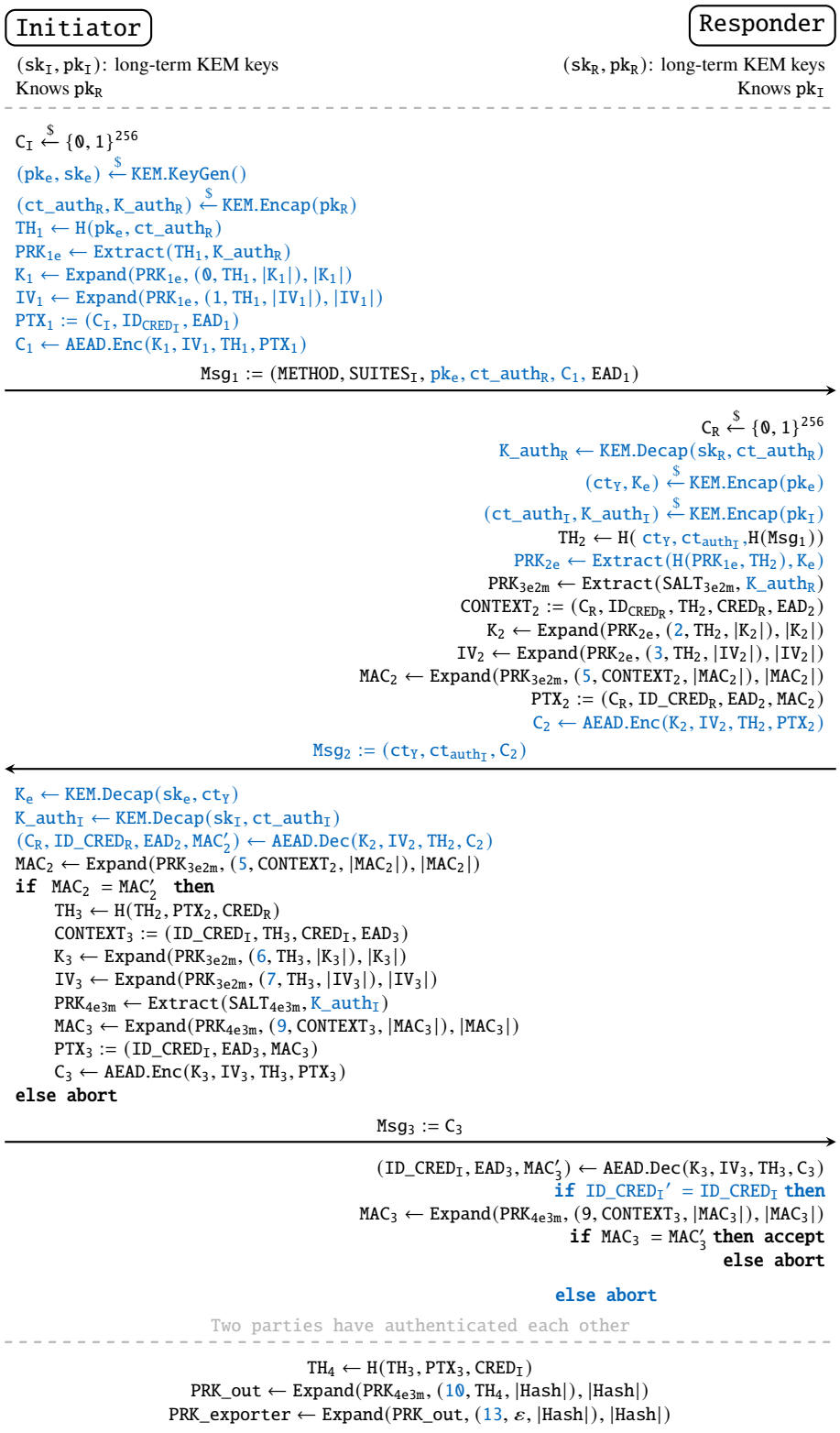



**Figure 9:** KEMEDHOC Protocol. The optional Message 4 is omitted for simplicity. Highlighted in blue are differences from EDHOC protocol Figure 2.

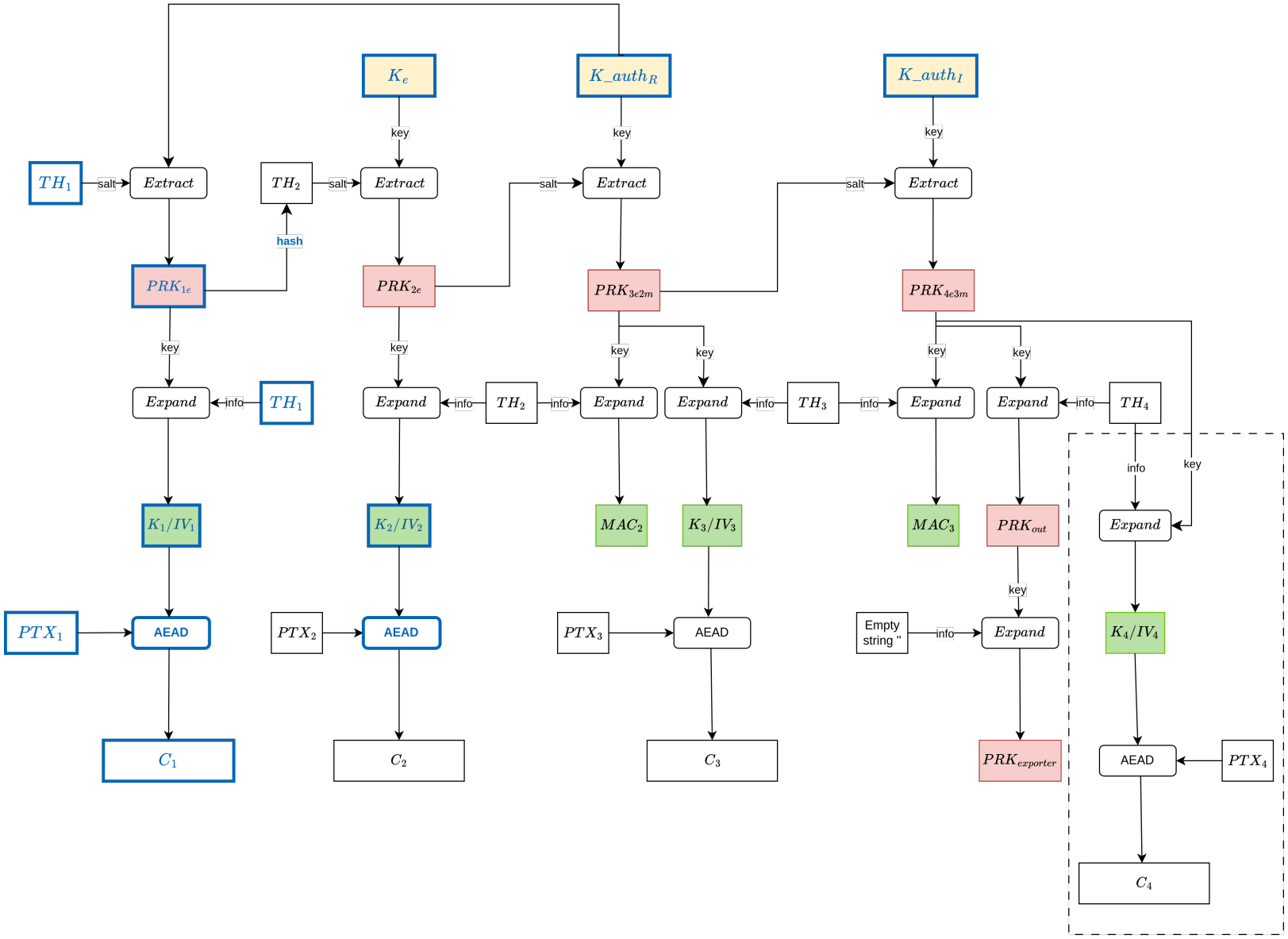



**Figure 10:** KEMEDHOC key schedule. Highlighted in blue are differences from EDHOC key schedule (Figure 3).

| IV | Definition |
|---|---|
| $\texttt{IV}_1$ | $\texttt{Expand}(\texttt{PRK}_{1e}, (1, \texttt{TH}_1, \lvert\texttt{IV}_1\rvert), \lvert\texttt{IV}_1\rvert)$ |
| $\texttt{IV}_2$ | $\texttt{Expand}(\texttt{PRK}_{2e}, (3, \texttt{TH}_2, \lvert\texttt{IV}_2\rvert), \lvert\texttt{IV}_2\rvert)$ |
| $\texttt{IV}_3$ | $\texttt{Expand}(\texttt{PRK}_{3e2m}, (7, \texttt{TH}_3, \lvert\texttt{IV}_3\rvert), \lvert\texttt{IV}_3\rvert)$ |
| $\texttt{IV}_4$ | $\texttt{Expand}(\texttt{PRK}_{4e3m}, (12, \texttt{TH}_4, \lvert\texttt{IV}_4\rvert), \lvert\texttt{IV}_4\rvert)$ |

**Table 11:** KEMEDHOC Initial Vectors.

| Key | Definition |
|---|---|
| $\texttt{K}_1$ | $\texttt{Expand}(\texttt{PRK}_{1e}, (0, \texttt{TH}_1, \lvert\texttt{K}_1\rvert), \lvert\texttt{K}_1\rvert)$ |
| $\texttt{K}_2$ | $\texttt{Expand}(\texttt{PRK}_{2e}, (2, \texttt{TH}_2, \lvert\texttt{K}_2\rvert), \lvert\texttt{K}_2\rvert)$ |
| $\texttt{K}_3$ | $\texttt{Expand}(\texttt{PRK}_{3e2m}, (6, \texttt{TH}_3, \lvert\texttt{K}_3\rvert), \lvert\texttt{K}_3\rvert)$ |
| $\texttt{K}_4$ | $\texttt{Expand}(\texttt{PRK}_{4e3m}, (11, \texttt{TH}_4, \lvert\texttt{K}_4\rvert), \lvert\texttt{K}_4\rvert)$ |

**Table 12:** KEMEDHOC Encryption Keys.

## 4.3 Changes from EDHOC design

This section describes what have been changed from EDHOC design and their motivation. Major changes are introduced as follows.

### 4.3.1 Changes in Message 1

**`Initiator`'s credential and credential identifier are sent in Message 1.** Following the authenticated key exchange model described in Figure 5, the `Responder` needs to know the public key of the `Initiator` $\texttt{pk}_\texttt{I}$ to perform $\texttt{KEM.Encap}(\texttt{pk}_\texttt{I})$. Although, this public key material is included in the credential stored locally, `Responder` needs a pointer indexing which stored credential it should retrieve, raising the need for transmitting `ID_CRED_I` on Message 1. Moreover, in the setting that the `Responder` has not yet stored the credential of `Initiator`, then indeed the credential payload has to be sent along with its identifier on the first message. Upon receiving Message 1, the `Responder` can retrieve `Initiator`'s credential; therefore, it can verify `Initiator` before proceeding to Message 2. This behaviour differs from EDHOC design in which the `Responder` is only able to verify the `Initiator` after receiving Message 3.

**Additional derived intermediate keys and initial vectors.** In order to protect `Initiator`'s credential against an active attacker, who is able to modify the message, an authenticated encryption has to be performed over credential payload and its identifier. This leads to the need for the derivation of encryption key and initial vector which serve as input for AEAD encryption scheme. Note that credential payload and credential identifier require guarantees of both confidentiality and integrity to be secure against an active attacker. Following the principle stated in RFC 9528 [SMP24], encryption materials are derived from as much information as possible and each

derived intermediate value serves its own purpose, two intermediate values $\texttt{TH}_1$ and $\texttt{PRK}_{1e}$ are constructed following the key schedule model of EDHOC.

Instead of AEAD encryption scheme, a binary additive stream cipher over a plaintext and its MAC can also offer confidentiality and integrity, similar to the encryption and $\texttt{MAC}_2$ construction flow in Message 2. In both approaches, message 1, though, has to carry an extra same-sized MAC value, deriving a MAC tag costs more memory than deriving an IV as input for encryption scheme. In particular, a MAC tag consumes 8 or 16 bytes depending on cipher suite, wheareas an IV costs only between 7 and 12 bytes depending on AEAD algorithm.

**Cryptographic fields sent to `Responder`.** As mentioned above, encryption of `Initiator`'s credential and credential identifier, needs to be sent to `Responder`. Besides, $\texttt{pk}_e$, an ephemeral KEM public key, and $\texttt{ct\_auth}_R$, a KEM ciphertext used for authenticating to `Responder`, are sent along Message 1. While $\texttt{pk}_e$ serves as a replacement to ephemeral DH public share $G_x$, $\texttt{ct\_auth}_R$ is an additional field that serves for KEM-based authentication.

### 4.3.2 Changes in Message 2

**Pseudorandom key derivation.** In EDHOC, $\texttt{PRK}_{2e}$ is derived from $\texttt{TH}_2$ and $G_{xy}$, where $G_{xy}$ is an ephemeral DH shared secret and $\texttt{TH}_2$ is the transcript hash bounds to values computed from both `Initiator` and `Responder`, ensuring that the key derived from information generated at both involving sides. In KEMEDHOC, the principle of maximal information is still preserved. In particular, $\texttt{PRK}_{2e}$ is derived from $K_e$ and $H(\texttt{PRK}_{1e}, \texttt{TH}_2)$, where $K_e$ is the KEM shared secret key and $H(\texttt{PRK}_{1e}, \texttt{TH}_2)$ is the hash digest of $\texttt{TH}_2$ and pseudorandom key $\texttt{PRK}_{1e}$. $\texttt{PRK}_{1e}$ is extracted from $\texttt{K\_auth}_R$, which is a KEM shared secret key used for authenticating `Initiator` to `Responder`. As a result, in KEMEDHOC, $\texttt{PRK}_{2e}$ is extracted from both confidentiality and authentication materials computed at both involving parties. In addition, static-ephemeral DH share used as a key for deriving pseudorandom intermediate key is replaced by KEM shared secret for authentication. For example, $G_{rx}$ is replaced by $\texttt{K\_auth}_R$ in the derivation of $\texttt{PRK}_{3e2m}$.

**Cryptographic fields sent to `Initiator`.** The ephemeral public DH share $G_y$ is replaced by the ephemeral KEM ciphertext $\texttt{ct}_Y$ which is then decapsulated by the `Initiator` to get the ephemeral KEM shared key $K_e$. Additionally, a KEM ciphertext for authentication $\texttt{ct\_auth}_I$, is sent along the Message 2 for authenticating `Responder` to the `Initiator`.

### 4.3.3 Changes in Message 3

**`Initiator`'s credential is excluded in Message 3.** Since the credential, and its identifer, of `Initiator` ($\texttt{ID\_CRED}_I$ and $\texttt{CRED}_I$) are already sent along the Message 1, they are not necessarily, as in EDHOC design, included in plaintext 3 $\texttt{PTX}_3$ which then is encrypted and sent along Message 3. However, in order to have explicit

credential confirmation, $\texttt{ID\_CRED}_\texttt{I}$ is included in $\texttt{MAC}_3$. To save memory footprint, only credential identifier $\texttt{ID\_CRED}_\texttt{I}$, rather than the credential payload $\texttt{CRED}_\texttt{I}$, is included in MAC computation.

## 4.4 Security Properties

As KEMEDHOC is built upon the exsiting design of EDHOC, it guarantees security properties described in Subsection 2.1.4 except for the following security differences:

- **Identity protection:** In EDHOC, while the `Initiator`'s credential identifier is secure against an active attacker, the `Responder`'s credential identifier is secure against a passive attacker. In KEMEDHOC, the same protection level of parties' identity still remains. However, in the case that both `Initiator` and `Responder` have the other party's credential stored locally, credential identifiers of both parties are secure against an active attacker. As the `Responder` verifies the peer's credential at the first message before responding its credential identifier, an active attacker cannot construct a valid Message 1 if it does not have a legitimate credential stored locally at `Responder`. From the `Initiator`'s side, it send its credential identifier to a specific addresss, `Responder`'s address, in an encrypted form, so the identity is leaked if and only if the confidentiality is broken.

- **Denial of Service (DoS) mitigation:** This property is guaranteed only in the case that both `Initiator` and `Responder` have stored the other party's credential locally before establishing a handshake protocol. The `Responder` can verify the `Initiator`'s credential even at the first message. Hence, if the `Initiator` is not legitimate, indicated by the failed credential verification, the `Responder` can abort the session early. This avoids wasting computing resources on malicious requests.

- **Non-repudiation:** As KEMEDHOC does not use signature for authentication, similar to STAT-based authentication method in EDHOC, the non-repudiation regarding to both parties are not provided.

## 4.5 Security considerations

**Encrypted $\texttt{EAD}_1$.** The external authorization data $\texttt{EAD}_1$ is considered as an unprotected field in EDHOC. $\texttt{EAD}_1$ can be transmitted as plain or encrypted outside of EDHOC—at the application that uses this data. For example, this external data can be utilized for third-party-assisted authorization [Sel+25]. From the attacking perspective, observing the plain $\texttt{EAD}_1$ across sufficiently large amount of network traffic can identify a particular party through fingerprint analysis. In KEMEDHOC, as $\texttt{EAD}_1$ is encrypted along with plaintext 1 $\texttt{PTX}_1$, and the ciphertext looks different across protocol sessions, the attacking technique of fingerprint analysis does not work.

**Reuse $\texttt{K\_auth}_\texttt{R}$.** Moreover, in KEMEDHOC key schedule Subsection 4.2, $\texttt{K\_auth}_\texttt{R}$ are used as input keying material for extracting both $\texttt{PRK}_{\texttt{3e2m}}$ and $\texttt{PRK}_{\texttt{1e}}$. At the first look, someone can argue that the key reuse attack can be applied. However, as recommended in HKDF, as long as the key-related information, also known as HKDF Info, is uniquely and cryptographically bound to the input key material, the KDF is secure [Kra10]. To guarantee this uniqueness, each derived intermediate materials, including pseudorandom keys, initial vectors, encryption keys, and MAC tags, have their own incremental counter as embedded in HKDF Info. In addition, if different `salt` values are used in $\texttt{Extract}(\texttt{salt}, \texttt{IKM})$, even with the same input keying material `IKM`, the output keys are distinct. In particular, $\texttt{PRK}_{\texttt{3e2m}}$ and $\texttt{PRK}_{\texttt{1e}}$ are derived as follows:

$$\texttt{PRK}_{\texttt{1e}} \leftarrow \texttt{Extract}(\texttt{TH}_1, \texttt{K\_auth}_\texttt{R}),$$
$$\texttt{PRK}_{\texttt{3e2m}} \leftarrow \texttt{Extract}(\texttt{SALT}_{\texttt{3e2m}}, \texttt{K\_auth}_\texttt{R}),$$

where $\texttt{SALT}_{\texttt{3e2m}} \leftarrow \texttt{Expand}(\texttt{PRK}_{\texttt{2e}}, (4, \texttt{TH}_2, |\texttt{Hash}|), |\texttt{Hash}|)$ (4 is a unique label).

## 4.6 Differences from the existing competitors

As we mentioned in Subsection 1.9, KEM-5 EDHOC [Fra+25a] and KEM-3 EDHOC [Fra+25b] are two recent IETF drafts that have common properties with our design KEMEDHOC. This section outlines the differences between these two drafts and our design KEMEDHOC as follows.

**KEM-5 EDHOC.** The first draft, KEM-5 EDHOC, extends the message flow to 5 mandatory messages, and considers a general setting where neither party has the other party's credential stored locally beforehand. Given the general setting, the `Initiator` cannot straighforwardly use the long-term KEM public key ($\texttt{pk}_\texttt{R}$) of `Responder` to generate authentication materials in Message 1. Instead, the `Initiator` sends its ephemeral KEM public key ($\texttt{pk}_\texttt{e}$), chosen authentication method, cipher suite, connection ID ($\texttt{C}_\texttt{I}$), and optional $\texttt{EAD}_1$ in Message 1 as pure plaintext. Upon receiving Message 1, the `Responder` encapsulates $\texttt{pk}_\texttt{e}$ to obtain the ephemeral shared key ($\texttt{ss}_{\texttt{eph}}$), and derive an encryption key from $\texttt{ss}_{\texttt{eph}}$ to encrypt its credential and credential identifier in Message 2. Upon receiving Message 2, the `Initiator` decapsulates the authentication ciphertext to obtain the ephemeral shared key ($\texttt{ss}_{\texttt{eph}}$) and derive the same encryption key to decrypt `Responder`'s credential and credential identifier. After having `Responder`'s credential, the `Initiator` can then encapsulate `Responder`'s long-term KEM public key ($\texttt{pk}_\texttt{R}$) to obtain the authentication shared key ($\texttt{ss}_{\texttt{auth}}$).

In other words, the protocol flow of KEM-5 EDHOC is similar to EDHOC regarding the first three messages. The `Initiator` is able to authenticate to `Responder` at Message 2, and the `Responder` is able to authenticate to `Initiator` at Message 3. Hence, the identity of `Responder` is not protected against an *active* attacker as the attacker can send a request as in Message 1 to the `Responder` and then retrieve `Responder`'s credential in Message 2 without presenting its credential. On the other hand, KEMEDHOC and KEM-3 EDHOC consider a more specific setting where both

parties have the other party's credential in advance through pre-provisioning. In this setting, the `Initiator` can use `Responder`'s long-term KEM public key ($\texttt{pk}_\texttt{R}$) to generate authentication materials and include its encrypted credential even in Message 1. As a result, only legitimate `Initiator` can construct a valid Message 1 as only the legitimate `Initiator` has `Responder`'s credential locally.

**KEM-3 EDHOC.** The second draft, KEM-3 EDHOC, has the similar setting assumption and protocol flow as KEMEDHOC. However, there are several differences in the design details. First, in Message 1, KEM-3 EDHOC uses binary additive stream cipher to encrypt fields in Message 1, while KEMEDHOC uses AEAD encryption scheme. Both approaches can provide confidentiality. Regarding the integrity, while AEAD scheme provides the integrity check at the decryption phase, when the `Responder` receives Message 1, then stream cipher approach requires an additional MAC tag, $\texttt{MAC}_2$, to be verified when the `Initiator` receives Message 2. In other words, the tampered Message 1 is detected earlier in KEMEDHOC, which avoids wasting computing resources and mitigates Denial of Service (DoS) attack.

Second, the key schedule of KEM-3 EDHOC has major changes from EDHOC, while KEMEDHOC preserves most of the key schedule structure of EDHOC. In particular, $\texttt{PRK}_{\texttt{3e2m}}$ and $\texttt{PRK}_{\texttt{4e3m}}$ are replaced by a single $\texttt{PRK}_{\texttt{2e3e3m}}$ pseudorandom key in KEM-3 EDHOC. As a result, the scheme is less complex and requires less memory footprint compared to KEMEDHOC. Furthermore, this simplified key schedule scheme also removes the need for extracting $\texttt{K\_auth}_\texttt{R}$ twice, as in KEMEDHOC. The security of key schedule in KEM-3 EDHOC and KEMEDHOC are not comparable at this moment as they are not formally analyzed yet. In future work, computational security models, based on multi-stage key exchange (MSKE) [FG14], of these key schedule schemes can be constructed to formally analyze and compare their security properties in a rigorous manner.

# 5 Security Analysis

In this section, we describe the formal verification of KEMEDHOC at the design level. We first explain the general protocol model, followed by the property-specific queries we used to verify the security properties defined in Subsection 4.4. For readers who are looking for the implementation-level verification, please refer to Subsection 6.3.

The symbolic proof of KEMEDHOC uses the ProVerif [Bla13] prover, building upon the symbolic model of EDHOC [Jac+23], without any relaxed-idealization models of cryptographic primitives. Our ProVerif model covers the DoS mitigation which is an additional property in KEMEDHOC. Our model does not allow the random the number generator of honest parties are compromised by an attacker. `KEM.Encap` is probabilistic, whose security depends on high entropy of randomness, if the random nubmer generator is compromised, then the `KEM.Encap` is trivially broken, leading to the leakage of KEM shared keys. Regarding **Identity Protection**, we consider a *passive* attacker who can only eavesdrop the exchanged messages but cannot modify them. On the other hand, an *active* attacker can freely intercept and modify messages.

In summary, our ProVerif model proves that the security properties of EDHOC, except **Non-repudiation**, are also guaranteed by KEMEDHOC with addtional requirement on IND-CCA KEM. Note that all cryptographic schemes are considered perfectly functional in our ProVerif model.

## 5.1 Protocol Model

Following the symbolic proof of EDHOC, we also model KEMEDHOC in an environment that there are unbounded number of participants, both honest parties and attackers. As the ProVerif prover employs the Dolev-Yao [DY81] threat model, an attacker is able to freely intercept and modify messages except otherwise is explicitly defined. For example, the attacker can choose the authentication method, cipher suite, and derive its own ephemeral shares ($\texttt{sk}_\texttt{e}/\texttt{pk}_\texttt{e}$) to engage with an honest Responder. In general, there are 4 protocol **Run Cases (RCs)**:

- **[RC1]:** an honest `Initiator` `I` communicates with an honest `Responder` `R`. In this model, the `Initiator` and the `Responder` are different entities, carrying on different credential identities.
- **[RC2]:** an attacker impersonates a `Responder` and engage with an honest `Initiator`.
- **[RC3]:** an attacker acts as an `Initiator` and initiate a connection with an honest `Responder`.
- **[RC4]:** a single party play both roles of `Initiator` and `Responder` to simulate the replay, or selfie, attack in which an honest party communicates to itself. The attack is successful if the party cannot notice that it is communicating to itself.

If the party notice the attack then it ends the protocol run early and optionally returns an error code.

**[RC1-RC3].** Based on the ProVerif model of EDHOC [Jac+23], we model KEMEDHOC protocol run that reflects run cases **[RC1-RC3]** with public authentication KEM key as an additional credential material for each involving party. Listing 12 shows that credentials of `Initiator` and `Responder` are revealed to an attacker through **`out`** keyword. Events `eShareLT,eHonest` are used to record that the input credentials are valid, generated by honest parties. This marks the distinction from credentials generated by an attacker through the keyword **`in`**. The acclaimation **`!`** denotes the replication of unbounded number of processes. Here, we model an arbitrary number of participants engaging to each other, to both an honest party or an attacker, as either an `Initiator` or a `Responder`. Moreover, those replicated sessions run in parallel, denoted by the vertical bar |.

In addition, Component processes `I(...), R(...)` represent the `Initiator`'s and `Responder`'s protocol runs respectively. **`phase`** `1` spawns a separate session, which will be activated after other sessions finish. In that post-completion phase, we let long-term, static, authentication keys of `Initiator` and `Responder` compromised to an attacker, modelling the post-compromise scenario which will be proved that the compromise of those long-term authentication keys does not reveal the shared keys established in sessions happened before the compromise.

```
process
    (*Create Initiator's credentials*)
    new computerId_1:bitstring;
    new sk_1:bitstring;
    new sk_auth_kem_1:bitstring;
    new ltdh_1:bitstring;
    let idd_1:bitstring = id(pk(sk_1), ltdh_1, kempk(sk_auth_kem_1)) in

    (*Reveal Initiator's credential to the attacker*)
    out(att, (idd_1, (pk(sk_1), ltdh_1, kempk(sk_auth_kem_1))));
    event eShareLT(ltdh_1);
    event eHonest(pk(sk_1));
    event eHonest(kempk(sk_auth_kem_1));

    (*Create Responder's credentials*)
    new sk_2:bitstring;
    new sk_auth_kem_2:bitstring;
    new ltdh_2:bitstring;
    new computerId_2:bitstring;
    let idd_2:bitstring=id(pk(sk_2), ltdh_2, kempk(sk_auth_kem_2)) in

    (*Reveal Responder's credential to the attacker*)
    out(att,(idd_2, (pk(sk_2), ltdh_2, kempk(sk_auth_kem_2))));
    event eShareLT(ltdh_2);
    event eHonest( pk(sk_2) );
    event eHonest( kempk(sk_auth_kem_2) );

    (!((*Session simulates that an honest Initiator communicates with an credential chosen by an attacker,
        it can be valid credential of Responder or an arbitrary credential*)
        in(att, cred_2:bitstring);
        I(computerId_1, sk_1, ltdh_1, sk_auth_kem_1, cred_2)
      ) | !(
        R(computerId_2, sk_2, ltdh_2, sk_auth_kem_2)
      ) | ( (*Session simulates post-compromise*)
        phase 1; event eCompromise(kempk(sk_auth_kem_1));
        event eCompromise(kempk(sk_auth_kem_2));
        out(att, sk_auth_kem_1); out(att, sk_auth_kem_2)))
```

**Listing 12:** KEMEDHOC protocol run model for **[RC1-RC4]**.

**[RC4].** To model the *Reflection attack*, we let participants act as both `Initiator` and `Responder` and engage with each other, including themselves, in an unbounded

number of sessions. Listing 13 shows that credentials of the two honest parties are given into both `I()`, `R()` component processes. Moreover, we again utilize post-completion phase, **`phase`** 1, to model the post-compromise scenario.

```
(* Simulate the Reflection attack when the Initiator establish with itself
    instead of an Honest Responder *)
!(I(computerId_1, sk_1, ltdh_1, sk_auth_kem_1, idd_2))
| !(R(computerId_2, sk_2, ltdh_2, sk_auth_kem_2))
| !(I(computerId_2, sk_2, ltdh_2, sk_auth_kem_2, idd_1))
| !(R(computerId_1, sk_1, ltdh_1, sk_auth_kem_1))
| (phase 1; event eCompromise(kempk(sk_auth_kem_1));
    event eCompromise(kempk(sk_auth_kem_2));
    out(att, sk_auth_kem_1); out(att, sk_auth_kem_2))
```

**Listing 13:** KEMEDHOC protocol run model for **[C4]**.

### 5.1.1 Cryptographic primitives modelling

KEMEDHOC is constructed based on the following cryptographic primitives: AEAD, MAC, HKDF, and KEM. While AEAD, MAC, and HKDF construction are given in ProVerif manual [Bla+23] and ProVerif model of EDHOC [Jac+23], KEM construction is added in our model.

**KEM.** The KEM construction is based on the following functions: KEM encapsulation `kemencap`, KEM decapsulation `kemdecap`, KEM shared key generation `kemkey`, KEM ciphertext generation `kemcipher`, KEM public key extraction `kempk`. Note that the KEM encapsulation is a probabilistic algorithm that takes a random seed and returns two outputs, namely ciphertext and shared key. As ProVerif does not support multiple-value return, we model the extraction of the two outputs through the two separate functions, `kemkey` and `kemcipher`, that takes an encapsulation result as input and returns the corresponding ciphertext and shared key.

We model the functional correctness of KEM by defining the rewritting rule stating that the KEM decapsulation will generate the same shared key produced by the KEM encapsulation given the same ciphertext and the public/private key pair. Listing 14 illustrates the KEM construction and rewritting rules in our model.

```
(* KEM Decapsulation gets KEM ciphertext and decap (secret) key as inputs and return the shared
    key *)
fun kemdecap(bitstring,bitstring):bitstring.
(* KEM Encapsulation gets random seed and encap (public) key as inputs *)
fun kemencap(bitstring,bitstring):bitstring.
(* KEM Key gets the KEM encapsulation result as input and return the shared key *)
fun kemkey(bitstring):bitstring.
(* KEM Ciphertext gets the KEM encapsulation result as input and return the ciphertext *)
fun kemcipher(bitstring):bitstring.
(* Return the corresponding public key given a private key *)
fun kempk(bitstring):bitstring.

(*Reduction rule*)
equation forall r:bitstring, sk:bitstring; kemdecap(kemcipher(kemencap(r, kempk(sk))), sk)
    = kemkey(kemencap(r, kempk(sk))).
```

**Listing 14:** KEM construction and rewritting rule in ProVerif.

### 5.1.2 Protocol run modelling

The main process of KEMEDHOC consists of two role processes defining how `Initiator`, and `Responder`, construct messages locally and send to the other party. This section explains the main differences between our model and EDHOC model [Jac+23]. In particular, this section introduces the changes on how both parties constructs and processes the first Message 1 ($\texttt{Msg}_1$). The remaining changes in $\texttt{Msg}_2$ and $\texttt{Msg}_3$ follow modification which is clarified in Subsection 4.3.

**`Initiator`'s process.** Listing 15 shows how the `Initiator` constructs the first message, $\texttt{Msg}_1$. The process `I(...)` is given the `Initiator`'s credential and the `Responder`'s credential `ID_CRED_R_2` as inputs. Note that the `ID_CRED_R_2` can be either a legitimate payload or attacker-forged credential as described in Listing 12. The `Initiator` selects cipher suite (`suitesI_2`), and authentication method (`method_2`), which are even freely chosen by an attacker, marked by the keyword **`in`** through the public channel **`att`**. This exposure simulates the unencrypted fields in Message 1. Indicated by **`new`** keyword, an ephemeral private KEM key, a random seed for KEM encapsulation, and optional external authorization data $\texttt{EAD}_1$ (`EAD_1_2`) are generated by the `Initiator`. They are fresh random values which are unknown to an attacker.

Next, the `Initiator` checks if the `Responder`'s credential is identical to its credential, preventing the *Replay attack*. Then, it encapsulates the KEM public key of `Responder` `pkR_2`, resulting in KEM ciphertext `CT_AUTH_R_2` and KEM shared key `K_AUTH_R_2`. The **`event`** `eHonestIInit (pkI_2, pkR_2)` used to specify that the `Initiator` has stored a credential that is not identical to itself, which is useful for verification queries discussed in the following section. The **`event`** does not affect the execution of the process, it serves solely as an annotation that snapshots a set of given values. Finally, the `Initiator` derives encryption key `K_1_2` and encrypts the plaintext 1 `(plaintext_1_2)`. The Message 1 `(m1_2)` is sent, being exposed to an attacker, through the keyword **`out`**.

```
let I(cid_2:bitstring, skI_2:bitstring, I_2:bitstring, sk_kem_authI:bitstring,
      ID_CRED_R_2:bitstring) =
    in(att,(method_2:bitstring, (suitesI_2:bitstring)));
    new skX_2:bitstring; (*secret ephemeral KEM key*)
    new C_I_2:bitstring; (*connection identifier of Initiator*)
    new random_authR:bitstring; (*random bits for KEM authentication R*)
    new EAD_1_2:bitstring; event eShare( X_2 );
    (( let CRED_I_2:bitstring = pk(skI_2) in (*For signature-based authentication*)
      let KEM_CRED_I_2:bitstring = kempk(sk_kem_authI) in (*For KEM-based authentication*)
      let ID_CRED_I_2:bitstring = id(CRED_I_2, I_2, KEM_CRED_I_2) in
      if ID_CRED_I_2 <> check_cred(ID_CRED_R_2) then
          (if method_2 = method_four then
              ((* KEM-KEM *)
                let pkR_2:bitstring = get_kem_auth(ID_CRED_R_2) in
                let encap_auth_R_2:bitstring = kemencap(random_authR, pkR_2) in
                let K_AUTH_R_2:bitstring = kemkey(encap_auth_R_2) in
                let CT_AUTH_R_2:bitstring = kemcipher(encap_auth_R_2) in
                let plaintext_1_2:bitstring = (C_I_2, ID_CRED_I_2, EAD_1_2, KEM_CRED_I_2) in
                let pkI_2:bitstring = get_kem_auth(ID_CRED_I_2) in
                let pkX_2:bitstring = kempk(skX_2) in (*public KEM key*)
                event eHonestIInit(pkI_2, pkR_2);
                let TH_1_2:bitstring = hash((pkX_2, CT_AUTH_R_2)) in
                let PRK_1e_2:bitstring = hkdfextract(TH_1_2, K_AUTH_R_2) in
                let K_1_2:bitstring = edhoc_kdf(PRK_1e_2, szero, TH_1_2, key_length) in
                let IV_1_2:bitstring = edhoc_kdf(PRK_1e_2, sone, TH_1_2, iv_length) in
                let CIPHERTEXT_1_2:bitstring = aeadenc(plaintext_1_2, srep, K_1_2, IV_1_2) in
                let m1_2:bitstring = (method_2, suitesI_2, pkX_2, CT_AUTH_R_2, CIPHERTEXT_1_2, EAD_1_2) in
                out(att,m1_2); (* Send Message 1*)
                ...
```

**Listing 15:** $\texttt{Msg}_1$ composing steps extracted from the process that models `Initiator`.

**Responder's process.** Listing 16 illustrates how the Responder processes the Message 1. Firstly, the Responder receives the Message 1 (m1_2), connection ID C_R_2, $EAD_2$ (EAD_2_2), and Initiator-selected cipher suite suitesR_2 from the public channel **att**, meaning that these values can be modified by an attacker. The Responder decapsulates the ciphertext CT_AUTH_R_2 using its KEM long-term private key sk_kem_authR_2, reconstructing the KEM shared authentication key K_AUTH_R_2. The Responder then derives the encryption key K_1_2 and decrypts the plaintext plaintext_1_2, extracting the Initiator's credential.

Next, the Responder checks whether the extracted Initiator's credential is identical to its credential, preventing the *Replay attack*. Then, the Responder freshly generates the random seed r_2 for performing KEM encapsulation to the ephemeral KEM public key of Inititator pkX_2, resulting ciphertext CT_Y_2 and the ephemeral shared key K_XY_2. The Responder randomly generates another random seed for the encapsulation of long-term KEM authentication key of Inititator pkI_2. The encapsulation results in the corresponding ciphertext CT_AUTH_I_2 and the shared key K_AUTH_I_2. Then, Responder proceeds to compose Message 2.

```
let R(cid_2:bitstring, skR_2:bitstring, R_2:bitstring, sk_kem_authR_2:bitstring) =
    (* Set up parameters *)
    in(att,(C_R_2:bitstring, (EAD_2_2:bitstring, suitesR_2:bitstring)));
    in(att, m1_2:bitstring);     (* Receive Message 1 *)
    let (method_2:bitstring, suitesI_2:bitstring, pkX_2:bitstring, C_I_2:bitstring,
        CT_AUTH_R_2:bitstring, CIPHERTEXT_1_2:bitstring) = m1_2 in
    ((* set up credentials of Responder *)
      let CRED_R_2:bitstring = pk(skR_2) in
      let KEM_CRED_R_2:bitstring = kempk(sk_kem_authR_2) in
      let ID_CRED_R_2:bitstring = id(CRED_R_2, G_R_2, KEM_CRED_R_2) in

      if method_2 = method_four then (
            (* KEM-KEM *)
            event eMethodOk( method_2 );

            let K_AUTH_R_2:bitstring = kemdecap(CT_AUTH_R_2, sk_kem_authR_2) in
            (* set up key for ciphertext_1 *)
            let TH_1_2:bitstring = hash((pkX_2, CT_AUTH_R_2)) in
            let PRK_1e_2:bitstring = hkdfextract(TH_1_2, K_AUTH_R_2) in
            let K_1_2:bitstring = edhoc_kdf(PRK_1e_2, szero, TH_1_2, key_length) in
            let IV_1_2:bitstring = edhoc_kdf(PRK_1e_2, sone, TH_1_2, iv_length) in
            let plaintext_1_2:bitstring = aeaddec(CIPHERTEXT_1_2, K_1_2, IV_1_2) in

            let (ID_CRED_I_2:bitstring, EAD_1_2:bitstring, KEM_CRED_I_2:bitstring) = plaintext_1_2 in
            (* Check if the CRED_I is stored in the local storage of Responder *)
            if ID_CRED_R_2 <> check_cred(ID_CRED_I_2) then (
                new r_2:bitstring; (*random bits for KEM encapsulation*)
                let G_R_2:bitstring = R_2 in
                let encapsulation:bitstring = kemencap(r_2, pkX_2) in
                let CT_Y_2:bitstring = kemcipher(encapsulation) in
                let K_XY_2:bitstring = kemkey(encapsulation) in

                let pkR_2:bitstring = get_kem_auth(ID_CRED_R_2) in
                let pkI_2:bitstring = get_kem_auth(ID_CRED_I_2) in

                new encap_auth_random_seed:bitstring;
                let encap_auth_I_2:bitstring = kemencap(encap_auth_random_seed, pkI_2) in
                let K_AUTH_I_2:bitstring = kemkey(encap_auth_I_2) in
                let CT_AUTH_I_2:bitstring = kemcipher(encap_auth_I_2) in
                ...
```

**Listing 16:** $Msg_1$ processing steps extracted from the process that models Responder.

## 5.2 Verification queries

This section introduces the events and queries that we used to verify the security properties of KEMEDHOC.

### 5.2.1 Events

In ProVerif, properties of a protocol are verified by reasoning the event activation and their logical relationship. In order to help readers easily follow the chain of events triggered during the protocol run, we illustrate the series of events, corresponding to `Initiator` and `Responder`, in Figure 11.

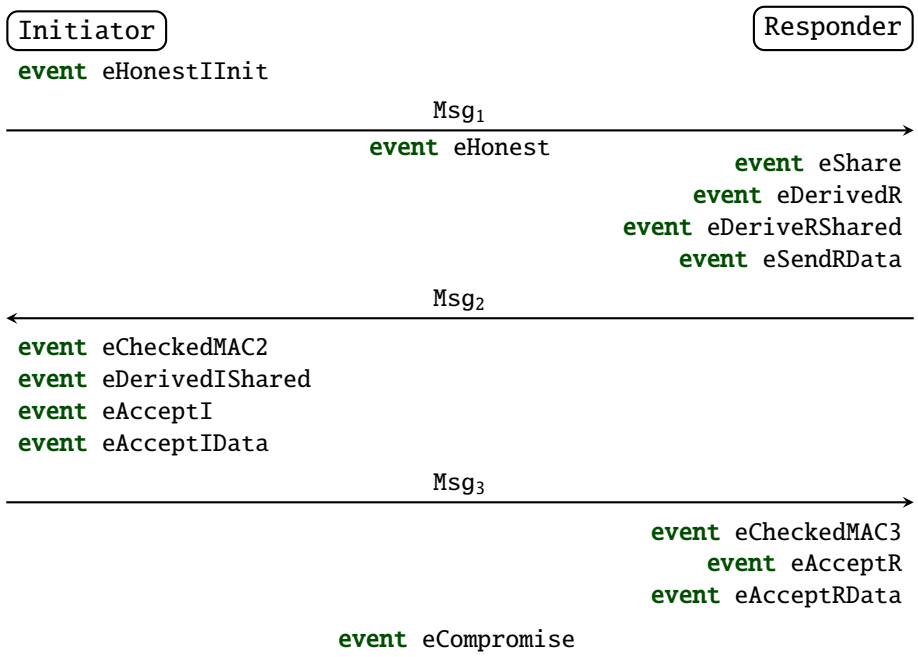



**Figure 11:** `event` flow of KEMEDHOC.

### 5.2.2 [SR1] Mutual authentication

```
query cid1:bitstring, cid2:bitstring, method:bitstring, pkI:bitstring, pkR:bitstring,
    prk3e2m:bitstring, prk4e3m:bitstring, prk_out:bitstring, x:bitstring, g_x:bitstring,
    y:bitstring, g_y:bitstring, t:time, i:time, k:time;
    inj-event(eAcceptR(cid1, method, pkI, pkR, prk3e2m, prk4e3m, prk_out, y, g_x))@i
    && event(eHonest(pkI))@k
    ==> (inj-event(eAcceptI(cid2, method, pkI, pkR, prk3e2m, prk4e3m, prk_out, x, g_y))@t && (t
     < i)) ||
        (event(eCompromise(pkI))@t && (t < i)). (*Key Compromise Impersonation KCI*)
```

**Listing 17:** Authentication query from the `Initiator`'s side.

The query shown in Listing 17 means that if the `Responder`, identified by `pkR`, approves a protocol session with a legitimate `Initiator`, identified by `pkI`, producing the shared session key `prk_out`, then the `Initiator` had derived the same shared key `prk_out` before, or the `Initiator` had compromised its long-term authentication key before. The injective events, denoted by **`inj-event`**, are events that happen only once during an arbitrary number of sessions. Those injective events guarantee that both `Initiator` and `Responder` authenticates each other only once for each completed session. The query regarding the `Responder`'s side follows the same pattern of events as shown in Listing 17.

*Key Compromise Impersonation (KCI)*. The query shown in Listing 17 also implies the security property against *KCI* since an attacker can compromise the `Responder`, identified by by `pkR`, without violating the query.

*Unknown Key Share (UKS)*. Two events `eAcceptI` and `eAcceptR` includes credentials of both local and remote party, `pkI` and `pkR`, and other intermediate, session-dependent, values generated during the session, guaranteeing that both parties share the same view of who they are communicating with.

*Reflection Attack*. Given the protocol run model in Listing 13, the above query, and its variant regarding the `Responder`'s side, are still true. The protocol is resistant to the *Reflection Attack* as each party locally check whether the retrieved credential from the remote party is identical to its own credential. This check is modeled as a `check_cred (id_cred)` process in our ProVerif model.

The local credential check is able to mitigate the *Reflection Attack*, but it unintentionally leaks the privacy of parties. In particular, the early termination of the protocol run provides an *active* with an oracle to determine whether the `Responder` and `Initiator` possess a common identity. However, in the Trust On First Use (TOFU) settings, parties are willing to exchange to each other and several connections can be established to the same identity without raising suspicion. The security of this unauthenticated operating mode is guaranteed by external verification, or the secure proximity-based communication [SMP24]. Thus, this privacy leakage is acceptable in the TOFU settings.

### 5.2.3 [SR2] Confidentiality

```
query cid:bitstring, method:bitstring, pkI:bitstring, pkR:bitstring,
    prk3e2m:bitstring, prk4e3m:bitstring, prk_out:bitstring,
    y:bitstring, g_x:bitstring, t:time, i:time, k:time, j:time;
    event(eAcceptR(cid, method, pkI, pkR, prk3e2m, prk4e3m, prk_out,
    y, g_x))@i && event(eHonest(pkI))@j && attacker(prk_out)@k ==>
    (event(eCompromise(pkI))@t && (t < i)) || (*Forward Secrecy*)
    (event(eLeakSessionKey(prk_out))@t) (*Session Key Independence*)
```

**Listing 18:** Confidentiality query for `Initiator`'s side.

The query shown in Listing 18 states that if the `Responder`, represented by `pkR`, approves a protocol session with a legitimate `Initiator`, represented by `pkI`, and the shared secret key is reachable to the attacker `attacker(prk_out)`, then the long-term authentication key of `Initiator` was previously leaked, or the shared session key was previously compromised. The secrecy property for `Responder` is stated in the similar pattern.

*Session key secrecy*. The query in Listing 18 proves that the established session key is compromised if and only if either `Responder` or `Initiator` was previously leaked or the key was intentionally compromised. In other words, the attacker cannot reproduce the shared session key by observing and interfering messages exchanging between `Initiator` and `Responder`.

*Forward secrecy*. As shown in Listing 18, only the compromise of the long-term authentication key happens previously can lead to the leakage of a shared session key.

In other words, if the shared session key established before the compromise of the long-term authentication key, the shared secret is still unreachable to an attacker.

*Session key independence*. The query above considers the leakage of the shared key at the current session, timestamp `t`, `eLeakSessionKey(prk_out)@t`. The compromise of shared keys from other sessions does not affect the shared secret of the current session since ephemeral shares are recomputed at each session.

```
query cid:bitstring, method:bitstring, pkI:bitstring, pkR:bitstring,
    prk3e2m:bitstring, prk4e3m:bitstring, prk_out:bitstring,
    x1:bitstring, x2:bitstring, g_y1:bitstring, g_y2:bitstring,
    i:time, j:time;
    event(eAcceptI(cid, method, pkI, pkR, prk3e2m, prk4e3m, prk_out,
        x1, g_y1))@i &&
    event(eAcceptI(cid, method, pkI, pkR, prk3e2m, prk4e3m, prk_out,
        x2, g_y2))@j ==> i = j.
```

**Listing 19:** Session key uniqueness query for `Initiator`'s side.

*Session key uniqueness*. The query shown in Listing 19 states that if the `Initiator` accepts the same shared session key `prk_out`, and other intermediate materials, at two timestamps (`i` and `j`), then the two recorded timestamps are identical. Intuitively, the query prove that shared session keys are different across protocol sessions.

### 5.2.4 [SR3] Identity Protection

```
!( in(att, cred_2:bitstring);
I(computerId_1, sk_1, ltdh_1, sk_auth_kem_1, cred_2))
| !( in(att, cred_3:bitstring);
I(computerId_2, sk_2, ltdh_2, sk_auth_kem_2, cred_3))
| !(R(computerId_1, sk_1, ltdh_1, sk_auth_kem_1))
| !(R(computerId_2, sk_2, ltdh_2, sk_auth_kem_2))
(* Checking the indistinguishability *)
| !(I(choice[computerId_1, computerId_2], choice[sk_1, sk_2],
    choice[ltdh_1, ltdh_2], choice[sk_auth_kem_1, sk_auth_kem_2], idd_2))
```

**Listing 20:** Obseravational equivalence model for verifying **Identity Protection**.

We model the **Identity Protection** by verifying the observational equivalence between processes. If the processes are observationally equivalent, which have the same pattern but only differ in the selection of terms, the identity of involving parties is not distinguishable to an attacker. In ProVerif, the observational equivalence check is defined as $\mathtt{P_A} \approx \mathtt{P_B}$. In ProVerif, the keyword **`choice`** is utilized to ask for observational equivalence verification. For example, to verify $\mathtt{P}(\mathtt{v}_1) \approx \mathtt{P}(\mathtt{v}_2)$, an user needs to write `P(choice[v1, v2])` in their model.

The processes shown in Listing 20 allows the two pairs of Initiator-Responder processes contacting to each other. Note that, in our model, the `Initiator` can retrieve both valid and invalid credential, which chosen by an attcker, and the `Responder` will responds whoever contacts it. Those two pairs of processes, 4 single processes, serve as the reference processes for the test process stated as `I(choice[...], choice[...], choice[...], choice[...])`. This test process checks whether the attacker can distinguish the `Initiator` process `I(...)` given two sets of input. Since only the Inititator's credential is protected against an *active* attacker in general, claimed in Subsection 4.4, only the `Initiator` process `I(...)` is verified.

**Identity Protection** is fully satisfied if and only if both parties' credentials are secure against both *passive* and *active* attackers. `Initiator`'s credential is protected against both two models of attacker in KEMEDHOC, and EDHOC, whereas `Responder`'s credential is secure against only the *passive* attacker in the general case, where the two parties need to exchange their credentials on the fly even before verifying the received credential. However, as KEMEDHOC assumes that the two parties have each other's valid credential before the protocol run, `Responder`'s credential is also protected against an *active* attacker in this specific case. Hence, we consider **Identity Protection** is completely achieved in KEMEDHOC.

### 5.2.5 [SR4] Cryptographic strength

KEMEDHOC re-utilizes AEAD schemes, which are at least 128-bit cryptographically strong, used in EDHOC for authenticated encryption. The supported AEAD suites: AES-CCM-16–64–128, AES-CCM-16–128–128, AES-128-GCM, AES-256-GCM, and ChaCha20-Poly1305.

### 5.2.6 [SR5] Protection of external security data

As shown in Figure 9, EAD values across all messages, namely $\texttt{EAD}_1$, $\texttt{EAD}_2$, $\texttt{EAD}_3$, $\texttt{EAD}_4$, are protected by AEAD encryption which provides both confidentiality and integrity. However, as explicitly stated in RFC 9528[SMP24], EAD should be considered unprotected by the protocol due to many reasons. Firstly, in KEMEDHOC, although $\texttt{EAD}_1$ and $\texttt{EAD}_2$ are encrypted, both parties, until Message 2, still do not guarantee the mutual authentication due to the lack of explicit key confirmation and credential confirmation. As a result, $\texttt{EAD}_1$ and $\texttt{EAD}_2$ are secure against a *passive* attacker but not an *active* attacker who can freely modify messages. On the other hand, $\texttt{EAD}_3$ and $\texttt{EAD}_4$, are encrypted after both parties establish credential confirmation in Message 3 and explicit key confirmation in Message 4.

Hence, within the scope of a single protocol run, these EADs are secure against *passive* and *active* attackers, depending on the message they are included in. However, note that these EADs can be used by applications even before the completion of message verification, which verifies MAC tags.

In summary, although EADs are well-protected within the protocol, application developers need to carefully consider the security implications of using EADs before the completion of protocol run and in other third-party services.

### 5.2.7 [SR6] Downgrade protection

```
query ead1:bitstring, ead2:bitstring, ead3:bitstring, suiteI:bitstring, method:bitstring,
    pkI:bitstring, pkR:bitstring, th1:bitstring, th2:bitstring, th3:bitstring, th4:bitstring,
    m1:bitstring, plaintext1:bitstring, plaintext2:bitstring, plaintext3:bitstring,
    prk_out:bitstring, x:bitstring, y:bitstring, g_x:bitstring, g_y:bitstring,
    i:time, t:time, k:time;
    inj-event(eAcceptRData(prk_out, method, pkI, pkR, y, g_x,
            (th1, th2, (th3, (th4, (suiteI, (ead1, (ead2, (ead3, (m1, (plaintext1, plaintext2,
     plaintext3))))))))))@i
    && event(eHonest(pkI))@k
    ==> (inj-event(eAcceptIData(prk_out, method, pkI, pkR, x, g_y,
            (th1, th2, (th3, (th4, (suiteI, (ead1, (ead2, (ead3, (m1, (plaintext1, plaintext2,
     plaintext3))))))))))@t
        && (t < i)) ||
        (event(eCompromise(pkI))@t && (t < i)).
```

**Listing 21:** Data authentication and integrity query for `Initiator`'s side.

The query shown Listing 21 states that if the `Responder` accepts all intermediate keys, transcript hashes, authentication method, cipher suite, plaintexts, External Authorization Data (EAD), shared session key `prk_out` and communicates with a legitimate `Initiator`, identified by `pkI`, then the `Initiator` already approved the same values or the `Initiator` was compromised previously. The query for `Responder`'s side is constructed in the same pattern. The query proves that both involving parties have the same set of values, including the chosen cipher suite, after finishing the handshake session, preventing against the *Downgrade* attack.

### 5.2.8 [SR7] Non-repudiation

Similar to static-DH-based authentication methods in EDHOC, KEMEDHOC uses MAC tag only for message authentication. It means that a party is not able to prove that it has not participated in the protocol as both parties can compute the same MAC tag. Therefore, KEMEDHOC does not provide non-repudiation.

### 5.2.9 DoS Mitigation

```
(** Tables for storing Initiator and Responder's credentials*)
table local_cred_I(bitstring).
table local_cred_R(bitstring).
event eHonestIInit(bitstring,bitstring).
...
process
    ...
    insert local_cred_I(idd_2);
    insert local_cred_R(idd_1);
    ...
(*DoS Mitigation query*)
query cid:bitstring, pkI:bitstring, pkR:bitstring, g_x:bitstring, x:bitstring, y:bitstring,
      prk3e2m:bitstring, i:time, t:time, j:time;
    event(eDerivedR(cid, pkI, pkR, prk3e2m, y, g_x))@i
    ==> (event(eHonestIInit(pkI,pkR))@j && j<i) || (event(eCompromise(pkR))@t && t<i).
```

**Listing 22:** DoS mitigation query.

We introduces local databases, **`table`**, to store legitimate credentials on `Initiator` and `Responder`. Listing 22 shows the protocol run that is based on the model shown

in Listing 12. In addition, we `insert` legitimate credentials to the newly-created local databases, modelling that each party has stored the legitimate credential of the other peer. We assume that the credential of `Initiator` is not compromised in the initial phase, `phase 0`, by disallowing `out` to `idd1`. The credentials of `Responder` and `Initiator` are compromised in the later phase, `phase 1`, to model the situation of post-compromise. The event `eHonestIInit(...)` is triggered only if `Initiator` has sent the first message to the legitimate `Responder` whose credential is stored on the `Initiator`. This helps reason the scenario that the `Responder` replies the message which is intentionally sent to it, not to other peers.

The query shown in Listing 22 states that if the `Responder` generates an ephemeral secret `y` given the ephemeral public share from the `Initiator` `g_x`, then the `Initiator` is legitimate and intentionally connects to the `Responder`, or the `Responder`'s static authentication key was leaked before. In other words, the `Responder` only derives ephemeral keys if and only if the `Initiator` is legitimate, having the credential stored locally in `Responder`, or the `Responder` itself was compromised before. As as result, the `Responder` aborts the session early, immediately in processing the Message 1, and does not waste resources on local computation, which significantly mitigates the DoS attacks.

Note that this security property is hold only in the case `Initiator`'s and `Responder`'s credentials are pre-provisioned to each other through a trusted channel, or onboarding, before they conduct a handshake session. Otherwise, in which the `Initiator` transmit its credential along with identity in the first message, the `Responder` does not have a straighforward approach, without contacting the trusted third-party, to verify `Initiator`'s credential.

**EDHOC's behaviour to DoS.** In EDHOC, the credential of `Initiator` is included in the Message 3; therefore, the `Responder` cannot verify the `Initiator` until retrieving the Message 3. Regardless of whether the `Initiator` is legitimate or not, the `Responder` has to compute and derive a bunch of values to construct the Message 2. This means that the `Responder` will consume a significant amount of resources for responding malicious requests.

## 5.3 Summary

Security properties of KEMEDHOC are briefly summarized in Table 13. Overall, KEMEDHOC satisfies security properties that are claimed in EDHOC. Additionally, KEMEDHOC provides a more efficient DoS mitigation, early terminating the session with a malicious `Initiator`, in a situation in which Respoder has stored a legitimate `Initiator`'s credential locally prior to the handshake session.

Our experiments were perfomed on a commercial laptop, equipped with 12 core 12th Gen Intel® i7–1260P @ 3.40 GHz and 16GB of RAM. ProVerif version 2.05 was used for the verification. The ProVerif verification process took 12 minutes to complete. Each verification query generates averagely 1460 Horn clauses.

| | [SR1] | [SR2] | [SR3] | [SR4] | [SR5] | [SR6] | [SR7] | DM | QR |
|---|---|---|---|---|---|---|---|---|---|
| **EDHOC** | ● | ● | ◐ | ● | ● | ● | ◐ | ○ | ○ |
| **KEMEDHOC** | ● | ● | ● | ● | ● | ● | ○ | ● | ● |
| **PQ-EDHOC** | ● | ● | ◐ | ● | ● | ● | ◐ | ○ | ● |
| **KEM-5 EDHOC** | ● | ● | ◐ | ● | ● | ● | ○ | ○ | ● |
| **KEM-3 EDHOC** | ● | ● | ● | ● | ● | ● | ○ | ● | ● |

●: claim satisfied ◐: claim satisfied in specific cases ○: violation of claim

**Table 13:** Security properties comparision among EDHOC, KEMEDHOC, PQ-EDHOC [Fra+25c], KEM-5 EDHOC [Fra+25a], and KEM-3 EDHOC [Fra+25b]. **[SR1]**: Mutual Authentication; **[SR2]**: Confidentiality; **[SR3]**: Identity Protection; **[SR4]**: Cryptographic Strength; **[SR5]**: Protection of External Security Data; **[SR6]**: Downgrade Protection; **[SR7]**: Non-repudiation. **DM**: DoS Mitigation; **QR**: Quantum Resistance. **[SR3]** is considered fully satisfied (●) if parties' credentials are protected against both *passive* and *active* attackers. Only signature-based authentication methods in EDHOC and PQ-EDHOC provide **[SR7]**.

# 6 Implementation

In this section, we present the verified implementation of EDHOC and KEMEDHOC, called EDHOC* and KEMEDHOC*, respectively. As KEMEDHOC is built on top of EDHOC, we define verified modules in EDHOC* first, and then build KEMEDHOC* on top of them.

The verifed implementations, EDHOC* and KEMEDHOC*, guarantee the following properties:

- **Functional correctness.**: The implementations conform to the protocol specifications, including the key derivation and message construction procedures. This is ensured by formalizing the protocol specifications in F*, and proving that the Low* implementations are functionally equivalent to the F* specifications.
- **Memory safety.**: The implementations are free from memory-related vulnerabilities, such as buffer overflows, use-after-free, and null pointer dereferences. This is achieved by implementing the protocols in Low*, which provides reasoning models for memory operations, e.g. buffer allocation, deallocation, and data modification.
- **Side-channel resistance.**: The implementations are resistant to side-channel attacks, such as timing attacks. Even if an attacker has access to the execution time or memory access patterns, they cannot distinguish between two runs that differ only in secret data. This is ensured by utilizing the `bytes_l SEC` secret byte type as the default type for any sequence-based data. This type represents constant-time integers. If a well-type function receives a secret abstract type as input, such as a secret integer, then its output should remain independent of that secret input, even if low-level event traces revealing the outcomes of all execution branches are exposed to an attacker. We refer readers to [Zin+17] for more details about the secrecy independence property provided by `bytes_l SEC`.

**CBOR encoding/decoding.** The protocol messages in EDHOC and KEMEDHOC are encoded using CBOR [BH20]. However, due to time constraints, we did not formally verify the CBOR encoding/decoding component. Instead, our verified implementations, EDHOC* and KEMEDHOC*, rely on the correctness of an unverified CBOR library (zcbor) for encoding and decoding CBOR messages. We acknowledge that this introduces a potential source of vulnerabilities, as any bugs in the CBOR library could compromise the security of the protocol implementations. However, we place this shortage outside the scope of this thesis and leave it as future work.

**Trusted Computing Base (TCB).** Our implementations are verified under the assumption that the F* typechecker, Z3 SMT solver, and the KaRaMel compiler function correctly. In addition, a user can compiler the C code generated by KaRaMel using the verifed C compiler CompCert [Ler09] or mainstream C compiler, such as GCC or Clang, with additional cost to TCB. Finally, we assume that the CBOR

encoding/decoding works as expected and a secure key-provision infrastucture that securely pre-provision static keys is deployed by the device manufacturer.

## 6.1 Architecture

The implementation architecture of EDHOC*, and KEMEDHOC* is shown in Figure 12. The protocol implementation consists of three main components: (1) **Protocol Core**, (2) **Key Schedule**, and (3) **Crypto Primitives**. The **Protocol Core** implements the protocol logic (illustrated in Figure 2 and Figure 9), including message construction and parsing, as well as machine state management. The **Key Schedule** implements the key derivation functions as specified in the EDHOC and KEMEDHOC key schedule specifications, illustrated in Figure 3 and Figure 10 respectively. The **Crypto Primitives** implement the cryptographic operations required by the protocols, such as hashing, hash-based key derivation function HKDF, AEAD encryption/decryption, signature generation/verification, KEM encapsulation/decapsulation, and Diffie-Hellman key exchange. We utilized the verified cryptographic library HACL* [Zin+17] to implement these cryptographic primitives.

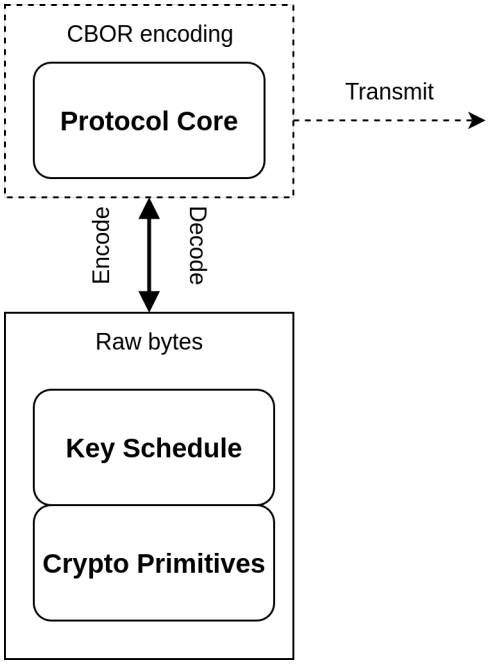



**Figure 12:** Implementation architecture of EDHOC* and KEMEDHOC*. The dashed box indicates the components that are unverified, namely CBOR encoding. On the other hand, the solid boxes indicate the components that are formally proved.

**Overview of building chain.** Each module, i.e. Protocol Core, key schedule, and Cryptographic Primitives, includes both F* specification and Low* implementation. The F* specification, which is faithfully transfered from the protocol specifications, serves for reasoning about the functional correctness. Low* implementation, on the

other hand, models the actual executable code at machine-byte level, guaranteeing memory safety and side-channel resistance. The F* specification and Low* implementation are connected through a series of lemmas that prove the functional correctness of the Low* implementation with respect to the corresponding F* specification. Finally, the Low* implementation is translated to C code utilizing the KaRaMel compiler [Pro+17]. Then, we utilize the GCC compiler to produce executable code from the generated C code.

## 6.2 EDHOC*

In this section, we describe the verified implementation of EDHOC, called EDHOC*. We first present the F* specification of EDHOC in Subsection 6.2.1, and then describe the corresponding Low* implementation of EDHOC in Subsection 6.2.2. In **Protocol Core** module, we only discuss the implementation and specification of the functions by which invoked by the `Responder` to send Message 2.

### 6.2.1 Formalizing EDHOC in F*

We specify a set of F* types that model EDHOC protocol error, result types, and authentication methods as enumerations, as shown in Listing 23. The error type `error` include both EDHOC standardized errors, defined in RFC 9528 [SMP24], and errors specific to our implementation that may arise during the execution of the protocol. The result type `result` is a parametric type that can represent either a successful result of type `a` or a failure result of type `b`. The authentication methods supported by EDHOC are represented as an enumeration type `method_enum`, which includes the four methods defined in EDHOC specification: `SigSig`, `SigStat`, `StatSig`, and `StatStat`. Those enumerated types are used in both function implementation and lemmas to ensure type safety and correctness.

```
type error =
    (*EDHOC standardized errors*)
    | UnspecifiedError | UnsupportedCipherSuite | UnknownCredentialID
    (*Implementation-specific errors*)
    | MissingKey | InputSize // too long or too short input
    | DecryptionFailed | UnsupportedAuthenticationMethod
    | InvalidECPoint | SerializationDeserializationFailed
    | SigningFailed | IntegrityCheckFailed
    | InvalidState | InvalidCredential

type result (a b:Type) =
    | Res : v:a -> result a b
    | Fail: v:b -> result a b

type method_enum =
    | SigSig | SigStat | StatSig | StatStat
```

**Listing 23:** F* specification of EDHOC error and result types.

**Crypto Primitives.** We wrap the cryptographic primitives provided by HACL* into cipher suite- and algorithm-driven F* interfaces. For example, `alg_aead_correctness` and `aead_correctness`, shown in Listing 24, are cipher suite and algorithm-agnostic lemmas that respectively prove the correctness of AEAD encryption and decryption algorithms. The two lemmas ensure that if a message is encrypted using an AEAD

algorithm and then decrypted using the same algorithm, key, IV, and Associated Authenticated Data (AAD), the original message is retrieved. The type `cipherSuite` represents a combination of `aeadAlg`, `hashAlg`, `dhAlg`, and `signatureAlg` which are the four types of AEAD algorithm, hash algorithm, Diffie-Hellman algorithm, and signature algorithm, respectively supported by EDHOC. The refinement type `supported_cipherSuite` represents the supported ciphersuites by EDHOC* implementation. Table 14 shows the supported ciphersuites in EDHOC* implementation.

| Cipher Suite | AEAD | Hash | DH | Signature |
|---|---|---|---|---|
| 6 | AES-GCM-16-64-128 | SHA-256 | X25519 | ES256 |
| 7 | AES-CCM-16-128-128 | SHA-256 | P-256 | ES256 |

**Table 14:** Supported ciphersuites in EDHOC* implementation.

```
type cipherSuite = aeadAlg & hashAlg & dhAlg & signatureAlg

val alg_aead_correctness:
  a:aeadAlg
  -> k:alg_aead_key a -> iv:aead_iv
  -> aad:bytes{length aad <= alg_aead_max_input_size a}
  -> msg:bytes{length msg <= alg_aead_max_input_size a}
  -> Lemma (
    let ciphertext = alg_aead_encrypt a k iv aad msg in
    match alg_aead_decrypt a k iv aad ciphertext with
      | None -> false
      | Some msg' -> Seq.equal msg msg')

let aead_correctness:
  #c:supported_cipherSuite
  -> k:aead_key c -> iv:aead_iv
  -> aad:valid_aead_input_bytes c
  -> msg:valid_aead_input_bytes c
  -> Lemma (
    let ciphertext = aead_encrypt k iv aad msg in
    match aead_decrypt k iv aad ciphertext with
      | None -> false
      | Some msg' -> Seq.equal msg msg')
  = fun #c k iv aad msg -> alg_aead_correctness (get_aead_alg c) k iv aad msg
```

**Listing 24:** Lemmas for AEAD correctness.

**Key Schedule.** We formalize EDHOC key schedule as step-wise F* functions that faithfully follow EDHOC key schedule specification shown in Figure 3. For example, the F* function `extract_prk3e2m`, shown in Listing 25, models the derivation of $\texttt{PRK}_{\texttt{3e2m}}$ as the output of the `Extract` function, taking `SALT3e2m` and the ephemeral-static Diffie-Hellman shared secret $\texttt{g}_{\texttt{rx}}$ as inputs. The refinement type `hash_out cs` represents the byte sequence that has the fixed length equals to the output size of the hash algorithm defined by cipher suite `cs`. This refinement type is used to ensure that the inputs and outputs of hash-based functions have the correct length, preventing potential buffer overflows or underflows.

```
val extract_prk3e2m:
    #cs:supported_cipherSuite -> salt3e2m:hash_out cs // salt
    -> g_rx:shared_secret cs // key -> hash_out cs
let extract_prk3e2m #cs salt3e2m g_rx
    = hkdf_extract cs salt3e2m g_rx
```

**Listing 25:** F* specification of $\texttt{PRK}_{\texttt{3e2m}}$ derivation.

**Protocol Core.** We define struct-like types to model the `party_state` and `handshake_state` that are used to manage machine states during protocol execution. The `party_state`, shown in Listing 26, represents the static state of a party, including its supported ciphersuites, static keys for authentication, its credential, the ephemeral shared secret generated during the handshake, and the public authentication keys and credential of the remote party. The type variable `cs` represents the cipher suite selected for the ongoing handshake, ensuring the types of component fields are consistent with the selected cipher suite.

```
noeq type party_state (#cs:supported_cipherSuite) = {
    suites:  supported_cipherSuite_label; static_dh: ec_keypair cs;
    signature_key: ec_signature_keypair cs; id_cred: id_cred_i_bytes;
    eph_ec_keypair: option (ec_keypair cs); remote_static_pub_key: (ec_pub_key cs);
    remote_signature_pub_key: (ec_signature_pub_key cs);
    remote_id_cred: id_cred_i_bytes;
}
```

**Listing 26:** F* specification of EDHOC party state.

The `handshake_state`, shown in Listing 27, represents the transient state of an ongoing handshake, including the selected cipher suite, the chosen authentication method, the ephemeral public share generated by the local party, the ephemeral public share received from the remote party, various intermediate values computed during the handshake, and the final session key derived at the end of the handshake $\texttt{PRK}_{\texttt{exporter}}$. There are some optional fields, which could be `None` or `Some v`, depending on the progress of the handshake. For example, the field `th2` represents the transcript hash $\texttt{TH}_2$ after Message 2, which is `None` before Message 2 is processed, and then becomes `Some v` after Message 2 is processed. We also define refinement types to represent the handshake state after processing each message. For example, the refinement type `handshake_state_after_msg2`, shown in Listing 28, represents the handshake state after processing Message 2, ensuring that the fields `th2`, `g_y`, `g_xy`, `prk3e2m`, and `prk2e` are all `Some v`, indicating that these values have been computed and are available for use in subsequent steps of the handshake.

```
noeq type handshake_state (#cs:supported_cipherSuite) = {
  suite_i: supported_cipherSuite_label;
  method: method_enum;
  g_x: ec_pub_key cs; msg1_hash: hash_out cs;
  g_y: option (ec_pub_key cs); g_xy: option (shared_secret cs);
  g_rx: option (shared_secret cs); g_iy: option (shared_secret cs);
  th2: option (hash_out cs); th3: option (hash_out cs);
  th4: option (hash_out cs); prk2e: option (hash_out cs);
  prk3e2m: option (hash_out cs); prk4e3m: option (hash_out cs);
  prk_out: option (hash_out cs); prk_exporter: option (hash_out cs);
}
```

**Listing 27:** F* specification of EDHOC handshake state.

```
type handshake_state_after_msg2 (#cs:supported_cipherSuite)
  = hs:handshake_state_after_msg1 #cs {
    Option.isSome hs.th2
    /\ Option.isSome hs.g_y
    /\ Option.isSome hs.g_xy /\ Option.isSome hs.prk3e2m
    /\ Option.isSome hs.prk2e
  }
```

**Listing 28:** F* specification of EDHOC handshake state after Message 2.

The handshake functions which define how each party processes and sends messages are modeled as F* functions that take the current party state and handshake state and necessary data as inputs and return the updated party state, handshake state,

and intermediate values, such as constructed messages and plaintexts, as outputs. For example, we describe the F* code for the `Responder` sending Message 2 in Listing 29. The function `responder_send_msg2` takes the supported cipher suite `cs`, an entropy source `entr`, the current party state `rs`, and the handshake state `hs` as inputs. The function operates as described in 2.1.2.2. In specific, the function first generates an ephemeral key pair $(y, g_y)$, if the key generation fails, it returns an error `InvalidECPoint`. Then, it generates a random connection identifier $c_r$ using the entropy source. Next, it computes the shared secret $g_{xy}$ using the Diffie-Hellman function `dh`, if the computation fails, it returns an error `InvalidECPoint`. After that, it computes intermediate keys, $PRK_{2e}$ and $PRK_{3e2m}$, and the transcript hash $TH_2$ using the corresponding functions defined in the **Key Schedule** module.

At the end, if every computation succeeds, the function should construct the $PTX_2$ and Message 2 according to the chosen authentication method. In addition, it updates the party state and handshake state to reflect the progress of the handshake. The assurance of state update is provided by the refinement types `party_state_shared_est` and `handshake_state_after_msg2`, which ensure that the updated party state has established a shared secret and the handshake state is at the correct stage after processing Message 2.

```
val responder_send_msg2:
  #cs:supported_cipherSuite
  -> entr: HACLRandom.entropy
  -> rs:party_state #cs
  -> hs:handshake_state_after_msg1 #cs
  -> Tot (eresult (plaintext2 #cs #(get_auth_material Responder hs.method)
                & message2 #cs #(get_auth_material Responder hs.method)
                & party_state_shared_est #cs & handshake_state_after_msg2 #cs))

let responder_send_msg2 #cs entr rs hs
    = let local_auth_material = get_auth_material Responder hs.method in
    let eph_ec_keypair = generate_ec_keypair cs entr in
    match eph_ec_keypair with
    | None -> Fail InvalidECPoint
    | Some (y_gy, entr1) -> (
        let y = y_gy.priv in
        let g_y = y_gy.pub in
        let g_x = hs.g_x in
        let (entr2, c_r) = HACLRandom.crypto_random entr1 c_id_size in
        match (dh y g_x) with
        | None -> Fail InvalidECPoint
        | Some g_xy -> (
            let id_cred_r:lbytes (length rs.id_cred) = rs.id_cred in
            let cred_r:lbytes (length rs.id_cred) = rs.id_cred in
            // construct EAD2: currently set it to None
            let ead2_op = None in
            let th2 = compute_th2_pre_hash #cs hs.msg1_hash g_y in
            let prk2e = extract_prk2e th2 g_xy in
            let prk3e2m_res:eresult (hash_out cs)
            = derive_prk3e2m #cs local_auth_material prk2e th2 (Some rs.static_dh.priv) (Some hs.g_x) in
            ...
```

**Listing 29:** F* specification of EDHOC `Responder` sending Message 2.

**Protocol Core lemmas.** To prove the correctness of the Protocol Core functions, we define a series of lemmas that establish the expected behaviour of each function with respect to the F* specifications of the protocol. For example, We define the lemma below, shown in Listing 30, to prove the expected correctness of the function `responder_send_msg2`. The lemma states that if the function `responder_send_msg2` returns an error, it must be one of the expected errors: `InvalidECPoint`, `SerializationDeserializationFailed`, or `SigningFailed`. If the function returns a successful result, it guarantees that the fields updated in the party

state and handshake state are consistent with the computations performed during the function execution, as specified in Section 2.1.2.2.

In addition, the lemma ensures that the handshake state transition from after Message 1 to after Message 2 is valid, as defined by the predicate `valid_hs_msg1_to_msg2`. The predicate `valid_hs_msg1_to_msg2` checks that the fields that were agreed upon in Message 1, such as the authentication method, selected cipher suite, the ephemeral public share $g_x$, and the hash of Message 1, remain unchanged in the handshake state after processing Message 2.

```
let valid_hs_msg1_to_msg2
      (#cs:supported_cipherSuite)
      (hs':handshake_state_after_msg1 #cs)
      (hs'':handshake_state_after_msg2 #cs)
      = hs'.method = hs''.method /\ hs'.suite_i = hs''.suite_i
      /\ lbytes_eq hs'.g_x hs''.g_x /\ lbytes_eq hs'.msg1_hash hs''.msg1_hash

val lemma_responder_send_msg2_consistence:
    #cs:supported_cipherSuite
    -> entr: HACLRandom.entropy
    -> rs:party_state #cs
    -> hs:handshake_state_after_msg1 #cs
    -> Lemma (ensures (
      match (responder_send_msg2 #cs entr rs hs) with
        | Fail InvalidECPoint | Fail SerializationDeserializationFailed | Fail SigningFailed -> True
        | Fail _ -> False
        | Res (ptx2, msg2, rs', hs'') -> (
          let th2 = compute_th2_pre_hash #cs hs.msg1_hash msg2.g_y in
          let g_xy = Some?.v hs''.g_xy in
          let prk2e = extract_prk2e th2 g_xy in
          let prk3e2m = match (get_auth_material Responder hs.method) with
                        | Signature -> prk2e
                        | MAC -> (
                          let r = rs.static_dh.priv in
                          let g_x = hs.g_x in
                          let g_rx = Some?.v (dh r g_x) in
                          let salt3e2m = expand_salt3e2m prk2e th2 in
                          extract_prk3e2m salt3e2m g_rx
                        ) in

          lbytes_eq msg2.g_y (Some?.v rs'.eph_ec_keypair).pub
          /\ lbytes_eq #(length ptx2.id_cred_r) #(length rs.id_cred) ptx2.id_cred_r rs.id_cred
          /\ lbytes_eq (Some?.v hs''.prk2e) prk2e /\ lbytes_eq (Some?.v hs''.prk3e2m) prk3e2m
          /\ valid_hs_msg1_to_msg2 hs hs'' /\ lbytes_eq (Some?.v hs''.th2) th2
        )
    ))
    [SMTPat (responder_send_msg2 #cs entr rs hs)]
```

**Listing 30:** Lemma for the functional correctness of `responder_send_msg2`.

**Protocol interactive run.** In order to model and verify the interacts between functions of both parties during the handshake, we define a series of bundle functions that represent the complete execution of the protocol up to a certain message. For example, the function `bundle_msg1_msg2`, shown in Listing 31, models the interactive run of EDHOC up to Message 2. Firstly, it calls the function `initiator_send_msg1` to simulate the `Initiator` sending Message 1. Then, it calls the function `responder_process_msg1` to simulate the `Responder` processing Message 1 generated by the `Initiator`. Next, the Message 2 is constructed from the `Responder` by calling the function `responder_send_msg2`. Finally, the function `initiator_process_msg2` is triggered to simulate the `Initiator` processing Message 2. At the end of the function, if every step succeeds, it returns the $\texttt{PTX}_2$ (`ptx2_i` and `ptx2_r`) from both parties, the constructed Message 2 (`msg2`), the updated party states (`is'` and `rs'`), and the updated handshake states (`hs_i''` and `hs_r''`).

```
val bundle_msg1_msg2:
  #cs:supported_cipherSuite
  -> entr: HACLRandom.entropy
  -> is:party_state #cs
  -> rs:party_state #cs
  -> method:method_enum
  -> (eresult (plaintext2 #cs #(get_auth_material Responder method)
              & plaintext2 #cs #(get_auth_material Responder method)
              & message2 #cs #(get_auth_material Responder method)
              & party_state_shared_est #cs & party_state_shared_est #cs
              & handshake_state_after_msg2 #cs & handshake_state_after_msg2 #cs))

let bundle_msg1_msg2 #cs entr is rs method
    = match (initiator_send_msg1 cs method entr is) with
        | Fail e -> Fail e
        | Res (msg1, is', hs_i') -> (
            match (responder_process_msg1 #cs rs msg1) with
            | Fail e -> Fail e
            | Res (hs_r') -> (
                lemma_responder_process_msg1_agreement #cs rs msg1;
                match (responder_send_msg2 #cs entr rs hs_r') with
                | Fail e -> Fail e
                | Res (ptx2_r, msg2, rs', hs_r'') -> (
                    match (initiator_process_msg2 #cs is' hs_i' msg2) with
                    | Fail e -> Fail e
                    | Res (ptx2_i, hs_i'') -> (
                        Res (ptx2_i, ptx2_r, msg2, is', rs', hs_i'', hs_r''))))))
```

**Listing 31:** F* specification of EDHOC interactive run up to Message 2.

In order to prove the correctness of the `bundle_msg1_msg2` function, we define the lemma below, shown in Listing 32. The lemma states that if the function `bundle_msg1_msg2` returns an error, it must be one of the expected errors shown in the Listing 32. If the function returns a successful result, it guarantees that the intermediate values computed during the Message 1 and Message 2 exchanges are consistent between both parties, defined by the predicate `post_hs_after_msg2`. Moreover, the lemma ensures that the $\texttt{PTX}_2$ constructed by both parties are identical, and the updated party states maintain the fixed values, such as static keys and credentials, unchanged. Note that the lemma is only valid if both parties have statisfied the defined precondition, requiring the two parties to be honest and have compatible ciphersuites and credentials. The similar pattern of lemmas are also defined to prove the correctness of complete interactive runs up to Message 3 and the full handshake.

```
let post_hs_after_msg2 (#cs:supported_cipherSuite)
    (hs_i'':handshake_state_after_msg2 #cs) (hs_r'':handshake_state_after_msg2 #cs)
    = post_hs_after_msg1 hs_i'' hs_r''
    /\ lbytes_eq (Some?.v hs_i''.th2) (Some?.v hs_r''.th2)
    /\ lbytes_eq (Some?.v hs_i''.g_xy) (Some?.v hs_r''.g_xy)
    /\ lbytes_eq (Some?.v hs_i''.prk2e) (Some?.v hs_r''.prk2e)
    /\ lbytes_eq (Some?.v hs_i''.prk3e2m) (Some?.v hs_r''.prk3e2m)
    /\ lbytes_eq (Some?.v hs_i''.g_y) (Some?.v hs_r''.g_y)

val lemma_bundle_msg1_msg2:
    #cs:supported_cipherSuite
    -> entr:HACLRandom.entropy
    -> is:party_state #cs
    -> rs:party_state #cs
    -> method:method_enum
    -> Lemma (requires precondition_integration_party_states is rs method)
    (ensures (
      match (bundle_msg1_msg2 #cs entr is rs method) with
        | Fail InvalidECPoint | Fail SigningFailed -> True
        | Fail _ -> False
        | Res (ptx2_i, ptx2_r, msg2, is', rs', hs_i'', hs_r'') ->
          post_hs_after_msg2 #cs hs_i'' hs_r''
          /\ hs_r''.method = method /\ hs_i''.method = method
          /\ lbytes_eq (concat_ptx2 ptx2_i) (concat_ptx2 ptx2_r)
          /\ post_ps_unchanged_fixed_vals is is'
          /\ post_ps_unchanged_fixed_vals rs rs'
    ))
    [SMTPat (bundle_msg1_msg2 #cs entr is rs method)]
```

**Listing 32:** Lemma for the functional correctness of EDHOC interactive run up to Message 2. It is valid if and only if the two communicating parties are honest.

### 6.2.2 Implementing EDHOC in Low*

In this section, we describe the Low* implementation of EDHOC which is provably correct with respect to the F* specification described in Subsection 6.2.1. Moreover, the Low* implementation can reason about memory safety by using `ST.Stack` effect for stateful computations and stack-allocated buffers. Our implementation does not employ heap-based memory allocation, thus the memory threat model relies on only stack-based computations. This stack-based-only memory model makes the memory model feasibly tractable since it eliminates the complexity of dynamic memory allocation and deallocation. Regarding side-channel resistance, we utilize the `eq_mask` from HACL* library which is a constant-time equality check function to compare secret integers `lbytes_l SEC`. This masked comparison prevents any branching based on secret data, mitigating timing side-channel attacks.

In each function having a stateful `ST.Stack` effect, the three predicates `live`, `modifies`, and `disjoint` are the heart of the memory safety guarantees. The predicate `live h b` ensures that the buffer `b` is live in the memory state `h`, meaning that the buffer has been allocated and not yet deallocated. The predicate `modifies loc h0 h1` ensures that only the specified location `loc` is modified between the memory states `h0` and `h1`, meaning that all other memory locations, excluding `loc`, remain unchanged. The predicate `disjoint buffer1 buffer2`, or `loc_disjoint loc1 loc2`, ensures that the two buffers or locations do not overlap in memory, preventing unexpected modifications and overflow of condition vectors during satisfiability checking.

**Crypto Primitives.** We encapsulate the Low* implementations of cryptographic primitives provided by HACL* [Zin+17] again into cipher suite- and algorithm-agnostic Low* interfaces. For example, the `aead_encrypt_st`, shown in Listing 33, is a return state Low* function that implements AEAD encryption. The function has the `ST.Stack` effect, which takes stack-allocated buffers that satisfy the precondition `requires fun h0` as inputs and ensures the post-condition `ensures fun h0 res h1`, where `h0` and `h1` are the memory state before and after the function execution respectively, as outputs. The precondition ensures that all input buffers are live in the initial stack and all input buffers are disjoint in memory, preventing potential memory safety issues, such as use-after-free and unexpected overlapping buffers. The post-condition ensures that if the function returns `CSuccess`, only the ciphertext buffer is modified, defined by the predicate `modifies1`. Additionally, the post-condition guarantees that the content of the ciphertext buffer, extracted through `as_seq` function, matches the expected output of the AEAD encryption function defined in the F* specification `Spec.alg_aead_encrypt`, ensuring that the Low* implementation is functionally correct with respect to the F* specification. If the function returns `CUnsupportedAlgorithmOrInvalidConfig`, no memory is modified, claimed by the predicate `modifies0`.

```
type aead_encrypt_st (a:Spec.aeadAlg) (pre:Type0)
    = key:aead_key_lbuffer a
    -> iv_len:size_t{iv_len = size Spec.aead_iv_size}
    -> iv:lbuffer uint8 iv_len
    -> ad_len:size_t{size_v ad_len <= (Spec.alg_aead_max_input_size a)}
    -> ad:lbuffer uint8 ad_len
    -> ptx_len:size_t{size_v ptx_len <= (Spec.alg_aead_max_input_size a)
                    /\ size_v ptx_len + (Spec.aead_tag_size a) <= max_size_t}
    -> ptx:lbuffer uint8 ptx_len
    -> ctxt:lbuffer uint8 (size (size_v ptx_len + (Spec.aead_tag_size a)))
    -> ST.Stack c_response
    (requires fun h0 ->
        live h0 key /\ live h0 iv /\ live h0 ad /\ live h0 ptx /\ live h0 ctxt
        /\ B.all_disjoint [loc key; loc iv; loc ad; loc ptx; loc ctxt])
    (ensures fun h0 res h1 ->
        (match res with
            | CSuccess -> modifies1 ctxt h0 h1 /\
                as_seq h1 ctxt
                    == Spec.alg_aead_encrypt a (as_seq h0 key) (as_seq h0 iv) (as_seq h0 ad) (as_seq h0 ptx)
            | CUnsupportedAlgorithmOrInvalidConfig -> modifies0 h0 h1
            | _ -> False))
val aead_aes128_gcm_encrypt:aead_encrypt_st SpecAEAD.AES128_GCM (ImplAEAD.config_pre SpecAEAD.AES128_GCM)
```

**Listing 33:** Low* implementation of AEAD encryption.

**Key Schedule.** We implement EDHOC key schedule functions in Low* that are memory safe and functionally correct with respect to their F* specification. For instance, as shown in Listing 34, the function `extract_prk3e2m` implements the derivation of $\mathsf{PRK}_{3e2m}$ which takes the cipher suite `cs`, stack-allocated buffers for $\mathsf{SALT}_{3e2m}$ (`salt3e2m_buff`), the ephemeral-static DH shared secret `g_rx_buff`, and the output buffer for $\mathsf{PRK}_{3e2m}$ (`prk3e2m_buff`) as inputs. The function requires that all input buffers are live in the initial stack, and all input buffers are disjoint in memory, defined by predicates `live` and `B.all_disjoint` respectively. The function returns a `unit` type, indicating that it does not return any meaningful value. Moreover, it ensures that if the function executes successfully, only the output buffer `prk3e2m_buff` is modified, defined by the predicate `modifies1`, and the content of the output buffer matches the expected output of the F* specification `Spec.extract_prk3e2m`.

```
val extract_prk3e2m:
  #cs:SpecCrypto.supported_cipherSuite
  -> salt3e2m_buff:hash_out_buff cs
  -> g_rx_buff:shared_secret_buf cs
  -> prk3e2m_buff:hash_out_buff cs
  -> ST.Stack unit
  (requires fun h0 -> (
    live h0 salt3e2m_buff /\ live h0 g_rx_buff /\ live h0 prk3e2m_buff
    /\ B.all_disjoint [loc salt3e2m_buff; loc g_rx_buff; loc prk3e2m_buff]))
  (ensures fun h0 _ h1 -> (
    let prk3e2m_abstract
      = Spec.extract_prk3e2m #cs (as_seq h0 salt3e2m_buff) (as_seq h0 g_rx_buff) in
    modifies1 prk3e2m_buff h0 h1
    /\ Seq.equal prk3e2m_abstract (as_seq h1 prk3e2m_buff)))

let extract_prk3e2m #cs salt3e2m_buff g_rx_buff prk3e2m_buff
  = hkdf_extract cs prk3e2m_buff (size (SpecCrypto.hash_size cs)) salt3e2m_buff
                 (size (SpecCrypto.shared_secret_size cs)) g_rx_buff
```

**Listing 34:** Low* implementation of $\mathsf{PRK}_{3e2m}$ derivation.

**Protocol Core.** In order to formalize the optional fields in the `handshake_state`, we introduce a machine-level type `lbufferOpt`, shown in Listing 35, which encapsulates two stack-allocated buffers: a one-byte buffer `is_some` to indicate whether the optional field is `None` or `Some v`, and a fixed-length data buffer `value` to store the actual data when it is `Some v`. The `is_some` fields contains only the values of 0 or 1, where 0 represents `None` and 1 represents `Some v`.

```
noeq type lbufferOpt (len: size_t) = {
    is_some: lbuffer uint8 1ul;
    value: lbuffer uint8 len
}
```

**Listing 35:** Low* implementation of `lbufferOpt`

The low-level model of the `handshake_state`, called `handshake_state_m`, is shown in Listing 36. The type definition, the optional fields are represented using the `lbufferOpt` type. For example, the field `g_y` is defined as the stateful type `lbufferOpt (ec_pub_key_size cs)`, where `ec_pub_key_size cs` is the size of the ephemeral public key specified by the cipher suite `cs`. We also introduce the predicate `is_valid_handshake_state_m` (Listing 37), which validates that the handshake state at given memory state `h` is live, if all fields of the struct are live in memory, and disjoint, if all fields are pair-wise disjoint.

```
noeq type handshake_state_m (#cs:SpecCrypto.supported_cipherSuite)
    = {
      suite_i: lbuffer uint8 1ul; method: lbuffer uint8 1ul;
      g_x: ec_pub_key_buf cs; msg1_hash: hash_out_buff cs;
      g_y: lbufferOpt (ec_pub_key_size cs); g_xy: lbufferOpt (shared_secret_size cs);
      g_rx: lbufferOpt (shared_secret_size cs); g_iy: lbufferOpt (shared_secret_size cs);
      th2: lbufferOpt (hash_out_size cs); th3: lbufferOpt (hash_out_size cs);
      th4: lbufferOpt (hash_out_size cs); prk2e: lbufferOpt (hash_out_size cs);
      prk3e2m: lbufferOpt (hash_out_size cs); prk4e3m: lbufferOpt (hash_out_size cs);
      prk_out: lbufferOpt (hash_out_size cs); prk_exporter: lbufferOpt (hash_out_size cs);
    }
```

**Listing 36:** Low* implementation of EDHOC handshake state.

```
let is_valid_handshake_state_m (#cs: supported_cipherSuite)
    (h: HS.mem) (hs: handshake_state_m #cs)
    = handshake_state_m_live h hs /\ handshake_state_m_disjoint hs
```

**Listing 37:** Low* predicate `is_valid_handshake_state_m`.

In addition, we define the predicate `is_valid_handshake_state_m_after_msg2`, shown in Listing 38, to represent the transient state of the handshake after processing Message 2. The predicate validates that the low-level model of handshake state, `handshake_state_m`, at a given memory state `h` is compatible to the high-level refinement type `handshake_state_after_msg2`, ensuring that the optional fields `th2`, `g_y`, `g_xy`, `prk3e2m`, and `prk2e` are all `Some v`, indicated by the predicate `lbufferOpt_is_Some`. To make a formal connection between the low-level model and the high-level specification, we define the function `handshake_state_m_eval` to extract the abstract value of the low-level model `handshake_state_m` to the high-level model `handshake_state`. This evaluation function is used to reason about the equivalence between the Low* implementation and the F* specification of the handshake state. A similar approach is also applied to the party state, in which we define the low-level model `party_state_m`, the predicate `is_valid_party_state_m`, and the evaluation function `party_state_m_eval`.

```
val handshake_state_m_eval:
    #cs: supported_cipherSuite -> h: HS.mem
    -> hs: handshake_state_m #cs -> hs_spec: Spec.handshake_state #cs

let is_valid_handshake_state_m_after_msg2 (#cs: supported_cipherSuite)
    (h: HS.mem) (hs: handshake_state_m #cs) =
    lbufferOpt_is_Some hs.th2 /\ lbufferOpt_is_Some hs.g_y /\ lbufferOpt_is_Some hs.g_xy
    /\ lbufferOpt_is_Some hs.prk3e2m /\ lbufferOpt_is_Some hs.prk2e
```

**Listing 38:** Post-conditions of `handshake_state_m` after Message 2.

In terms of **Protocol Core** handshake functions, we implement the machine-level versions of handshake functions that have their high-level specifications presented in Subsection 6.2.1. For instance, the function `responder_send_msg2`, presented in Listing 39, defines steps taken by the `Responder` to send Message 2, executing at the bytecode level. The function takes the cipher suite `cs`, the authentication method `auth_material`, stack-allocated buffers for plaintext 2 `ptx2_buff` and Message 2 `msg2_buff`, the party state memory `rs_m`, and the handshake state memory `hs_m` as inputs. The function requires that all input buffers are live in the initial stack, and all input buffers are disjoint in memory.

Moreover, the function requires that the given authentication method matches the one specified in the handshake state, and if the authentication method is `MAC`, the ephemeral-static DH shared secret `g_rx` must be `Some v`, ensuring the consistency between the role of party , `Responder` or `Initiator`, and the authentication method specified in the handshake state. The function returns a `c_response` which is an enumeration type representing the possible outcomes of the function execution. The function ensures that only the three following return values are possible: `CInvalidECPoint`, `CSigningFailed`, and `CSuccess`.

The post-condition of the function considers two aspects: memory safety and functional correctness. Regarding memory safety, the function has the `ST.Stack` effect, indicating that it performs stateful computations only on the stack. In addition, the post-condition specifies the memory locations that should be modified based on the return value of the function. If the function early terminates with an error, such as `CInvalidECPoint` or `CSigningFailed`, only the memory locations related to the ephemeral keypair and the optional fields in the handshake state that are computed before the error occurs are modified. If the function succeeds, indicated by the return value `CSuccess`, the locations of buffers for $\texttt{PTX}_2$ and Message 2 are also modified, in addition to the ephemeral keypair and the optional fields in the handshake state.

In terms of functional correctness, the postconidition states that if the Low* implementation returns an error, the error must match the expected errors returned by the corresponding F* specification function, `Spec.responder_send_msg2`. If the Low* implementation returns `CSuccess`, the updated handshake state must satisfy the predicate `is_valid_handshake_state_m_after_msg2`, ensuring that the optional fields that should be `Some v` after Message 2 are indeed `Some v`. Furthermore, the updated handshake state and party state must be equivalent to the ones computed by the F* specification function. Additionally, the content of the $\texttt{PTX}_2$ and Message 2 buffers must equal to the ones returned by the high-level specification function.

During the protocol run, each party has to compare the MAC tag ($\texttt{MAC}_2$ and $\texttt{MAC}_3$), or signature ($\sigma_2$ and $\sigma_3$), in the received message with the locally computed one to verify the authenticity of the message. This comparison is handled by the constant-time function `lbytes_eq` performing on the `SEC` byte buffers, ensuring the execution time does not depend on the content of the buffers.

```
val responder_send_msg2: #cs:SpecCrypto.supported_cipherSuite
    -> #auth_material:SpecCrypto.authentication_material
    -> #ptx2_len:size_t{ptx2_len = plaintext2_len_fixed_v cs auth_material}
    -> #msg2_len:size_t{msg2_len = size (size_v ptx2_len + SpecCrypto.ec_pub_key_size (SpecCrypto.get_ec_alg cs))}
    -> ptx2_buff:lbuffer uint8 ptx2_len -> msg2_buff:lbuffer uint8 msg2_len
    -> rs_m:party_state_mem_shared_est #cs -> hs_m:hs_mem #cs
    -> ST.Stack c_response
    (requires fun h0 -> live h0 entropy_p /\
      live h0 msg2_buff /\ live h0 ptx2_buff
      /\ is_valid_party_state_mem h0 rs_m /\ is_valid_hs_mem h0 hs_m
      /\ ecdsa_sig_keypair_mem_live h0 (ps_mem_get_sig_kp rs_m)
      // disjoint clauses
      /\ (hs_mem_disjoint_to_ps_mem hs_m rs_m /\ hs_mem_disjoint_to hs_m msg2_buff
        ... // (other disjoint clauses omitted)
      /\ auth_material == SpecCrypto.get_auth_material Responder (eval_hs_mem h0 hs_m).method
      /\ (auth_material == SpecCrypto.MAC ==> ~(g_is_null hs_m.g_rx)))
    (ensures fun h0 cres h1 -> // modifies clauses
      ( match cres with
        | CInvalidECPoint -> ( modifies0 h0 h1
            \/ modifies (ec_keypair_loc rs_m.eph_keypair |+| lbufferOpt_loc hs_m.g_y
                |+| lbufferOpt_loc hs_m.g_xy |+| lbufferOpt_loc hs_m.th2 |+| lbufferOpt_loc hs_m.prk2e ) h0 h1)
        | CSigningFailed -> ( modifies (ec_keypair_loc rs_m.eph_keypair
                |+| lbufferOpt_loc hs_m.g_y |+| lbufferOpt_loc hs_m.g_xy
                |+| lbufferOpt_loc hs_m.th2 |+| lbufferOpt_loc hs_m.prk2e
                |+| lbufferOpt_loc hs_m.prk3e2m ) h0 h1)
        | CSuccess -> ( modifies (loc ptx2_buff |+| loc msg2_buff
                |+| ec_keypair_loc rs_m.eph_keypair |+| lbufferOpt_loc hs_m.g_y
                |+| lbufferOpt_loc hs_m.g_xy |+| lbufferOpt_loc hs_m.th2
                |+| lbufferOpt_loc hs_m.prk2e |+| lbufferOpt_loc hs_m.prk3e2m ) h0 h1)
        | _ -> False)
      // functional correctness
      /\ ( let hs_init = handshake_state_m_eval h0 hs_m in
        let rs_init = party_state_m_eval h0 rs_m in
        let e0_v = let e0_v = B.deref h0 (entropy_p <: B.buffer (Ghost.erased HACLRandom.entropy)) in
        let res_s = Spec.responder_send_msg2 #cs e0_v rs_init hs_init in
        match cres with
            | CInvalidECPoint | CSigningFailed -> Fail? res_s /\ cres == error_to_c_response res_s
            | CSuccess -> (
                Res? res_s /\ (match res_s with
                    | Res (p2_s, msg2_s, rs_s, hs_s) -> (
                        is_valid_handshake_state_m_after_msg2 h1 hs_m /\ hs_equal hs_s (handshake_state_m_eval h1 hs_m)
                        /\ ps_equal rs_s (party_state_m_eval h1 rs_m) /\ Seq.equal ptx2_s (as_seq h1 ptx2_buff)
                        /\ Seq.equal msg2_s (as_seq h1 msg2_buff)))))))
```

**Listing 39:** Low* implementation of EDHOC `Responder` sending Message 2.

A similar pattern of pre- and post-conditions is also applied to other functions handling the handshake messages. We only discuss the Low* implementation of `responder_send_msg2`, but the readers can refer to the full implementation of EDHOC* at this repository.

## 6.3 KEMEDHOC*

In this section, we describe KEMEDHOC*, a verified implementation of KEMEDHOC. Since KEMEDHOC* is built upon EDHOC*, we only discuss partly the additional and modified parts of KEMEDHOC* implementation with respect to EDHOC*.

To the best of our knowledge, among quantum-safe KEM schemes, only ML-KEM [Alm+24] has a verified implementation [Bha+24b]. Thus, we adopt ML-KEM as the instantiation of KEM scheme in our KEMEDHOC* implementation.

### 6.3.1 Formalizing KEMEDHOC in F*

We re-utilize the error and result types defined in EDHOC* for KEMEDHOC*. On the other hand, KEMEDHOC* supports only one authentication method, namely `KEMKEM`, thus we define a new enumeration type `method_enum`, shown in Listing 40, to represent the supported authentication method in KEMEDHOC. This KEM-based authentication method is encoded into the value of 5 as specified in the Section 4.

```
type method_enum = | KEMKEM
let method_enum_to_nat (m: method_enum): nat
  = match m with | KEMKEM -> 5 // 1-4 for EDHOC methods, 5 for KEMEDHOC
```

**Listing 40:** F* specification of KEMEDHOC method enumeration. In KEMEDHOC, only one method, `KEMKEM`, is supported.

**Crypto Primitives.** Following the cipher suite- and algorithm-agnostic approach in EDHOC*, we define the type `kemCipherSuite` to represent a cipher suite that consists of a KEM algorithm (`kemAlg`), an AEAD algorithm (`aeadAlg`), and a hash algorithm (`hashAlg`). Besides the cryptographic primitives defined in EDHOC*, we introduce wrappers for KEM algorithms, including key generation, encapsulation, and decapsulation, provided by HACL*. For example, the function `kem_encaps`, shown in Listing 41, implements the KEM encapsulation algorithm. The function takes a supported cipher suite `kcs`, an entropy source, and the recipient's KEM public key `pub_key` as inputs. The function returns a tuple consisting of the KEM ciphertext and the shared secret, defined by the type `kemEncapsOutput`. Since KEM encapsulation algorithm is probabilistic, the function requires an entropy source to generate randomness.

```
type kemCipherSuite = kemAlg & EdhocCrypto.aeadAlg & EdhocCrypto.hashAlg
type alg_kemEncapsOutput (a: kemAlg) = (lbytes (alg_kem_ciphertext_size a)) & (lbytes (alg_kem_shared_secret_size a))
type kemEncapsOutput (kcs:supportedKemCipherSuite) =
      alg_kemEncapsOutput (get_kem_alg kcs)
val kem_encaps:
    kcs: supportedKemCipherSuite
    -> entr: entropy
    -> pub_key: kemPublicKey kcs
    -> kemEncapsOutput kcs
```

**Listing 41:** F* specification of KEMEDHOC KEM encapsulation.

**Key Schedule.** Based on KEMEDHOC key schedule illustrated in Figure 10, we formalize the key derivation functions in F*.as step-wise functions. For instance, the function `extract_prk3e2m`, shown in Listing 42, implements the derivation of $\mathtt{PRK}_{\mathtt{3e2m}}$ which takes the cipher suite `kcs`, the salt `salt3e2m`, and the KEM shared secret `k_auth_R` as inputs. The refinement type `kemSharedSecret kcs` represents the byte sequence that has the length defined by the cipher suite `kcs`. In details, this function is powered by the `Extract` function which takes `salt3e2m` as a salt and `k_auth_R` as an IKM.

```
val extract_prk3e2m:
    #kcs: supportedKemCipherSuite -> salt3e2m: hash_out kcs
    -> k_auth_R: kemSharedSecret kcs -> hash_out kcs

let extract_prk3e2m #kcs salt3e2m k_auth_R
  = hkdf_extract kcs salt3e2m k_auth_R
```

**Listing 42:** F* specification of $\mathtt{PRK}_{\mathtt{3e2m}}$ derivation in KEMEDHOC.

**Protocol Core.** We again introduce two struct-like types for the party state and the handshake state management. Compared to EDHOC*, KEMEDHOC party state, shown in Listing 43, replaces the static DH keypair and signature key with a long-term KEM keypair, `static_kem_kp`. Moreover, the ephemeral KEM private key,

eph_kem_priv_key, is only required by the Initiator while processing Message 1 as illustrated in Figure 9. Hence, this field is defined as an option type. In addition, the party state consists the ID credential of the remote party, remote_id_cred, for the purpose of re-validating the identity of Initiator in Message 3.

```
noeq type party_state (#kcs: supportedKemCipherSuite) = {
    suite: supportedKemCipherSuiteLabel;
    static_kem_kp: kemKeyPair kcs;
    id_cred: id_cred_byte;
    // only Initiator needs the below ephemeral private key
    eph_kem_priv_key: option (kemPrivateKey kcs);
    remote_static_kem_pub_key: kemPublicKey kcs;
    remote_id_cred: id_cred_byte;
}
```

**Listing 43:** F* specification of KEMEDHOC party state.

KEMEDHOC handshake state, shown in Listing 44, is largely similar to EDHOC handshake state, except that it replaces DH keys and shared secrets with KEM shared secrets. Specifically, the k_xy, k_auth_R, and k_auth_I fields are introduced to replace g_xy, g_rx, g_iy, g_x, and g_y fields. In addition, there is an extra field, remote_id_cred, to store the extracted ID credential of the remote party from the received message. This field is needed for the Responder to verify the ID credential of the Initiator in Message 1 and re-validate it in Message 3. Regarding the transition of handshake state after each message, we refine some conditions in EDHOC* handshake state transition to fit KEMEDHOC protocol flow. For example, the post-condition of the handshake state after Message 2, shown in Listing 45, requires that the KEM ephemeral shared secret k_xy, the initiator authentication shared secret k_auth_I, the transcript hash th2, and the intermediate keys prk2e and prk3e2m are all Some v. On the other hand, the remaining optional fields are still None. This predicate validates the transition of handshake stater from the initial state after Message 1 to the transient state after Message 2.

```
noeq type handshake_state (#kcs: supportedKemCipherSuite) = {
    suite_i: supportedKemCipherSuiteLabel; msg1_hash: hash_out kcs;
    k_xy: option (kemSharedSecret kcs); k_auth_R: kemSharedSecret kcs;
    k_auth_I: option (kemSharedSecret kcs); th1: hash_out kcs;
    th2: option (hash_out kcs); th3: option (hash_out kcs);
    th4: option (hash_out kcs); prk1e: hash_out kcs;
    prk2e: option (hash_out kcs); prk3e2m: option (hash_out kcs);
    prk4e3m: option (hash_out kcs); prk_out: option (hash_out kcs);
    prk_exporter: option (hash_out kcs); remote_id_cred: option id_cred_byte;}
```

**Listing 44:** F* specification of KEMEDHOC handshake state.

```
let is_valid_handshake_state_after_msg2 (#kcs: supportedKemCipherSuite) (hs: handshake_state #kcs)
    = hs.suite_i = Some?.v (get_kemCipherSuite_label kcs)
      /\ Option.isSome hs.k_auth_I /\ Option.isSome hs.k_xy
      /\ Option.isSome hs.th2 /\ Option.isSome hs.prk2e
      /\ Option.isSome hs.prk3e2m /\ Option.isNone hs.prk4e3m
      /\ Option.isNone hs.th3 /\ Option.isNone hs.th4
      /\ Option.isNone hs.prk_out /\ Option.isNone hs.prk_exporter

type handshake_state_after_msg2 (#kcs: supportedKemCipherSuite)
    = hs:handshake_state #kcs {
    is_valid_handshake_state_after_msg2 #kcs hs}
```

**Listing 45:** Post-conditions of handshake_state after Message 2 in KEMEDHOC.

We define the handshake functions in KEMEDHOC* following the same pattern in EDHOC* but adapting to KEM primitives and KEMEDHOC protocol flow. For example, the function responder_send_msg2, shown in Listing 46, implements the steps taken by the Responder to send Message 2. The function takes the cipher suite

kcs, the party state of the Responder rs, the initial handshake state hs, the received Message 1 msg1, and an entropy source entr as inputs. The function returns either an error or a tuple consisting of Message 2 and the updated handshake state, as a refinement type handshake_state_after_msg2, ensuring that the optional fields that should be Some v after Message 2 are indeed Some v. The entropy source is needed for the KEM encapsulation algorithm to generate KEM shared secrets $K_e$, $K\_auth_I$, and their corresponding ciphertexts. The function follows the steps illustrated in Subsection 4.1 for Message 2 construction and handshake-state update.

```
val responder_send_msg2:
    kcs: supportedKemCipherSuite
    -> rs: party_state #kcs -> hs: handshake_state_init #kcs
    -> msg1: message1 #kcs -> entr: entropy
    -> eresult (message2 #kcs & handshake_state_after_msg2 #kcs)

let responder_send_msg2 kcs rs hs msg1 entr
    = let ct_y, k_xy = kem_encaps kcs entr msg1.pk_e in
    let pk_I = rs.remote_static_kem_pub_key in
    let ct_auth_I, k_auth_I = kem_encaps kcs entr pk_I in
    let th2 = compute_th2 ct_y k_auth_I msg1 in
    let prk2e = extract_prk2e hs.prk1e th2 k_xy in
    let salt3e2m = expand_salt info_label_salt3e2m prk2e th2 in
    let prk3e2m = extract_prk3e2m salt3e2m hs.k_auth_R in
    let c_R = unsound_crypto_random2 c_id_size in
    ...
```

**Listing 46:** F* specification of Responder sending Message 2 in KEMEDHOC.

**Protocol Core lemmas.** Besides refinement types, we also introduce lemmas that reason about the validity of handshake state transitions and the expected return values of handshake functions. For example, the lemma shown in Listing 47 reasons about the functional correctness of the function responder_send_msg2. The lemma states that the function always return the Res type, and the updated handshake state satisfies the predicate valid_hs_msg1_to_msg2, ensuring only the optional fields expected to be changed after Message 2 are indeed changed, while the fields established after Message 1 stay the same.

```
let hs_equal_after_msg1 (#kcs: supportedKemCipherSuite)
    (hs1 hs2: handshake_state #kcs)
    = hs1.suite_i = hs2.suite_i
      /\ Seq.equal hs1.msg1_hash hs2.msg1_hash /\ Seq.equal hs1.k_auth_R hs2.k_auth_R
      /\ Seq.equal hs1.th1 hs2.th1 /\ Seq.equal hs1.prk1e hs2.prk1e
let valid_hs_msg1_to_msg2 (#kcs: supportedKemCipherSuite)
    (hs1: handshake_state_init #kcs) (hs2: handshake_state_after_msg2 #kcs)
    = hs_equal_after_msg1 hs1 hs2

val lemma_responder_send_msg2_always_res:
    kcs: supportedKemCipherSuite
    -> rs: party_state #kcs -> hs: handshake_state_init #kcs
    -> msg1: message1 #kcs -> entr: entropy
    -> Lemma (ensures ( match (responder_send_msg2 kcs rs hs msg1 entr) with
        | Fail _ -> False
        | Res (msg2, hs') -> (
          valid_hs_msg1_to_msg2 hs hs')))
```

**Listing 47:** Lemma for the functional correctness of responder_send_msg2.

**Protocol interative run.** Following the similar aproach in EDHOC*, we define the function bundle_msg1_msg2, shown in Listing 48, to simulate the interactive run of KEMEDHOC up to Message 2 between two parties. The function firstly calls the function bundle_msg1 to simulate the Initiator sending Message 1 and the Responder processing Message 1. If the first step succeeds, the function continues

to call `responder_send_msg2` to simulate the `Responder` sending Message 2, and then calls `initiator_process_msg2` to simulate the `Initiator` processing Message 2. The function returns either an error or a tuple consisting of Message 2 (`msg2`), the updated handshake states (`hs_i''` and `hs_r''`), and the updated party states after Message 2 (`is'` and `rs`), and the plaintext (`ptx2`).

```
val bundle_msg1_msg2:
  kcs: supportedKemCipherSuite
  -> is: party_state #kcs
  -> rs: party_state #kcs
  -> entr: entropy
  -> eresult (message2 #kcs & handshake_state_after_msg2 #kcs
    & handshake_state_after_msg2 #kcs & party_state_eph_est #kcs
    & party_state #kcs & plaintext2 #kcs)

let bundle_msg1_msg2 (kcs: supportedKemCipherSuite)
    (is: party_state #kcs) (rs: party_state #kcs) (entr: entropy)
    : eresult (message2 #kcs & handshake_state_after_msg2 #kcs & handshake_state_after_msg2 #kcs
              & party_state_eph_est #kcs & party_state #kcs & plaintext2 #kcs)
    = match (bundle_msg1 kcs is rs entr) with
      | Fail e -> Fail e
      | Res (msg1, hs_i', hs_r', is', rs, ptx1) -> (
        match (responder_send_msg2 kcs rs hs_r' msg1 entr) with
          | Res (msg2, hs_r'') -> (
            match (initiator_process_msg2 kcs is' hs_i' msg2) with
              | Fail e -> Fail e
              | Res (hs_i'', ptx2) -> (
                Res (msg2, hs_i'', hs_r'', is', rs, ptx2))))
```

**Listing 48:** F* specification of KEMEDHOC interactive run up to Message 2.

In order to prove the functional correctness of the interactive run up to Message 2, we define the lemma `lemma_bundle_msg1_msg2_correctness`, shown in Listing 49. The lemma states that if both parties are correctly set up, both parties are honest and store the ID credential and static KEM public key of the other party, the function `bundle_msg1_msg2` always returns the `Res` type. Moreover, the updated handshake states of both parties after Message 2 are equal, the `Responder` correctly extracts the ID credential of the `Initiator` from Message 1, and the party state of `Initiator` (`is`) only changes the ephemeral KEM private key, while the party state of `Responder` (`rs`) remains the same. Note that only the `Initiator` has to generate a ephemeral KEM keypair for each protocol run; therefore, only the party state of `Initiator` is expected to change after Message 2.

```
val lemma_bundle_msg1_msg2_correctness:
  kcs: supportedKemCipherSuite
  -> is: party_state #kcs
  -> rs: party_state #kcs
  -> entr: entropy
  -> Lemma (requires (precond_parties_setup kcs is rs entr))
  (ensures ( match (bundle_msg1_msg2 kcs is rs entr) with
      | Fail _ -> False
      | Res (msg2, hs_i'', hs_r'', is', rs', decrypted_ptx2) -> (
          hs_equal hs_i'' hs_r'' /\ Seq.equal rs.id_cred decrypted_ptx2.id_cred_R
          /\ Seq.equal rs.id_cred decrypted_ptx2.cred_R
          /\ ps_equal_non_eph is is' /\ ps_equal_all rs rs')))
```

**Listing 49:** Lemma for the functional correctness of KEMEDHOC interactive run up to Message 2. It is valid only if both parties are honest and correctly pre-provisioned.

### 6.3.2 Implementing KEMEDHOC in Low*

In this section, we describe the Low* implementation of KEMEDHOC*. The low-level implementation is provably correct with respect to the F* specification presented in Subsection 6.3.1. In addition, following the same pattern in EDHOC*, we formally verify that the machine-level functions in KEMEDHOC* are memory safe by utilizing

the stateful effect `ST.Stack` and a combination of pre- and post-conditions specified by `live`, `disjoint`, and `modifies` predicates. We utilize the constant-time comparison function `lbytes_eq` to handle secret-depedent comparision on $\texttt{MAC}_2/\texttt{MAC}_3$, ensuring the execution time does not depend on the content of computed MAC tags.

**Crypto Primitives.** Besides re-using the cryptographic primitives defined in EDHOC*, we introduce interfaces for KEM algorithms provided by HACL*. For example, the function signature `kem_encaps_st`, presented in Listing 50, defines the pre- and post-conditions of the Low* implementation of KEM encapsulation. The implementation requires that all input buffers, including the entropy source (`entropy_p`), the recipient's KEM public key (`pk`), the output ciphertext buffer (`ct`), and the output shared secret buffer (`ss`), are live in the initial stack and inclusively disjoint in memory. The implementation ensures that only the memory locations of `entropy_p`, `kem_state`, `ct`, and `ss` are modified. Moreover, the post-values of `ct` and `ss` have to equal to the ciphertext and shared secret computed by the F* specification function `Spec.alg_kem_encaps`. Note that the `entroy_p` and `kem_state` buffers are allowed to be modified as they are used to store the updated entropy state and matrix of KEM state during the encapsulation.

```
type kem_encaps_st (a: supportedKemAlg)
    = pk: alg_kem_pub_key_buff a
    -> ct: alg_kem_ciphertext_buff a
    -> ss: alg_kem_shared_secret_buff a
    -> ST.Stack unit
    (requires fun h0 ->
      live h0 entropy_p /\ live h0 pk /\ live h0 ct /\ live h0 ss
      /\ B.all_disjoint [loc pk; loc entropy_p; loc kem_state; loc ct; loc ss])
    (ensures fun h0 _ h1 ->
      modifies (loc entropy_p |+| loc kem_state |+| loc ct |+| loc ss) h0 h1
      /\ ( let e0_v = B.deref h0 (entropy_p <: B.buffer (Ghost.erased HACLRandom.entropy)) in
        let pk_s = as_seq h0 pk in
        let (ct_s, ss_s) = Spec.alg_kem_encaps a e0_v pk_s in
        Seq.equal (as_seq h1 ct) ct_s
        /\ Seq.equal (as_seq h1 ss) ss_s))
```

**Listing 50:** Low* implementation of KEMEDHOC KEM encapsulation.

**Key Schedule.** We implement the key derivation functions, illustrated in Figure 10, at machine-byte level in a step-wise manner. For example, the function `extract_prk3e2m`, shown in Listing 51, implements the derivation of $\texttt{PRK}_{\texttt{3e2m}}$. The function takes byte buffers for $\texttt{SALT}_{\texttt{3e2m}}$ (`salt3e2m`) and the KEM authentication shared secret $\texttt{K\_auth}_{\texttt{R}}$ (`k_auth_R`) as inputs, and writes the derived $\texttt{PRK}_{\texttt{3e2m}}$ into the output buffer `prk3e2m`. The pre-condition requires that all input and output buffers are live in the initial stack and disjoint to each other. The post-condition ensures that only the memory location of `prk3e2m` is modified, and the content of `prk3e2m` equals to the value computed by the F* specification function `Spec.extract_prk3e2m`.

```
val extract_prk3e2m:
    #kcs: supportedKemCipherSuite -> salt3e2m: hash_out_buff kcs
    -> k_auth_R: kem_shared_secret_buff kcs -> prk3e2m: hash_out_buff kcs
    -> ST.Stack unit
    (requires fun h0 ->
        live h0 salt3e2m /\ live h0 k_auth_R /\ live h0 prk3e2m
        /\ B.all_disjoint [loc salt3e2m; loc k_auth_R; loc prk3e2m])
    (ensures fun h0 _ h1 ->
        let prk3e2m_s = Spec.extract_prk3e2m #kcs (as_seq h0 salt3e2m) (as_seq h0 k_auth_R) in
        modifies1 prk3e2m h0 h1 /\ Seq.equal (as_seq h1 prk3e2m) prk3e2m_s)
```

**Listing 51:** Low* implementation of $\texttt{PRK}_{\texttt{3e2m}}$ derivation in KEMEDHOC.

**Protocol Core.** Based on the state management types defined in EDHOC*, we redefine stateful types for static party state and session handshake state adapting to KEMEDHOC specification. The major changes are replacing DH keys and shared secrets with KEM keys and shared secrets, as discussed in Subsection 6.3.1. For instance, the Low* implementation of KEMEDHOC handshake state, shown in Listing 52, introduces the fields `k_xy`, `k_auth_R`, and `k_auth_I`. The `lbufferOpt` type is also re-utilized to represent optional fields in the handshake state. In addition, the field `remote_id_cred` is added to store the extracted ID credential of the remote party, serving for the purpose of credential re-verification in Message 3.

```
noeq type handshake_state_m (kcs: supportedKemCipherSuite) = {
    suite_i: lbuffer uint8 1ul;
    msg1_hash: hash_out_buff kcs;
    k_xy: lbufferOpt (kem_shared_secret_size_t kcs);
    k_auth_R: kem_shared_secret_buff kcs;
    k_auth_I: lbufferOpt (kem_shared_secret_size_t kcs);
    th1: hash_out_buff kcs;
    th2: lbufferOpt (hash_size_t kcs);
    th3: lbufferOpt (hash_size_t kcs);
    th4: lbufferOpt (hash_size_t kcs);
    prk1e: hash_out_buff kcs;
    prk2e: lbufferOpt (hash_size_t kcs);
    prk3e2m: lbufferOpt (hash_size_t kcs);
    prk4e3m: lbufferOpt (hash_size_t kcs);
    prk_out: lbufferOpt (hash_size_t kcs);
    prk_exporter: lbufferOpt (hash_size_t kcs);
    // ID credential of the remote party
    remote_id_cred: lbufferOpt (size SpecParser.cred_size);
    }
```

**Listing 52:** Low* specification of KEMEDHOC handshake state.

In addition, we define predicates and lemmas to reason about the validity the transition of handshake state across messages. For example, the predicate `is_valid_handshake_state_m_after_msg2`, shown in Listing 53, validates optional fields that should be `Some v` after Message 2 are indeed `Some v`, while the remaining optional fields are still `None`. This predicate ensures that there is no unexpected modification in the handshake state stored at each party. The lemma illustrated in Listing 54 proves the equivalence between the Low* predicate and its corresponding F* predicate, ensuring that the validity of a Low* implementation is consistent with the F* specification.

```
let is_valid_handshake_state_m_after_msg2 (#kcs: supportedKemCipherSuite)
    (h: HS.mem) (hs: handshake_state_m kcs)
    = // should be Some after Msg2
    /\ lbufferOpt_is_Some h hs.k_auth_I /\ lbufferOpt_is_Some h hs.k_xy
    /\ lbufferOpt_is_Some h hs.th2 /\ lbufferOpt_is_Some h hs.prk2e
    /\ lbufferOpt_is_Some h hs.prk3e2m
    // should be None
    /\ lbufferOpt_is_None h hs.prk4e3m /\ lbufferOpt_is_None h hs.prk_out
    /\ lbufferOpt_is_None h hs.prk_exporter
```

**Listing 53:** Post-conditions of `handshake_state_m` after Message 2 in KEMEDHOC.

```
let lemma_is_valid_handshake_state_m_after_msg2_equiv_spec
    (#kcs: supportedKemCipherSuite) (h: HS.mem) (hs: handshake_state_m kcs)
    : Lemma (requires is_valid_handshake_state_m h hs)
    (ensures ( let hs_eval = handshake_state_m_eval #kcs h hs in
      is_valid_handshake_state_m_after_msg2 h hs
      <==> Spec.is_valid_handshake_state_after_msg2 hs_eval))
```

**Listing 54:** Lemma proving the equivalence between Low* predicate `is_valid_handshake_state_m_after_msg2` and F* specification predicate `is_valid_handshake_state_after_msg2` in KEMEDHOC.

Concerning the Low* implementation of handshake functions, we discuss here the implementation of the fucntion `responder_send_msg2`, shown in Listing 55, which implements the steps taken by the `Responder` to send Message 2. The function takes party state (`rs`), handshake state (`hs`), the received Message 1 buffer (`msg1`), and the Message 2 to be sent (`msg2`) buffer as inputs. The pre-condition requires that all input are live and disjoint to each other in memory. As state management types, such as `handshake_state_m`, `party_state_m`, are struct-like types composed of multiple buffers, we also define predicates that reason about the disjointness between these types and within themselves, such as `handshake_state_m_disjoint_to_msg1`. In addition, the function requires that the handshake state is in the initial state, which is expected after processing Message 1, and the party state of `Responder` is in the ephemeral-key-established state, which is expected after establising an ephemeral KEM shared secret $\texttt{K}_\texttt{e}$.

The post-condition ensures that that only the expected memory locations are indeed modified. Depending on the return value, the modified locations vary. For example, if the function returns `CUnsupportedAlgorithmOrInvalidConfig`, which means the `Responder` does not support AEAD encryption for $\texttt{PTX}_2$, then only the memory locations of buffers defined by `base_modified_locs` are modified. On the other hand, if the function returns `CSuccess`, which means `Responder` successfully encrypts $\texttt{PTX}_2$ and constructs Message 2, then the memory location of `msg2.c2` is also modified besides those defined by `base_modified_locs`. Moreover, we also reason that the party state and handshake state structs are still live and within-disjoint after the function call. These memory-safety properties are crucial for application developers to foresee the memory behaviour of the function and avoid memory-safety bugs when integrating the function into their applications.

Respecting the functional correctness, the post-condition states that if the function does not return `CUnsupportedAlgorithmOrInvalidConfig` error , which is a implementation-specific error not covered by the high-level specification, then the response should be either a success (`CSuccess`) or an error corresponding to the high-level specification. In the case of success, the updated handshake state of `Responder` and the constructed Message 2 should be equal to the values computed by the F* specification function `Spec.responder_send_msg2`.

The constant-time comparision function `lbytes_eq` is utilized to handle MAC tags ($\texttt{MAC}_2$/$\texttt{MAC}_3$) comparision between the locally computed MAC tag and the one decrypted from the received message. This prevents the potential timing side-channel leakage that allows an attacker infer the correctness of a forged MAC tag by measuring the execution time of the function.

```
val responder_send_msg2:
    kcs: supportedKemCipherSuite
    -> rs: party_state_m kcs -> hs: handshake_state_m kcs
    -> msg1: message1 kcs -> msg2: message2 kcs
    -> ST.Stack c_response
    (requires fun h0 ->
      is_valid_handshake_state_m h0 hs /\ is_party_state_eph_est_m h0 rs
      ... // (other validity conditions omitted)
      // Disjointness
      /\ handshake_state_m_disjoint_to_party_state hs rs
      /\ handshake_state_m_disjoint_to_msg1 hs msg1 /\ handshake_state_m_disjoint_to_msg2 hs msg2
        ... // (other disjointness conditions omitted)
    )
    (ensures fun h0 res h1 ->
      let base_modified_locs = loc kem_state |+| loc entropy_p |+| lbufferOpt_loc hs.k_xy
                |+| loc msg2.ct_y |+| lbufferOpt_loc hs.k_auth_I |+| loc msg2.ct_auth_I
                |+| lbufferOpt_loc hs.th2 |+| lbufferOpt_loc hs.prk2e |+| lbufferOpt_loc hs.prk3e2m in
      // memory modification post-condition
      (match res with
        | TypeEdhoc.CUnsupportedAlgorithmOrInvalidConfig -> modifies base_modified_locs h0 h1
        | TypeEdhoc.CSuccess -> modifies (base_modified_locs |+| loc msg2.c2) h0 h1
        | _ -> False)
      // validity of management states and messages
      /\ (is_valid_handshake_state_m h1 hs
        /\ Spec.is_valid_handshake_state_after_msg2 (handshake_state_m_eval h1 hs)
        /\ is_valid_party_state_m h1 rs /\ is_valid_message2 h1 msg2)
      // functional correctness respect to the high-level specification
      /\ ( res <> TypeEdhoc.CUnsupportedAlgorithmOrInvalidConfig
        ==> ( let rs_init = party_state_m_eval h0 rs in
        let hs_init = handshake_state_m_eval h0 hs in let msg1_init = message1_eval h0 msg1 in
        let entr = B.deref h0 (entropy_p <: B.buffer (Ghost.erased HACLRandom.entropy)) in
        let res_s = Spec.responder_send_msg2 kcs rs_init hs_init msg1_init entr in

        let hs_s_final = handshake_state_m_eval h1 hs in let msg2_s_final = message2_eval h1 msg2 in
        match res_s with
          | Fail e -> res == error_to_c_response e
          | Res (m2_s, hs_s) -> (
            res == TypeEdhoc.CSuccess /\ Spec.hs_equal hs_s_final hs_s
            /\ SpecParser.message2_equal msg2_s_final m2_s))))
```

**Listing 55:** Low* implementation of `Responder` sending Message 2 in KEMEDHOC.

## 6.4 Proof engineering

While the F* high-level specification of EDHOC* and KEMEDHOC* are relatively straighforward and trivial for Z3 SMT solver to discharge, the Low* implementation of them, which consists of complicated and nested pre- and post-conditions, poses significant challenges for automatic verification. In addition, the Low* implementation of handshake functions in **Protocol Core** module have several execution paths depending on the input values and intermediate computation results, resulting in a large number of condition vectors to be expanded during the discharge of verification conditions. To tackle these challenges, we adopt two proof engineering techniques: (1) Abstraction via interfaces, and (2) Proof decomposition via modular lemmas.

**Abstraction via interfaces.** F* provides an abstraction mechanism via function interfaces, allowing the caller, or the client, to reason about the post-condition of a callee, or the server, without expanding the implementation detail of the callee. This reduces the size of queries sent to the SMT solver since irrevalent implementation details are abstracted away. For example, the function signature of `extract_prk3e2m`, shown in Listing 51, serves as an interface for the implementation of `extract_prk3e2m`. When the function is called by a client, its implementation is not expanded as long as the implementaion equips the same function signature, including the same pre- and post-conditions. Hence, we add the implementation of `extract_prk3e2m` in a separate file that is only expanded when verifying the module defining the function

itself. On the other hand, the client module only imports the interface file, which contains the function signature, but not the implementation file. When F* verifies the client module, the implementation of `extract_prk3e2m` is not in scope of SMT solver, only the interface file is encoded into SMT queries.

**Proof decomposition via modular lemmas.** When verifying a complicated function, such as `responder_send_msg2` as shown in Listing 55, F* and Z3 SMT solver need to reason about a large amount of condition vectors, which include cryptgraphic primitives, serialization, and state management. When all of these aspects are encoded into a single query, the SMT solver often fails to discharge the query within a reasonable time frame, or sometimes even fails due to memory overflow. To solve this issue, we decompose a complex function with large proof obligations into auxiliary functions that each handles a specific aspect, or a small amount of steps with manageable proof obligations. We then prove modular lemmas that reason about the functional correctness of these auxiliary functions. Finally, we bundle these auxiliary functions and lemmas in the main function that we want to verify. For example, the function `responder_send_msg2`, is decomposed into two auxiliary functions, `responder_send_msg2_set_up` and `responder_construct_msg2` , as illustrated in Listing 56. The first function, `responder_send_msg2_set_up`, derive all necessary keys and handshake state updates, while the second function, `responder_construct_msg2`, constructs Message 2 based on the derived keys and updated handshake state. By separating into smaller proof obligations, we reduce drastically the verification time and memory consumption of the SMT solver for verifying `responder_send_msg2`.

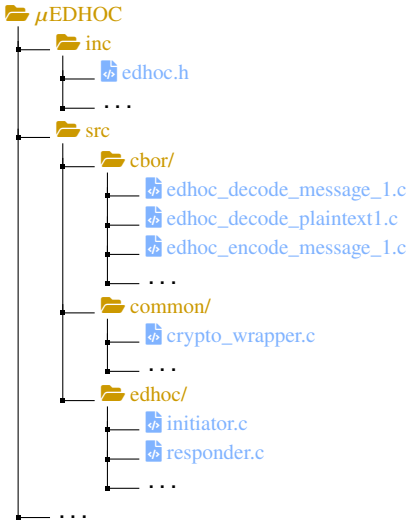
```
val responder_send_msg2_set_up:
    kcs: supportedKemCipherSuite
    -> rs: party_state_m kcs
    -> hs: handshake_state_m kcs
    -> msg1: message1 kcs
    -> msg2: message2 kcs
    -> ST.Stack unit
    (requires fun h0 ->
      is_valid_handshake_state_m h0 hs
      /\ is_valid_party_state_m h0 rs
      /\ is_valid_message1 h0 msg1 /\ is_legit_message1 h0 msg1
      /\ is_valid_message2 h0 msg2 /\ live h0 kem_state /\ live h0 entropy_p
    ... // other pre-conditions
    )
    (ensures fun h0 _ h1 ->
      let modified_locs
        = loc kem_state |+| loc entropy_p |+| lbufferOpt_loc hs.prk3e2m in
        |+| lbufferOpt_loc hs.k_xy |+| loc msg2.ct_y
        |+| loc msg2.ct_auth_I |+| lbufferOpt_loc hs.k_auth_I
        |+| lbufferOpt_loc hs.th2 |+| lbufferOpt_loc hs.prk2e

      modifies modified_locs h0 h1
      /\ lbufferOpt_is_Some h1 hs.k_xy /\ lbufferOpt_is_Some h1 hs.k_auth_I
      /\ lbufferOpt_is_Some h1 hs.th2 /\ lbufferOpt_is_Some h1 hs.prk2e
      ... // other post-conditions)

let responder_send_msg2 kcs rs hs msg1 msg2
    = (**) let h0 = ST.get () in
    responder_send_msg2_set_up kcs rs hs msg1 msg2;
    let res = responder_construct_msg2 kcs rs hs msg2 in
    ... // the rest of implementation
    res
```

**Listing 56:** Proof decomposition of `responder_send_msg2` in KEMEDHOC. The function is decomposed into two auxiliary functions: `responder_send_msg2_set_up` and `responder_construct_msg2`.

## 6.5 Integration into $\mu$EDHOC

Both EDHOC* and KEMEDHOC* stand-alone are libraries that provides application protocol interfaces (APIs) for other applications to use. They are not ready-to-use applications due to the lack of CBOR encoding and transport-layer protocol working under the hood. Therefore, we integrate EDHOC* and KEMEDHOC* into $\mu$EDHOC [Hri+21], a lightweight implementation of EDHOC suporting CBOR encoding and CoAP transport, to be able to run end-to-end experiments.

$\mu$EDHOC is a well-structured project; therefore, we can systematically replace and refactor the existing modules of EDHOC implementation, making them compatible to our verified EDHOC* and KEMEDHOC* libraries. At the high-level, we introduce major changes in three aspects: crypto wrappers, CBOR routines, and context management. Below is the directory tree of $\mu$EDHOC, partly showing the files and folders that we modify or add to integrate EDHOC* and KEMEDHOC*.




**Crypto wrappers.** In the original $\mu$EDHOC implementation, the cryptographic operations are powered by unverified third-party libraries, such as `tinycrypt` [Cor], `mbedtls` [Fir]. We replace these libraries with verified C implementation of HACL*.

The original $\mu$EDHOC implementation does not provide crypto-agile interfaces. The crypto agility is defined as the ability to switch between different cryptographic algorithms without changing the high-level protocol logic or manually modifying the codebase. To achieve crypto agility, we introduce an abstraction layer between the high-level protocol logic and the low-level cryptographic operations. This layer is cipher suite-driven, which means that only the cipher suite is needed to be specified when initializing an instance of this layer. This abstraction helps application developers to easily switch between different ciphersuites they define in the high-level interfaces without the need to modify the low-level implementation. The abstraction layer exactly follows the cipher suite-centered design principle of EDHOC* and KEMEDHOC*. crypto_wrapper.c contains the implementation of abstraction layer and the wrappers of cryptographic operations.

**CBOR routine modifications.** Since KEMEDHOC introduces extra fields in Message 1 and Message 2, the CBOR encoding/decoding routines need to be modified accordingly. In particular, we implement a new function `encode_message_1_kem_kem()` in edhoc_encode_message_1.c, that adds two encoding routines `zcbor_bstr_encode()` for $\texttt{ct\_auth}_\texttt{R}$ and ciphertext 1 ($\texttt{C}_1$). From the decoding side, we also create a new function `decode_message_1_kem_kem()` in edhoc_decode_message_1.c, that adds two decoding routines `zcbor_bstr_decode()` for $\texttt{ct\_auth}_\texttt{R}$ and ciphertext 1 ($\texttt{C}_1$). A similar approach is taken to modify the encoding and decoding routines of Message 2 for additional fields.

The plaintext 1, $\texttt{PTX}_1$, itself has a composite structure, which consists of $\texttt{ID\_CRED}_\texttt{I}$ and $\texttt{EAD}_1$; therefore, we implement a new routine for parsing $\texttt{PTX}_1$ into its components. The function `decode_ptxt1()` first decodes the CBOR map structure of $\texttt{PTX}_1$, then extracts the byte strings of $\texttt{ID\_CRED}_\texttt{I}$ and $\texttt{EAD}_1$ separately through union-based decoding routines of zcbor.

Compared to EDHOC, the $\texttt{TH}_2$ in KEMEDHOC replaces $\texttt{G}_\texttt{Y}$ with $\texttt{ct}_\texttt{Y}$ and $\texttt{K\_auth}_\texttt{I}$. We modify the encoding/decoding routines of $\texttt{TH}_2$ accordingly. The function `encode_th2()` encodes $\texttt{ct}_\texttt{Y}$, $\texttt{K\_auth}_\texttt{I}$, and $\texttt{H}(\texttt{Msg}_1)$ into separate byte strings, then concatenates them to form $\texttt{TH}_2$. The order of concatenation has to follow the specification to ensure interoperability and the functionality of the decoding routine.

**Context management.** The runtime context and party context management structs, `runtime_context`, `edhoc_initiator_context`, and `edhoc_responder_context` in edhoc.h, are responsible for maintaining the state of a protocol instance and a party in $\mu$EDHOC. We modify these structs to accommodate the changes brought by EDHOC* and KEMEDHOC*. In principle, C code from EDHOC* and KEMEDHOC* already provides state management types, such as `party_state_m` and `handshake_state_m` , which can be directly used in $\mu$EDHOC. Hence, we replace the existing state management fields in the context structs with a single pointer, e.g. `*hs_m_ptr`, pointing to the corresponding state management struct defined in EDHOC* and KEMEDHOC*. We preserve fields in the context structs that are specific to network transport, such as socker handler pointer `*sock`.

**Application interfaces.** In order to maintain backward compatibility, we preserve the existing protocol run interfaces, e.g. `edhoc_initiator_run_extended()` function interface in intiator.c. However, the implementation of these interfaces is refactored to handle conditionally EDHOC and KEMEDHOC protocol runs based on the authentication method and cipher suite specified in the context structs. As the high-level interfaces are preserved, application developers can seamlessly switch between EDHOC and KEMEDHOC without the need to modify their application codebase.

Note that both original $\mu$EDHOC and our modified versions both do not support cipher suite negotiation. The cipher suite is statically defined in the context structs before the protocol run. Moreover, the cipher suite agreement between the two parties is out of scope of our implementation and should be guaranteed by the applications relying on the protocol.

# 7 Evaluation

In this section, we desribe the experimental setup and present the benchmark results of EDHOC* and KEMEDHOC*. The evaluation include spatial and temporal performance of the protocols, as well as the additional overhead introduced by the formal verification.

## 7.1 Experimental Setup

Our experiments were perfomed on a commercial laptop, equipped with 12 core 12th Gen Intel® i7–1260P @ 3.40 GHz and 16GB of RAM. The operating system is Ubuntu 22.04.3 LTS. F* version 2025.08.07 dev and Z3 version 4.13.3. were used for verification. KaRaMel version (commit hash) b05a192 was used for extraction to C. Compiler GCC 11.4.0 was used for compiling the C code.

Following the examples provided by $\mu$EDHOC, we set up a CoAP server and a CoAP client to perform EDHOC and KEMEDHOC handshake between two internal network sockets on the same commercial laptop described above. The CoAP server acts as the `Responder`, while the CoAP client acts as the `Initiator`. Both parties, CoAP server and client, are statically pre-provisioned with long-term public keys and certificate chains of each other.

## 7.2 Benchmark Results

**Baselines.** In our evaluation, we compare the performance of EDHOC* and KEMEDHOC* against two baselines: (1) EDHOC SIG-SIG (Ed25519), the most space-consuming authentication method in EDHOC [FVW24], and (2) PQ-EDHOC (ML-KEM-512/FALCON1) [Fra+25c], a post-quantum variant of EDHOC that utilizes ML-KEM-512 [ST24] for key exchange and FALCON1 [Fou+20], a post-quantum digital signature scheme, for authentication. The ML-KEM-512/FALCON1 instantiation of PQ-EDHOC is selected as it offers a comparable security level to ML-KEM-512 used in KEMEDHOC and shows the best overall performance, especially in terms of message size, among all the instantiations of PQ-EDHOC [Fra+25c].

Regarding F*-related metrics, we compare EDHOC* and KEMEDHOC* with Noise* [Ho+22], a verified implementation of the Noise protocol framework that EDHOC is built upon. Noise* serves as a relevant baseline since it is one of the most mature F* project in the domain of verified cryptographic protocols and has a similar scope to EDHOC* and KEMEDHOC*.

**Handshake time.** Figure 13 presents the total handshake time in milliseconds (ms) for EDHOC SIG-SIG (Ed25519), PQ-EDHOC (ML-KEM-512/FALCON1), and KEMEDHOC (ML-KEM-512). The total handshake time is the sum of the time taken by each party, `Initiator` and `Responder`, to complete the three-message handshake. For a `Initiator`, the handshake starts when it generates ephemeral KEM key pair and ends when it derives the shared secret $\mathtt{PRK}_{\mathtt{exporter}}$. For a `Responder`, the handshake

starts when it receives the first message and ends when it derives the shared secret $\text{PRK}_{\texttt{exporter}}$. Each measurement is the average of 100 handshake runs. Generally, `Initiator` takes slightly longer time than `Responder` due to the additional time required for generating the ephemeral KEM key pair at the beginning of the handshake.

Specifically, EDHOC SIG-SIG (Ed25519) has the shortest total handshake time for both parties, 6.86 ms for Initiator and 6.68 ms for `Responder`. Taking the handshake time of EDHOC as the baseline, KEMEDHOC (ML-KEM-512) incurs an overhead of 2.00 ms (≈**29.2%**) for `Initiator` and 1.85 ms (≈**27.7%**) for `Responder`. This overhead is mainly due to the more expensive ML-KEM-512 operations compared to the classical X25519 and Ed25519 operations used in EDHOC. PQ-EDHOC (ML-KEM-512/FALCON1) has the longest handshake time, with an overhead of 3.32 ms (≈**48.4%**) for `Initiator` and 3.35 ms (≈**50.1%**) for `Responder` compared to EDHOC. This drastical increase in handshake time is primarily attributed to the computationally intensive both ML-KEM-512 and FALCON1 operations involved in PQ-EDHOC. Between the two post-quantum variants, KEMEDHOC (ML-KEM-512) outperforms PQ-EDHOC (ML-KEM-512/FALCON1) by 1.32 ms (≈**15.5%**) for `Initiator` and 1.50 ms (≈**17.3%**) for `Responder`.

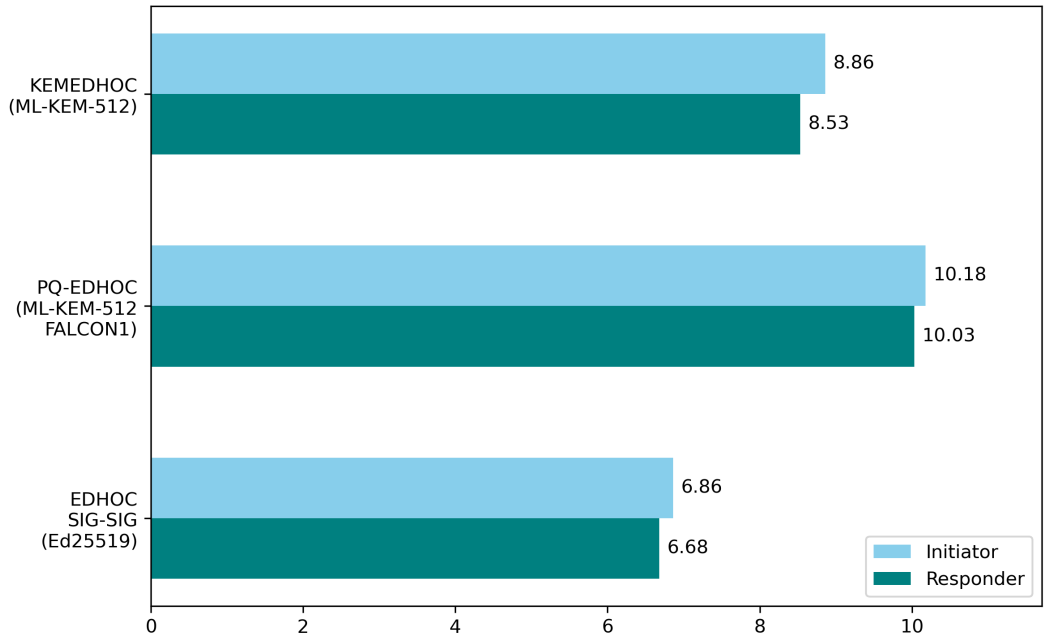



**Figure 13:** Total Handshake Time (ms).

**Size of handshake messages.** Since the compact message size is one of the main design goals of EDHOC [SMP24], we compare the size of handshake messages of EDHOC SIG-SIG (Ed25519), PQ-EDHOC (ML-KEM-512/FALCON1), and KEMEDHOC (ML-KEM-512). We instanitiate EDHOC with SIG-SIG authentication method as this is the mode that consumes the most space among all authentication methods in EDHOC [FVW24]. Comparing the most space-consuming mode of

pre-quantum EDHOC with the post-quantum variants gives us a better understanding of the space overhead introduced by post-quantum cryptography.

Table 15 presents the results of the size of handshake messages in bytes across the three protocols. It is evident that EDHOC substantially surpasses both quantum-safe variants in terms of compactness, the smaller the better. KEMEDHOC introduces more than 4× the message size compared to EDHOC, while PQ-EDHOC incurs even more overhead, with a message size exceeding 6× that of EDHOC. In KEMEDHOC, the size of the third message remains unchanged compared to EDHOC, as KEMEDHOC preserves the structure of the third message in EDHOC. However, the sizes of the first and second messages increase significantly due to the larger ML-KEM-512 public keys and ciphertexts compared to the classical X25519 public keys and Ed25519 signatures used in EDHOC. In PQ-EDHOC, the sizes of all three messages increase drastically, as both the ML-KEM-512 public keys/ciphertexts and FALCON1 signatures are considerably larger than their legacy counterparts. In addition, the the large size of quantum-safe variant of X.509 certificate chain is also the main contributor to the increased size of messages in PQ-EDHOC.

In summary, although both KEMEDHOC and PQ-EDHOC provide post-quantum security, this enhancement results in a substantial increase in handshake message size relative to the original EDHOC. Nevertheless, KEMEDHOC achieves a comparatively more compact message size, approximately *two-thirds* that of PQ-EDHOC, by adopting the KEM algorithm for both key exchange and authentication.

| | **EDHOC SIG-SIG (Ed25519) [Hög+24]** | **PQ-EDHOC (ML-KEM-512/FALCON1) [Fra+25c]** | **KEMEDHOC (ML-KEM-512)** |
|---|---|---|---|
| Message 1 | 37 | 806 | 1884 |
| Message 2 | 398 | 3140 | 1902 |
| Message 3 | 373 | 2381 | 373 |
| **Total** | 808 | 6327 | **4159** |

**Table 15:** Size of handshake messages (in bytes) for EDHOC SIG-SIG, PQ-EDHOC, and KEMEDHOC.

**Size of the Codebase.** Table 16 shows the lines of code (LOC) of EDHOC* and KEMEDHOC* in F*, Low*, and C, excluding whitespace and comments. EDHOC* contains 4,563 LOC in F* and 4,838 LOC in Low*, resulting in 9,401 LOC in total for proof-oriented implementation. On the other hand, KEMEDHOC* has 1,769 LOC in F* and 4,455 LOC in Low*, totaling 6,224 LOC. As a point of comparision, the Core Protocol module of Noise* [Ho+22] has 15,506 LOC in total, making EDHOC* and KEMEDHOC* relatively compact projects in the realm of verified protocol implementations. The C code extracted from EDHOC* has 3,548 LOC. In comparision, the C codebase of $\mu$EDHOC, excluding crypto libraries, CBOR encoding,

and CoAP implementation, has 2,855 LOC, showing that the verified implementation EDHOC* is approximately 1.22× the size of the unverified counterpart in $\mu$EDHOC. This extra size could be problematic for resource-constrained devices which typically have around 100's of KBs of code memory.

The protocol core module, excluding crypto libraries, CBOR encoding, and CoAP implementation, of PQ-EDHOC [Fra+25c] has 3,840 LOC in C, while KEMEDHOC* has 2,593 LOC in C, making KEMEDHOC* approximately 0.67× the size of PQ-EDHOC. Moreover, since KEMEDHOC* only requires the ML-KEM-512 algorithm for both key exchange and authentication, while PQ-EDHOC needs both ML-KEM-512 and FALCON1, KEMEDHOC* has a smaller overall codebase size when considering the crypto libraries as well.

| | F* LOC | Low* LOC | C LOC | Verification time |
|---|---|---|---|---|
| **EDHOC*** | 4,563 | 4,838 | 3,548 | 150m13s |
| **KEMEDHOC*** | 1,769 | 4,455 | 2,593 | 110m3s |

**Table 16:** LOC, excluding whitespace and comments, and verification time for EDHOC* and KEMEDHOC*. The time shown above does not include the verification time for HACL*.

**Proof Overhead.** A common approach to assess the proof overhead of verified implementations is to compare the lines of proof-oriented code (Low* LOC) with the lines of extracted code (C LOC). This proof-to-code ratio provides insights into the additional human effort, quantified in lines of Low* code, required to implement each line of C. In our experiment, EDHOC* has a proof-to-code ratio of approximately 1.36 (4,838 Low* LOC to 3,548 C LOC), while KEMEDHOC* has a ratio of about 1.71 (4,455 Low* LOC to 2,593 C LOC). These ratios are relatively close to the proof-to-code ratio of 1.0 reported in the Noise* project [Ho+22] for the all-pattern model. However, if we consider the single-pattern model in Noise*, which is more comparable to our single-protocol implementations, the proof-to-code ratio is 7.0 [Ho+22], significantly higher than both EDHOC* and KEMEDHOC*.

Another metric to evaluate the proof overhead is the verification time, which indicates the computational resources required to verify the implementation. Table 16 shows the verification time for EDHOC* and KEMEDHOC* at the very first verification run without any cached results from previous runs. EDHOC* takes 150 minutes and 13 seconds for verification, while KEMEDHOC* requires 110 minutes and 3 seconds. The verification time for both implementations is relatively long, making this verification approach seems impractical for continuous integration (CI) purposes. However, it is important to note that the verification time can be substantially reduced in subsequent verification runs due to the caching and incremental verification mechanism of F*. Furthermore, the verification can be performed in parallel across multiple cores, which can further reduce the wall-clock time.

# 8 Discussion

In this section, we discuss future directions and propose potential improvements to the current work. We organize this section into two subsections corresponding to the two research objectives of this thesis project.

## 8.1 Post-quantum migration

One of the notable challenges in post-quantum migration is the noticeable overhead, especially in memory usage, introduced by quantum-safe cryptographic primitives. As shown in Section 7, the total message size of KEMEDHOC is around 4 times larger than that of EDHOC, mainly due to the large sizes of KEM ciphertexts and public keys. At the time of writing, we have observed two potential directions to mitigate this issue. First, refactoring an existing KEM scheme to reduce the size of ciphertexts and public keys, which are two values that are transmitted over the network [Has+25; Hül+21]. Second, adopting a post-quantum non-interactive key exchange (NIKE) scheme, which drop-in replaces the DH key exchange, instead of using KEMs for both ephemeral key exchange and peer authentication [Gaj+24; Cas+18]. We discuss these two directions as follows.

**Reinforcing KEM (RKEM).** Table 15 shows that the message 2 is the most memory-consuming across evaluated protocols. In KEMEDHOC design, the `Responder` needs to send two KEM ciphertexts in the message 2, one for ephemeral key exchange and another one for peer authentication. We notice that the *reinforcing* KEM scheme [Has+25] can offer a single ciphertext that serves both confidentiality and authentication purposes. Informally, RKEM is a special KEM that takes the ephemeral public key $\texttt{pk}_\texttt{e}$ as a *reinfrocing* material to enhance the security of the long-term static public key $\texttt{pk}_\texttt{s}$ [Has+25]. In particular, the ciphertext generated by Rebar, a *reinforcing* KEM, is 1088 bytes, approximately **30%** smaller than the total size of two ML-KEM-512 ciphertexts (1536 bytes). On the other hand, the ephemeral public key from RKEM is 64 bytes larger than that of ML-KEM-512. Therefore, the expected decrease of 512 bytes in the total message size in KEMEDHOC, around **12%**, can be achieved by adopting Rebar.

**Non-interactive key exchange (NIKE).** A non-interactive key exchange protocol enables multiple parties to agree on a shared secret without any interaction [Fre+13]. A party can established a shared secret by using its own private key and the other party's public key, and the other party can derive the same secret using its own private key and the first party's public key. For instance, the static-DH-based authentication methods in EDHOC operates as a NIKE scheme. Furthermore, it is studied that using NIKE for authenticating channels can reduce communication cost, which is essential for resource-constrained devices [Cap+13; BMP04].

CTIDH [Ban+21] emerges as a promising candidate for post-quantum NIKE due to its small key size and constant-time computations. In particular, the public key size

of CTIDH-1024 is 128 bytes, only 96 bytes larger than that of ellitpic curve DH (32 bytes). By replacing the DH key exchange with CTIDH-1024, the quantum-safe variant of EDHOC introduces only 192 bytes overhead (two public keys need to be exchange between two parties), which is a negligible overhead for achieving quantum-safe security even in the resource-constrained environments.

Furthermore, as CTIDH and DH key exchange share a similar syntactical structure, the integration of CTIDH into EDHOC is straightforward. Recently, a faster and deterministic version of CTIDH, namely dCTIDH [Cam+25], is published and boosts the performance of CTIDH by 17%. This improvement further enhances the feasibility of using CTIDH to replace DH key exchange in EDHOC.

Although the aforementioned post-quantum schemes discussed above show promising potential, there is no verified implementation available for them at the time of writing. In a wider scope, only ML-KEM has a verified implementation among NIST PQC finalists and alternatives. Hence, more efforts are needed to develop verified implementations of post-quantum cryptographic primitives to facilitate the post-quantum migration in security protocols in a high-assurance manner.

## 8.2 Protocol Verification and Implementation

**Computational security analysis, addressing [L1].** We provide a symbolic proof of KEMEDHOC that comprehensively covers security properties required for a lightweight authenticated key exchange protocol. However, the symbolic proof does make idealizing assumptions on cryptographic primitives, a computational proof that provides stronger security guarantees is needed to cover corner cases. The *multi-stage* model, which is adopted in the analysis of EDHOC key schedule [GM23], can be a feasiable reference for conducting a computational proof of KEMEDHOC. Regarding the protcol flow, the security analysis of PQ-WireGuard [Hül+21; Has+25] presents a good reference on how to conduct a computational proof for KEM-based authenticated key exchange protocols. It is important to note that symbolic proofs and computational proofs are not mutually exclusive, and they can complement each other to provide a more comprehensive security analysis.

**Verified parsing and serialization, addressing [L2].** In our implementations of EDHOC* and KEMEDHOC*, we do not use a verified CBOR parsing library as there is no such library available. Instead, we use an unverified CBOR library, zcbor, with the risk of potential vulnerabilities in the parsing routines. However, at the time of writing, PulseParse [Ram+25], a verified library for parsing and serializing that supports CBOR, CDDL, and COSE, is published. In future work, PulseParse can be integrated into our implementations to achieve a more robust and higher-assurance implementation. Additionally, C.509 certificates are CBOR-encoded X.509 certificates. A verified module for parsing C.509 certificates based on PulseParse, making our implementations more reliable.

**Formal correspondence between symbolic model and F* specification, addressing [L3].** We build the symbolic model of KEMEDHOC by thoroughly transcribing its F* specification in the input language of ProVerif protocol analyzer. This transcription is conducted manually, which is not machine-checkable. In future work, a correct correspondence between the symbolic model and the F* specification can be established by using the DY* [Bha+21] framework. DY* is a formal verification framework for cryptographic protocols written in F*. Hence, queries for verifying security properties can be embedded directly in the F* specification of KEMEDHOC, and then transfered into DY* to produce verification results. By applying this approach, the provably correct correpondence between the symbolic model and the F* specification can be obtained without extensive manual efforts.

**Comprehensive benchmarking in a low-bandwidth environment, addressing [L4].** In the current experiments, the handshake protocol is operated by internal sockets within a commercial laptop. This setting does not reflect the typical usage scenario of lightweight key exchange protocol in which both computing and communicating power are limited. In future work, we could deploy KEMEDHOC* and EDHOC* on microcontrollers, such as nRF52840 chip [Sem24], and conduct experiments in a low-bandwidth and high-latency communicating environment, limited to 46 kbps [GW22] or even lower. Furthermore, the memory consumption in Random Access Memory (RAM) during execution and the energy consumption on low-power devices can be investigated to have a more comprehensive understanding of the computing overhead of KEMEDHOC itself and the verified implementations.

Furthermore, once the implementations of KEM-5 EDHOC [Fra+25a] and KEM-3 EDHOC [Fra+25b], the two KEM-based post-quantum alternatives, are available, a comprehensive performance can be conducted to provide comparison among KEMEDHOC, KEM-5 EDHOC, and KEM-3 EDHOC to provide more insights into the performance of different KEM-based post-quantum variants of EDHOC. Although KEM-3 EDHOC and KEMEDHOC have the same protocol flow, the simplified key schedule in KEM-3 EDHOC may lead to considerable performance differences, especially in terms of handshake time and memory/power consumption.

**Enhanced usability and scalability of formal methods in software projects.** Many formal verification tools are still academic prototypes which lacks easy-to-follow documentation and stable backend. Moreover, the over-complicated toolchain and the steep learning curve of formal methods discourage software engineers, even those have several years of programming experience, from adopting formal methods in their works [Mer23]. In fact, verified code deployed in production systems is still mainly written by verification researchers, or proof engineers. The development of formally verified software depends heavily on the expertise of verification researchers and specific tools, which undermines the scalability of formal methods in real-world software projects.

To address this issue, a new framework, `hax` [Bha+24a], is proposed to make formal verification more scalable and accessible to software developers. `hax` is a

verification framework that translate a subset of Rust into formal languages, such as F*, Lean, Coq, and ProVerif. Hence, a Rust programmer can include annotations, lemmas, and proofs directly within the Rust code, and subsequently use `hax` to produce executable and verifiable models in multiple formal languages [Bha+24a; Bha+25].

In future work, `hax` can be utilized to implement KEMEDHOC and EDHOC in Rust, and then generate the corresponding models in F* and ProVerif to verify the security properties, functional correctness, and memory safety of the implementations without maintaining separate codebases for each proof backend. Moreover, the centralized codebase in Rust eliminates the concern of formal correspondence between the symbolic model and the F* specification raised by **[L3]**.

# 9 Conclusion

This thesis addresses two main research gaps:

- **[RG1]**: Post-quantum migration of EDHOC.
- **[RG2]**: Design-to-code formal verification of EDHOC.

For **[RG1]**, we present KEMEDHOC, a KEM-based, signature-free, post-quantum variant of EDHOC. Formal analysis of KEMEDHOC using ProVerif shows that it preserves all core security properties of EDHOC—**[SR1] Mutual Authentication**, **[SR2] Confidentiality**, **[SR3] Identity Protection**, **[SR4] Cryptographic Strength**, **[SR5] Downgrade Protection**, and **[SR6] Protection of external security data** — while providing quantum resillience and DoS mitigation.

In terms of performance, KEMEDHOC incurs a moderate handshake time overhead of **27–29%** compared to EDHOC. On the other hand, KEMEDHOC introduces a noticeable increase in message size, resulting in around 4k bytes for total 3 mandatory messages, which is undesirable for constrained IoT devices. Despite these drawbacks, KEMEDHOC is substantially more efficient than PQ-EDHOC [Fra+25c], another post-quantum EDHOC variant using KEMs and quantum-safe signatures. KEMEDHOC achieves a message size reduction of over **33%** and decreases the handshake time by over **15%** compared to PQ-EDHOC.

For **[RG2]**, we introduce EDHOC*, the first formally verified implementation of the core of EDHOC protocol using the F*/Low* ecosystem. The implementation ensures functional correctness, memory safety, and timing side-channel resistance from high-level F* specification down to low-level C code. EDHOC* introduces only a modest code-size increase of **22%** compared to an unverified baseline implementation $\mu$EDHOC, while providing higher assurance. Regarding the extra cost of verification, EDHOC* has a proof-to-code ratio of 1.36 and the verification time of 150 minutes, demonstrating that strong formal guarantees can be achieved with reasonable engineering effort. In summary, EDHOC* is a trustworthy implementation baseline for future deployments of EDHOC in real-world settings.

Moreover, we demonstrate KEMEDHOC*, a formally verified implementation of KEMEDHOC. KEMEDHOC* is the first end-to-end verified implementation of a post-quantum variant of EDHOC. KEMEDHOC* also ensures functional correctness, memory safety, and timing side-channel resistance.

In conclusion, this thesis delivers both a post-quantum-secure redesign and an end-to-end verified implementation of EDHOC. Through KEMEDHOC and its verified implementation, KEMEDHOC*, this work demonstrates that post-quantum security can be achieved without compromising formal assurance or practical efficiency. Together with EDHOC*, the first formally verified implementation of EDHOC, these contributions establish a foundation for developing secure and verifiable lightweight key exchange protocols in the quantum era. Table 17 summarizes the verification progress achieved in this thesis. Moreover, it highlights the remaining open components, such as verified parsing and computational-model proofs, which require further verification and optimization efforts.

| | Verified Specification | Verified Implementation |
|---|---|---|
| EDHOC | ✓ [Jac+23; GM23; CP22; Fer24] | ~ (this work) |
| KEMEDHOC | ✓ (this work) | ~ (this work) |

✓: verified in the cited work(s) ~: partially verified

**Table 17:** Verification status of EDHOC and KEMEDHOC after this thesis project. The verified implementations of EDHOC and KEMEDHOC are incomplete as they lack a verified implementation for handling CBOR and C.509 parsing.

# References


[AF01] Martín Abadi and Cédric Fournet. "Mobile values, new names, and secure communication". In: *SIGPLAN Not.* 36.3 (Jan. 2001), pp. 104–115. ISSN: 0362-1340. DOI: 10.1145/373243.360213.

[Akm+23] Gizem Akman, Mohamed Taoufiq Damir, Philip Ginzboorg, Sampo Sovio, and Valtteri Niemi. "Split keys for station-to-station (STS) protocols". In: *Journal of Surveillance, Security and Safety* 4.3 (2023). ISSN: 2694-1015. DOI: 10.20517/jsss.2023.16.

[Alm+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. "Jasmin: High-Assurance and High-Speed Cryptography". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. Ed. by Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM, 2017, pp. 1807–1823. DOI: 10.1145/3133956.3134078.

[Alm+24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub. "Formally Verifying Kyber - Episode V: Machine-Checked IND-CCA Security and Correctness of ML-KEM in EasyCrypt". In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part II*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14921. Lecture Notes in Computer Science. Springer, 2024, pp. 384–421. DOI: 10.1007/978-3-031-68379-4.

[Ang+22] Yawning Angel, Benjamin Dowling, Andreas Hülsing, Peter Schwabe, and Florian Weber. "Post Quantum Noise". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. CCS '22. Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 97–109. ISBN: 9781450394505. DOI: 10.1145/3548606.3560577.

[AP13] Nadhem J. AlFardan and Kenneth G. Paterson. "Lucky Thirteen: Breaking the TLS and DTLS Record Protocols". In: *2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19-22, 2013*. IEEE Computer Society, 2013, pp. 526–540. DOI: 10.1109/SP.2013.42.

[Avi+16] Nimrod Aviram, Sebastian Schinzel, Juraj Somorovsky, Nadia Heninger, Maik Dankel, Jens Steube, Luke Valenta, David Adrian, J. Alex Halderman, Viktor Dukhovni, Emilia Kasper, Shaanan Cohney, Susanne Engels, Christof Paar, and Yuval Shavitt. "DROWN: Breaking TLS

Using SSLv2". In: *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*. Ed. by Thorsten Holz and Stefan Savage. USENIX Association, 2016, pp. 689–706. URL: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/aviram.

[Ban+21] Gustavo Banegas, Daniel J. Bernstein, Fabio Campos, Tung Chou, Tanja Lange, Michael Meyer, Benjamin Smith, and Jana Sotáková. "CTIDH: faster constant-time CSIDH". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021.4 (Aug. 2021). Artifact available at https://artifacts.iacr.org/tches/2021/a20, pp. 351–387. DOI: 10.46586/tches.v2021.i4.351-387.

[Bar+11] Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella-Béguelin. "Computer-Aided Security Proofs for the Working Cryptographer". In: *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings*. Ed. by Phillip Rogaway. Vol. 6841. Lecture Notes in Computer Science. Springer, 2011, pp. 71–90. DOI: 10.1007/978-3-642-22792-9_5.

[Bar+20] Gilles Barthe, Sandrine Blazy, Benjamin Grégoire, Rémi Hutin, Vincent Laporte, David Pichardie, and Alix Trieu. "Formal verification of a constant-time preserving C compiler". In: *Proc. ACM Program. Lang.* 4.POPL (2020), 7:1–7:30. DOI: 10.1145/3371075.

[Bar+21] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. "SoK: Computer-Aided Cryptography". In: *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 2021, pp. 777–795. DOI: 10.1109/SP40001.2021.00008.

[BB03] David Brumley and Dan Boneh. "Remote Timing Attacks Are Practical". In: *Proceedings of the 12th USENIX Security Symposium, Washington, D.C., USA, August 4-8, 2003*. USENIX Association, 2003. URL: https://www.usenix.org/conference/12th-usenix-security-symposium/remote-timing-attacks-are-practical.

[BBK17] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate". In: *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 2017, pp. 483–502. DOI: 10.1109/SP.2017.26.

[BDS15] David A. Basin, Jannik Dreier, and Ralf Sasse. "Automated Symbolic Proofs of Observational Equivalence". In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*. Ed. by Indrajit Ray, Ninghui Li, and Christopher Kruegel. ACM, 2015, pp. 1144–1155. DOI: 10.1145/2810103.2813662.

[Ber08] Daniel Bernstein. *ChaCha, a variant of Salsa20*. https://cr.yp.to/chacha/chacha-20080120.pdf. 2008. (Visited on 06/10/2025).

[Beu+15] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. "A Messy State of the Union: Taming the Composite State Machines of TLS". In: *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, 2015, pp. 535–552. DOI: 10.1109/SP.2015.39.

[BFK16] Karthikeyan Bhargavan, Cédric Fournet, and Markulf Kohlweiss. "miTLS: Verifying Protocol Implementations against Real-World Attacks". In: *IEEE Secur. Priv.* 14.6 (2016), pp. 18–25. DOI: 10.1109/MSP.2016.123.

[BGL18] Gilles Barthe, Benjamin Grégoire, and Vincent Laporte. "Secure Compilation of Side-Channel Countermeasures: The Case of Cryptographic "Constant-Time"". In: *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*. IEEE Computer Society, 2018, pp. 328–343. DOI: 10.1109/CSF.2018.00031.

[BH20] Carsten Bormann and Paul E. Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949. Dec. 2020. DOI: 10.17487/RFC8949.

[Bha+21] Karthikeyan Bhargavan, Abhishek Bichhawat, Quoc Huy Do, Pedram Hosseyni, Ralf Küsters, Guido Schmitz, and Tim Würtele. "DY*: A Modular Symbolic Verification Framework for Executable Cryptographic Protocol Code". In: *IEEE European Symposium on Security and Privacy, EuroS&P 2021, Vienna, Austria, September 6-10, 2021*. IEEE, 2021, pp. 523–542. DOI: 10.1109/EUROSP51992.2021.00042.

[Bha+24a] Karthikeyan Bhargavan, Maxime Buyse, Lucas Franceschino, Lasse Letager Hansen, Franziskus Kiefer, Jonas Schneider-Bensch, and Bas Spitters. "hax: Verifying Security-Critical Rust Software Using Multiple Provers". In: *Verified Software. Theories, Tools and Experiments - 16th International Conference, VSTTE 2024, Prague, Czech Republic, October 14-15, 2024, Revised Selected Papers*. Ed. by Jonathan Protzenko and Azalea Raad. Vol. 15525. Lecture Notes in Computer Science. Springer, 2024, pp. 96–119. DOI: 10.1007/978-3-031-86695-1_7.

[Bha+24b] Karthikeyan Bhargavan, Lucas Franceschino, Franziskus Kiefer, and Goutam Tamvada. *Verifying Libcrux's ML-KEM*. 2024. URL: https://web.archive.org/web/20250912025104/https://cryspen.com/post/ml-kem-verification/.

[Bha+25] Karthikeyan Bhargavan, Lasse Letager Hansen, Franziskus Kiefer, Jonas Schneider-Bensch, and Bas Spitters. "Formal Security and Functional Verification of Cryptographic Protocol Implementations in Rust". In:

*IACR Cryptol. ePrint Arch.* (2025), p. 980. URL: https://eprint.iacr.org/2025/980.

[Bla+23] Bruno Blanchet, Ben Smyth, Vincent Cheval, and Marc Sylvestre. *ProVerif 2.05: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial*. https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf. Inria, online manual. 2023. (Visited on 06/05/2025).

[Bla13] Bruno Blanchet. "Automatic Verification of Security Protocols in the Symbolic Model: The Verifier ProVerif". In: *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures*. Ed. by Alessandro Aldini, Javier López, and Fabio Martinelli. Vol. 8604. Lecture Notes in Computer Science. Springer, 2013, pp. 54–87. DOI: 10.1007/978-3-319-10082-1_3.

[Bla23] Bruno Blanchet. "CryptoVerif: a Computationally-Sound Security Protocol Verifier (Initial Version with Communications on Channels)". In: *CoRR* abs/2310.14658 (2023). DOI: 10.48550/ARXIV.2310.14658. arXiv: 2310.14658.

[Blu+25] Uri Blumenthal, Brandon Luo, Sean O'Melia, Gabriel Torres, and David A. Wilson. *PQuAKE - Post-Quantum Authenticated Key Exchange*. Internet-Draft draft-uri-lake-pquake-00. Work in Progress. Internet Engineering Task Force, Apr. 2025. 17 pp. URL: https://datatracker.ietf.org/doc/draft-uri-lake-pquake/00/.

[BMP04] Colin Boyd, Wenbo Mao, and Kenneth G. Paterson. "Key Agreement Using Statically Keyed Authenticators". In: *Applied Cryptography and Network Security, Second International Conference, ACNS 2004, Yellow Mountain, China, June 8-11, 2004, Proceedings*. Ed. by Markus Jakobsson, Moti Yung, and Jianying Zhou. Vol. 3089. Lecture Notes in Computer Science. Springer, 2004, pp. 248–262. DOI: 10.1007/978-3-540-24852-1_18.

[Boe+08] Sharon Boeyen, Stefan Santesson, Tim Polk, Russ Housley, Stephen Farrell, and David Cooper. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. May 2008. DOI: 10.17487/RFC5280.

[Bog+10] Andrey Bogdanov, Thomas Eisenbarth, Christof Paar, and Malte Wienecke. "Differential Cache-Collision Timing Attacks on AES with Applications to Embedded CPUs". In: *Topics in Cryptology - CT-RSA 2010, The Cryptographers' Track at the RSA Conference 2010, San Francisco, CA, USA, March 1-5, 2010. Proceedings*. Ed. by Josef Pieprzyk. Vol. 5985. Lecture Notes in Computer Science. Springer, 2010, pp. 235–251. DOI: 10.1007/978-3-642-11925-5_17.

[Bon+17] Barry Bond, Chris Hawblitzel, Manos Kapritsos, K. Rustan M. Leino, Jacob R. Lorch, Bryan Parno, Ashay Rane, Srinath T. V. Setty, and Laure Thompson. "Vale: Verifying High-Performance Cryptographic Assembly Code". In: *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16-18, 2017*. Ed. by Engin Kirda and Thomas Ristenpart. USENIX Association, 2017, pp. 917–934. URL: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/bond.

[BR06] Mihir Bellare and Phillip Rogaway. "The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs". In: *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings*. Ed. by Serge Vaudenay. Vol. 4004. Lecture Notes in Computer Science. Springer, 2006, pp. 409–426. DOI: 10.1007/11761679_25.

[BT11] Billy Bob Brumley and Nicola Tuveri. "Remote Timing Attacks Are Still Practical". In: *Computer Security - ESORICS 2011 - 16th European Symposium on Research in Computer Security, Leuven, Belgium, September 12-14, 2011. Proceedings*. Ed. by Vijay Atluri and Claudia Díaz. Vol. 6879. Lecture Notes in Computer Science. Springer, 2011, pp. 355–371. DOI: 10.1007/978-3-642-23822-2_20.

[BT18] Clark W. Barrett and Cesare Tinelli. "Satisfiability Modulo Theories". In: *Handbook of Model Checking*. Ed. by Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem. Springer, 2018, pp. 305–343. DOI: 10.1007/978-3-319-10575-8_11.

[Cam+25] Fabio Campos, Andreas Hellenbrand, Michael Meyer, and Krijn Reijnders. "dCTIDH: Fast & Deterministic CTIDH". In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2025.3 (2025), pp. 516–541. DOI: 10.46586/TCHES.V2025.I3.516-541.

[Can01] Ran Canetti. "Universally Composable Security: A New Paradigm for Cryptographic Protocols". In: *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*. IEEE Computer Society, 2001, pp. 136–145. DOI: 10.1109/SFCS.2001.959888.

[Cap+13] Cagatay Capar, Dennis Goeckel, Kenneth G. Paterson, Elizabeth A. Quaglia, Don Towsley, and Murtaza Zafer. "Signal-flow-based analysis of wireless security protocols". In: *Inf. Comput.* 226 (2013), pp. 37–56. DOI: 10.1016/J.IC.2013.03.004.

[Cas+18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. "CSIDH: An Efficient Post-Quantum Commutative Group Action". In: *Advances in Cryptology – ASIACRYPT 2018*. Vol. 11274. Lecture Notes in Computer Science. Springer, 2018, pp. 395–427. DOI: 10.1007/978-3-030-03332-3_15.

[CB15] David Cadé and Bruno Blanchet. “Proved generation of implementations from computationally secure protocol specifications”. In: *J. Comput. Secur.* 23.3 (2015), pp. 331–402. DOI: 10.3233/JCS-150524.

[CK01] Ran Canetti and Hugo Krawczyk. “Analysis of Key-Exchange Protocols and Their Use for Building Secure Channels”. In: *Proceedings of the International Conference on the Theory and Application of Cryptographic Techniques: Advances in Cryptology*. EUROCRYPT ’01. Berlin, Heidelberg: Springer-Verlag, 2001, pp. 453–474. ISBN: 3540420703. DOI: 10.1007/3-540-44987-6.

[CKR18] Vincent Cheval, Steve Kremer, and Itsaka Rakotonirina. “DEEPSEC: Deciding Equivalence Properties in Security Protocols Theory and Practice”. In: *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*. IEEE Computer Society, 2018, pp. 529–546. DOI: 10.1109/SP.2018.00033.

[Cor] Intel Corporation. *TinyCrypt*. URL: https://github.com/intel/tinycrypt (visited on 10/04/2025).

[Cor+08] Ricardo Corin, Pierre-Malo Deniélou, Cédric Fournet, Karthikeyan Bhargavan, and James J. Leifer. “A secure compiler for session abstractions”. In: *J. Comput. Secur.* 16.5 (2008), pp. 573–636. DOI: 10.3233/JCS-2008-0334.

[Cor17] Mozilla Corporation. *[meta] Integrate verified cryptographic primitives into NSS from the HACL* library*. https://web.archive.org/web/20250526025631mp_/https://bugzilla.mozilla.org/show_bug.cgi?id=1387183. 2017.

[CP22] Baptiste Cottier and David Pointcheval. “Security Analysis of Improved EDHOC Protocol”. In: *Foundations and Practice of Security - 15th International Symposium, FPS 2022, Ottawa, ON, Canada, December 12-14, 2022, Revised Selected Papers*. Ed. by Guy-Vincent Jourdan, Laurent Mounier, Carlisle M. Adams, Florence Sèdes, and Joaquín García-Alfaro. Vol. 13877. Lecture Notes in Computer Science. Springer, 2022, pp. 3–18. DOI: 10.1007/978-3-031-30122-3_1.

[CS03] Ronald Cramer and Victor Shoup. “Design and Analysis of Practical Public-Key Encryption Schemes Secure against Adaptive Chosen Ciphertext Attack”. In: *SIAM Journal on Computing* 33.1 (2003), pp. 167–226. DOI: 10.1137/S0097539702403773.

[CZB20] Bharat S. Chaudhari, Marco Zennaro, and Suresh Borkar. “LPWAN Technologies: Emerging Application Characteristics, Requirements, and Design Considerations”. In: *Future Internet* 12.3 (2020). ISSN: 1999-5903. DOI: 10.3390/fi12030046.

[Dam+22] Mohamed Taoufiq Damir, Tommi Meskanen, Sara Ramezanian, and Valtteri Niemi. "A Beyond-5G Authentication and Key Agreement Protocol". In: *Network and System Security: 16th International Conference, NSS 2022, Denarau Island, Fiji, December 9–12, 2022, Proceedings*. Denarau Island, Fiji: Springer-Verlag, 2022, pp. 249–264. ISBN: 978-3-031-23019-6. DOI: 10.1007/978-3-031-23020-2_14.

[Dat14] National Vulnerability Database. *Heartbleed bug*. Available from NVD, CVE-2014-0160. Apr. 2014. URL: https://web.nvd.nist.gov/view/vuln/detail?vulnId=CVE-2014-0160 (visited on 03/12/2025).

[Del+21] Antoine Delignat-Lavaud, Cédric Fournet, Bryan Parno, Jonathan Protzenko, Tahina Ramananandro, Jay Bosamiya, Joseph Lallemand, Itsaka Rakotonirina, and Yi Zhou. "A Security Model and Fully Verified Implementation for the IETF QUIC Record Layer". In: *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 2021, pp. 1162–1178. DOI: 10.1109/SP40001.2021.00039.

[Den03] Alexander W. Dent. "A Designer's Guide to KEMs". In: *Cryptography and Coding*. Ed. by Kenneth G. Paterson. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 133–151. ISBN: 978-3-540-40974-8.

[DH76] Whitfield Diffie and Martin E. Hellman. "New directions in cryptography". In: *IEEE Trans. Inf. Theory* 22.6 (1976), pp. 644–654. DOI: 10.1109/TIT.1976.1055638.

[Dij75] Edsger W. Dijkstra. "Guarded Commands, Nondeterminacy and Formal Derivation of Programs". In: *Commun. ACM* 18.8 (1975), pp. 453–457. DOI: 10.1145/360933.360975.

[DY81] Danny Dolev and Andrew Chi-Chih Yao. "On the Security of Public Key Protocols (Extended Abstract)". In: *22nd Annual Symposium on Foundations of Computer Science, Nashville, Tennessee, USA, 28-30 October 1981*. IEEE Computer Society, 1981, pp. 350–357. DOI: 10.1109/SFCS.1981.32.

[Erb+19] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. "Simple High-Level Code for Cryptographic Arithmetic - With Proofs, Without Compromises". In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1202–1219. DOI: 10.1109/SP.2019.00005.

[Ero+10] Pasi Eronen, Yoav Nir, Paul E. Hoffman, and Charlie Kaufman. *Internet Key Exchange Protocol Version 2 (IKEv2)*. RFC 5996. Sept. 2010. DOI: 10.17487/RFC5996.

[Fer24] Loïc Ferreira. "Computational Security Analysis of the Full EDHOC Protocol". In: *Topics in Cryptology - CT-RSA 2024 - Cryptographers' Track at the RSA Conference 2024, San Francisco, CA, USA, May 6-9, 2024, Proceedings*. Ed. by Elisabeth Oswald. Vol. 14643. Lecture Notes

in Computer Science. Springer, 2024, pp. 25–48. DOI: 10.1007/978-3-031-58868-6_2.

[FG14] Marc Fischlin and Felix Günther. "Multi-Stage Key Exchange and the Case of Google's QUIC Protocol". In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*. Ed. by Gail-Joon Ahn, Moti Yung, and Ninghui Li. ACM, 2014, pp. 1193–1204. DOI: 10.1145/2660267.2660308.

[FGR09] Cédric Fournet, Gurvan Le Guernic, and Tamara Rezk. "A security-preserving compiler for distributed programs: from information-flow policies to cryptographic mechanisms". In: *Proceedings of the 2009 ACM Conference on Computer and Communications Security, CCS 2009, Chicago, Illinois, USA, November 9-13, 2009*. Ed. by Ehab Al-Shaer, Somesh Jha, and Angelos D. Keromytis. ACM, 2009, pp. 432–441. DOI: 10.1145/1653662.1653715.

[Fir] Trusted Firmware. *Mbed TLS*. URL: https://www.trustedfirmware.org/projects/mbed-tls/ (visited on 10/04/2025).

[Fou+20] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*
*Specification v1.2*. Tech. rep. Available at https://falcon-sign.info/. NIST Post-Quantum Cryptography Standardization Process, Oct. 2020. URL: https://falcon-sign.info/falcon.pdf.

[Fra+25a] Lidia Pocero Fraile, Christos Koulamas, Apostolos P. Fournaris, and Evangelos Haleplidis. *KEM-based Authentication for EDHOC*. Internet-Draft draft-pocero-authkem-edhoc-00. Work in Progress. Internet Engineering Task Force, July 2025. 33 pp. URL: https://datatracker.ietf.org/doc/draft-pocero-authkem-edhoc/00/.

[Fra+25b] Lidia Pocero Fraile, Christos Koulamas, Apostolos P. Fournaris, and Evangelos Haleplidis. *KEM-based Authentication for EDHOC in Initiator-Known Responder (IKR) Scenarios*. Internet-Draft draft-pocero-authkem-ikr-edhoc-00. Work in Progress. Internet Engineering Task Force, July 2025. 28 pp. URL: https://datatracker.ietf.org/doc/draft-pocero-authkem-ikr-edhoc/00/.

[Fra+25c] Lidia Pocero Fraile, Georgios Tasopoulos, Christos Koulamas, Raymond K. Zhao, Nazatul Haque Sultan, Francesco Regazzoni, and Apostolos P. Fournaris. "Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation". In: *IEEE Access* 13 (2025), pp. 75861–75884. DOI: 10.1109/ACCESS.2025.3554010.

[Fre+13] Eduarda S. V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. “Non-Interactive Key Exchange”. In: *Public-Key Cryptography - PKC 2013 - 16th International Conference on Practice and Theory in Public-Key Cryptography, Nara, Japan, February 26 - March 1, 2013. Proceedings*. Ed. by Kaoru Kurosawa and Goichiro Hanaoka. Vol. 7778. Lecture Notes in Computer Science. Springer, 2013, pp. 254–271. DOI: 10.1007/978-3-642-36362-7_17.

[Fuj+12] Atsushi Fujioka, Koutarou Suzuki, Keita Xagawa, and Kazuki Yoneyama. “Strongly Secure Authenticated Key Exchange from Factoring, Codes, and Lattices”. In: *Public Key Cryptography - PKC 2012 - 15th International Conference on Practice and Theory in Public Key Cryptography, Darmstadt, Germany, May 21-23, 2012. Proceedings*. Ed. by Marc Fischlin, Johannes Buchmann, and Mark Manulis. Vol. 7293. Lecture Notes in Computer Science. Springer, 2012, pp. 467–484. DOI: 10.1007/978-3-642-30057-8_28.

[FVW24] Geovane Fedrecheski, Malisa Vucinic, and Thomas Watteyne. “Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments”. In: *IEEE Wireless Communications and Networking Conference, WCNC 2024, Dubai, United Arab Emirates, April 21-24, 2024*. IEEE, 2024, pp. 1–6. DOI: 10.1109/WCNC57260.2024.10570830.

[Gaj+24] Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. “SWOOSH: Efficient Lattice-Based Non-Interactive Key Exchange”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 487–504. ISBN: 978-1-939133-44-1. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/gajland.

[GCR23] Alexandre Augusto Giron, Ricardo Custódio, and Francisco Rodríguez-Henríquez. “Post-quantum hybrid key exchange: a systematic mapping study”. In: *J. Cryptogr. Eng.* 13.1 (2023), pp. 71–88. DOI: 10.1007/S13389-022-00288-9.

[Geo+12] Martin Georgiev, Subodh Iyengar, Suman Jana, Rishita Anubhai, Dan Boneh, and Vitaly Shmatikov. “The most dangerous code in the world: validating SSL certificates in non-browser software”. In: *the ACM Conference on Computer and Communications Security, CCS’12, Raleigh, NC, USA, October 16-18, 2012*. Ed. by Ting Yu, George Danezis, and Virgil D. Gligor. ACM, 2012, pp. 38–49. DOI: 10.1145/2382196.2382204.

[GM23] Felix Günther and Marc Ilunga Tshibumbu Mukendi. “Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol”. In: *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*. IEEE, 2023, pp. 773–796. DOI: 10.1109/EUROSP57164.2023.00051.

[Goo17] Google. *Fiat Cryptography in BoringSSL*. https://web.archive.org/web/20250124100638mp_/https://boringssl.googlesource.com/boringssl/+/HEAD/third_party/fiat/. 2017. (Visited on 06/11/2025).

[GW22] Ruben Gonzalez and Thom Wiggers. "KEMTLS vs. Post-quantum TLS: Performance on Embedded Systems". In: *Security, Privacy, and Applied Cryptography Engineering - 12th International Conference, SPACE 2022, Jaipur, India, December 9-12, 2022, Proceedings*. Ed. by Lejla Batina, Stjepan Picek, and Mainack Mondal. Vol. 13783. Lecture Notes in Computer Science. Springer, 2022, pp. 99–117. DOI: 10.1007/978-3-031-22829-2_6.

[Hal05] Shai Halevi. "A plausible approach to computer-aided cryptographic proofs". In: *IACR Cryptol. ePrint Arch.* (2005), p. 181. URL: http://eprint.iacr.org/2005/181.

[Has+25] Keitaro Hashimoto, Shuichi Katsumata, Guilhem Niot, and Thom Wiggers. "Revisiting PQ WireGuard: A Comprehensive Security Analysis With a New Design Using Reinforced KEMs". In: *Cryptology ePrint Archive* (2025). URL: https://eprint.iacr.org/2025/1758.pdf.

[Hax+18] Jetmir Haxhibeqiri, Eli De Poorter, Ingrid Moerman, and Jeroen Hoebeke. "A Survey of LoRaWAN for IoT: From Technology to Application". In: *Sensors* 18.11 (2018). ISSN: 1424-8220. DOI: 10.3390/s18113995.

[Hla+15] Clemens Hlauschek, Markus Gruber, Florian Fankhauser, and Christian Schanes. "Prying Open Pandora's Box: KCI Attacks against TLS". In: *9th USENIX Workshop on Offensive Technologies, WOOT '15, Washington, DC, USA, August 10-11, 2015*. Ed. by Aurélien Francillon and Thomas Ptacek. USENIX Association, 2015. URL: https://www.usenix.org/conference/woot15/workshop-program/presentation/hlauschek.

[Ho+22] Son Ho, Jonathan Protzenko, Abhishek Bichhawat, and Karthikeyan Bhargavan. "Noise*: A Library of Verified High-Performance Secure Channel Protocol Implementations". In: *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 2022, pp. 107–124. DOI: 10.1109/SP46214.2022.9833621.

[Hoa69] C. A. R. Hoare. "An Axiomatic Basis for Computer Programming". In: *Commun. ACM* 12.10 (1969), pp. 576–580. DOI: 10.1145/363235.363259.

[Hög+24] Rikard Höglund, Marco Tiloca, Göran Selander, John Preuß Mattsson, Mališa Vučinić, and Thomas Watteyne. "Secure Communication for the IoT: EDHOC and (Group) OSCORE Protocols". In: *IEEE Access* 12 (2024), pp. 49865–49877. DOI: 10.1109/ACCESS.2024.3384095.

[Hor51] Alfred Horn. “On Sentences Which are True of Direct Unions of Algebras”. In: *J. Symb. Log.* 16.1 (1951), pp. 14–21. DOI: 10.2307/2268661.

[Hri+21] Stefan Hristozov, Manuel Huber, Lei Xu, Jaro Fietz, Marco Liess, and Georg Sigl. “The Cost of OSCORE and EDHOC for Constrained Devices”. In: *CODASPY ’21: Eleventh ACM Conference on Data and Application Security and Privacy, Virtual Event, USA, April 26-28, 2021*. Ed. by Anupam Joshi, Barbara Carminati, and Rakesh M. Verma. ACM, 2021, pp. 245–250. DOI: 10.1145/3422337.3447834.

[Hül+21] Andreas Hülsing, Kai-Chun Ning, Peter Schwabe, Fiona Johanna Weber, and Philip R. Zimmermann. “Post-quantum WireGuard”. In: *2021 IEEE Symposium on Security and Privacy (SP)*. 2021, pp. 304–321. DOI: 10.1109/SP40001.2021.00030.

[Ian21] Google Project Zero Ian Beer. *CVE-2021-30737, @xerub’s 2021 iOS ASN.1 Vulnerability*. 2021. URL: https://www.cve.org/CVERecord?id=CVE-2021-30737 (visited on 01/10/2025).

[Inc25] Apple Inc. *Get ahead with quantum-secure cryptography*. https://developer.apple.com/videos/play/wwdc2025/314/. 2025. (Visited on 06/11/2025).

[Jac+23] Charlie Jacomme, Elise Klein, Steve Kremer, and Maı́wenn Racouchot. “A comprehensive, formal and automated analysis of the EDHOC protocol”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5881–5898. ISBN: 978-1-939133-37-3. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/jacomme.

[Kau+16] Thierry Kaufmann, Hervé Pelletier, Serge Vaudenay, and Karine Villegas. “When Constant-Time Source Yields Variable-Time Binary: Exploiting Curve25519-donna Built with MSVC 2015”. In: *Cryptology and Network Security - 15th International Conference, CANS 2016, Milan, Italy, November 14-16, 2016, Proceedings*. Ed. by Sara Foresti and Giuseppe Persiano. Vol. 10052. Lecture Notes in Computer Science. 2016, pp. 573–582. DOI: 10.1007/978-3-319-48965-0_36.

[KBB17] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. “Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach”. In: *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*. IEEE, 2017, pp. 435–450. DOI: 10.1109/EUROSP.2017.38.

[KE10] Hugo Krawczyk and Pasi Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869. May 2010. DOI: 10.17487/RFC5869.

[KNB19] Nadim Kobeissi, Georgio Nicolas, and Karthikeyan Bhargavan. “Noise Explorer: Fully Automated Modeling and Verification for Arbitrary Noise Protocols”. In: *IEEE European Symposium on Security and Privacy, EuroS&P 2019, Stockholm, Sweden, June 17-19, 2019*. IEEE, 2019, pp. 356–370. DOI: 10.1109/EUROSP.2019.00034.

[Koc+19] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. “Spectre Attacks: Exploiting Speculative Execution”. In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1–19. DOI: 10.1109/SP.2019.00002.

[Kra03] Hugo Krawczyk. “SIGMA: The ’SIGn-and-MAc’ Approach to Authenticated Diffie-Hellman and Its Use in the IKE-Protocols”. In: *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*. Ed. by Dan Boneh. Vol. 2729. Lecture Notes in Computer Science. Springer, 2003, pp. 400–425. DOI: 10.1007/978-3-540-45146-4_24.

[Kra10] Hugo Krawczyk. “Cryptographic Extraction and Key Derivation: The HKDF Scheme”. In: *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*. Ed. by Tal Rabin. Vol. 6223. Lecture Notes in Computer Science. Springer, 2010, pp. 631–648. DOI: 10.1007/978-3-642-14623-7_34.

[KSH25] Panos Kampanakis, Douglas Stebila, and Torben Hansen. *PQ/T Hybrid Key Exchange in SSH*. Internet-Draft draft-ietf-sshm-mlkem-hybrid-kex-02. Work in Progress. Internet Engineering Task Force, Apr. 2025. 14 pp. URL: https://datatracker.ietf.org/doc/draft-ietf-sshm-mlkem-hybrid-kex/02/.

[KW16] Hugo Krawczyk and Hoeteck Wee. “The OPTLS Protocol and TLS 1.3”. In: *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. 2016, pp. 81–96. DOI: 10.1109/EuroSP.2016.18.

[Law+03] Laurie Law, Alfred Menezes, Minghua Qu, Jerome A. Solinas, and Scott A. Vanstone. “An Efficient Protocol for Authenticated Key Agreement”. In: *Des. Codes Cryptogr.* 28.2 (2003), pp. 119–134.

[LBB19] Benjamin Lipp, Bruno Blanchet, and Karthikeyan Bhargavan. “A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol”. In: *IEEE European Symposium on Security and Privacy, EuroS&P 2019, Stockholm, Sweden, June 17-19, 2019*. IEEE, 2019, pp. 231–246. DOI: 10.1109/EUROSP.2019.00026.

[Ler09] Xavier Leroy. “Formal verification of a realistic compiler”. In: *Commun. ACM* 52.7 (2009), pp. 107–115. DOI: 10.1145/1538788.1538814.

[Lip+18] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. *Meltdown*. 2018. arXiv: 1801.01207 [cs.CR]. URL: https://arxiv.org/abs/1801.01207.

[Mat+25] John Preuß Mattsson, Göran Selander, Shahid Raza, Joel Höglund, and Martin Furuhed. *CBOR Encoded X.509 Certificates (C509 Certificates)*. Internet-Draft draft-ietf-cose-cbor-encoded-cert-15. Work in Progress. Internet Engineering Task Force, Aug. 2025. 86 pp. URL: https://datatracker.ietf.org/doc/draft-ietf-cose-cbor-encoded-cert/15/.

[MB08] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. "Z3: An Efficient SMT Solver". In: *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*. Ed. by C. R. Ramakrishnan and Jakob Rehof. Vol. 4963. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.

[McC+07] Jay A. McCarthy, Shriram Krishnamurthi, Joshua D. Guttman, and John D. Ramsdell. "Compiling cryptographic protocols for deployment on the web". In: *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada, May 8-12, 2007*. Ed. by Carey L. Williamson, Mary Ellen Zurko, Peter F. Patel-Schneider, and Prashant J. Shenoy. ACM, 2007, pp. 687–696. DOI: 10.1145/1242572.1242665.

[MDK14] Bodo Móller, Thai Duong, and Krzysztof Kotowicz. *This POODLE Bites: Exploiting The SSL 3.0 Fallback*. 2014. URL: https://openssl-library.org/files/ssl-poodle.pdf (visited on 05/31/2025).

[Mer23] Thyla van der Merwe. *Using Formal Methods at Google*. 2023. URL: https://datatracker.ietf.org/meeting/118/materials/slides-118-ufmrg-using-formal-methods-at-google (visited on 10/03/2025).

[Mil85] Victor S. Miller. "Use of Elliptic Curves in Cryptography". In: *Advances in Cryptology - CRYPTO '85, Santa Barbara, California, USA, August 18-22, 1985, Proceedings*. Ed. by Hugh C. Williams. Vol. 218. Lecture Notes in Computer Science. Springer, 1985, pp. 417–426. DOI: 10.1007/3-540-39799-X_31.

[MLW12] Elke Mackensen, Matthias Lai, and Thomas M. Wendt. "Bluetooth Low Energy (BLE) based wireless sensors". In: *SENSORS, 2012 IEEE*. 2012, pp. 1–4. DOI: 10.1109/ICSENS.2012.6411303.

[NOT06] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. “Solving SAT and SAT Modulo Theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)”. In: *J. ACM* 53.6 (Nov. 2006), pp. 937–977. ISSN: 0004-5411. DOI: 10.1145/1217856.1217859.

[NSB20] Karl Norrman, Vaishnavi Sundararajan, and Alessandro Bruni. “Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices”. In: *CoRR* abs/2007.11427 (2020). arXiv: 2007.11427. URL: https://arxiv.org/abs/2007.11427.

[OPp19] David Ott, Christopher Peikert, and other workshop participants. *Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility*. 2019. arXiv: 1909.07353 [cs.CY]. URL: https://arxiv.org/abs/1909.07353.

[ORZ20] Lukasz Olejnik, Robert Riemann, and Thomas Zerdick. *EDPS TechDispatch on Quantum Computing and Cryptography*. Tech. rep. Technology and Privacy understandingnit of the European Data Protection Supervisor (EDPS), 2020. URL: https://www.edps.europa.eu/sites/default/files/publication/06-08-2020_techdispatch_quantum_computing_en.pdf.

[Pér+24] Elsa López Pérez, Göran Selander, John Preuß Mattsson, Thomas Watteyne, and Mališa Vučinić. “EDHOC Is a New Security Handshake Standard: An Overview of Security Analysis”. In: *Computer* 57.9 (2024), pp. 101–110. DOI: 10.1109/MC.2024.3415908.

[Per18] Trevor Perrin. *The Noise Protocol Framework*. https://web.archive.org/web/20250620103640/http://www.noiseprotocol.org/noise.html. 2018. (Visited on 06/22/2025).

[Pie08] Letouzey Pierre. “Extraction in Coq: An Overview”. In: *Logic and Theory of Algorithms*. Ed. by Beckmann Arnold, Dimitracopoulos Costas, and Löwe Benedikt. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 359–369. ISBN: 978-3-540-69407-6. DOI: 10.1007/978-3-540-69407-6_39.

[Pos21] Emil L. Post. “Introduction to a General Theory of Elementary Propositions”. In: *American Journal of Mathematics* 43.3 (1921), pp. 163–185. DOI: 10.2307/2370324.

[Pro+17] Jonathan Protzenko, Jean Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella Béguelin, Antoine Delignat-Lavaud, Catalin Hritcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. “Verified low-level programming embedded in F”. In: *Proc. ACM Program. Lang.* 1.ICFP (2017), 17:1–17:29. DOI: 10.1145/3110261.

[Pro+19] Jonathan Protzenko, Benjamin Beurdouche, Denis Merigoux, and Karthikeyan Bhargavan. "Formally Verified Cryptographic Web Applications in WebAssembly". In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1256–1274. DOI: 10.1109/SP.2019.00064.

[QC25] Mingping Qi and Chi Chen. "HPQKE: Hybrid Post-Quantum Key Exchange Protocol for SSH Transport Layer From CSIDH". In: *IEEE Transactions on Information Forensics and Security* 20 (2025), pp. 2122–2131. DOI: 10.1109/TIFS.2025.3539943.

[Rad+22] Michael Rademacher, Hendrik Linka, Jannis Konrad, Thorsten Horstmann, and Karl Jonas. "Bounds for the Scalability of TLS over LoRaWAN". In: *Mobile Communication - Technologies and Applications; 26th ITG-Symposium*. 2022, pp. 1–6. URL: https://ieeexplore.ieee.org/document/9861875.

[Ram+25] Tahina Ramananandro, Gabriel Ebner, Guido Martínez, and Nikhil Swamy. "Secure Parsing and Serializing with Separation Logic Applied to CBOR, CDDL, and COSE". In: *CoRR* abs/2505.17335 (2025). DOI: 10.48550/ARXIV.2505.17335. arXiv: 2505.17335.

[Res18] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Aug. 2018. DOI: 10.17487/RFC8446.

[Rog02] Phillip Rogaway. "Authenticated-encryption with associated-data". In: *Proceedings of the 9th ACM Conference on Computer and Communications Security*. CCS '02. Washington, DC, USA: Association for Computing Machinery, 2002, pp. 98–107. ISBN: 1581136129. DOI: 10.1145/586110.586125.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems". In: *Commun. ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.

[RTB20] Gabriele Restuccia, Hannes Tschofenig, and Emmanuel Baccelli. "Low-Power IoT Communication Security: On the Performance of DTLS and TLS 1.3". In: *9th IFIP International Conference on Performance Evaluation and Modeling in Wireless Networks, PEMWN 2020, Berlin, Germany, December 1-3, 2020*. IEEE, 2020, pp. 1–6. DOI: 10.23919/PEMWN50727.2020.9293085.

[RTM22] Eric Rescorla, Hannes Tschofenig, and Nagendra Modadugu. *The Datagram Transport Layer Security (DTLS) Protocol Version 1.3*. RFC 9147. Apr. 2022. DOI: 10.17487/RFC9147.

[Sch22] Jim Schaad. *CBOR Object Signing and Encryption (COSE): Structures and Process*. RFC 9052. Aug. 2022. DOI: 10.17487/RFC9052.

[Sel+19] Göran Selander, John Preuß Mattsson, Francesca Palombini, and Ludwig Seitz. *Object Security for Constrained RESTful Environments (OSCORE)*. RFC 8613. July 2019. DOI: 10.17487/RFC8613.

[Sel+25] Göran Selander, John Preuß Mattsson, Mališa Vučinić, Geovane Fedrecheski, and Michael Richardson. *Lightweight Authorization using Ephemeral Diffie-Hellman Over COSE (ELA)*. Internet-Draft draft-ietf-lake-authz-05. Work in Progress. Internet Engineering Task Force, July 2025. 48 pp. URL: https://datatracker.ietf.org/doc/draft-ietf-lake-authz/05/.

[Sem24] Nordic Semiconductor. *nRF52840 Product Specification*. 2024. URL: https://docs.nordicsemi.com/bundle/ps_nrf52840/page/keyfeatures_html5.html (visited on 10/03/2025).

[SFG25] Douglas Stebila, Scott Fluhrer, and Shay Gueron. *Hybrid key exchange in TLS 1.3*. Internet-Draft draft-ietf-tls-hybrid-design-12. Work in Progress. Internet Engineering Task Force, Jan. 2025. 24 pp. URL: https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/12/.

[SHB14] Zach Shelby, Klaus Hartke, and Carsten Bormann. *The Constrained Application Protocol (CoAP)*. RFC 7252. June 2014. DOI: 10.17487/RFC7252.

[Sho04] Victor Shoup. "Sequences of games: a tool for taming complexity in security proofs". In: *IACR Cryptol. ePrint Arch.* (2004), p. 332. URL: http://eprint.iacr.org/2004/332.

[Sho94] Peter W. Shor. "Algorithms for Quantum Computation: Discrete Logarithms and Factoring". In: *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, November 20-22, 1994*. IEEE Computer Society, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.

[Sho97] Peter W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM J. Comput.* 26.5 (1997), pp. 1484–1509. DOI: 10.1137/S0097539795293172.

[SKD20] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. "Post-Quantum Authentication in TLS 1.3: A Performance Study". In: *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020. URL: https://www.ndss-symposium.org/ndss-paper/post-quantum-authentication-in-tls-1-3-a-performance-study/.

[SMP24] Göran Selander, John Preuß Mattsson, and Francesca Palombini. *Ephemeral Diffie-Hellman Over COSE (EDHOC)*. RFC 9528. Mar. 2024. DOI: 10.17487/RFC9528.

[SN17] National Institute of Standards and Technology (NIST). *Post-Quantum Cryptography Standardization*. 2017. URL: https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization (visited on 10/08/2025).

[SPA19] Mohit Sethi, Aleksi Peltonen, and Tuomas Aura. "Misbinding Attacks on Secure Device Pairing and Bootstrapping". In: *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, AsiaCCS 2019, Auckland, New Zealand, July 09-12, 2019*. Ed. by Steven D. Galbraith, Giovanni Russello, Willy Susilo, Dieter Gollmann, Engin Kirda, and Zhenkai Liang. ACM, 2019, pp. 453–464. DOI: 10.1145/3321705.3329813.

[SSW20] Peter Schwabe, Douglas Stebila, and Thom Wiggers. "Post-Quantum TLS Without Handshake Signatures". In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. CCS '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 1461–1480. ISBN: 9781450370899. DOI: 10.1145/3372297.3423350.

[ST24] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Tech. rep. Washington, D.C.: U.S. Department of Commerce, 2024. DOI: 10.6028/NIST.FIPS.203.

[Swa+16] Nikhil Swamy, Catalin Hritcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean Karim Zinzindohoue, and Santiago Zanella Béguelin. "Dependent types and multi-monadic effects in F". In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*. Ed. by Rastislav Bodík and Rupak Majumdar. ACM, 2016, pp. 256–270. DOI: 10.1145/2837614.2837655.

[Tao+21] Zhe Tao, Aseem Rastogi, Naman Gupta, Kapil Vaswani, and Aditya V. Thakur. "DICE*: A Formally Verified Implementation of DICE Measured Boot". In: *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. Ed. by Michael D. Bailey and Rachel Greenstadt. USENIX Association, 2021, pp. 1091–1107. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/tao.

[Tec89] Massachusetts Institute of Technology. *Kerberos: The Network Authentication Protocol*. 1989. URL: https://web.mit.edu/kerberos/ (visited on 01/10/2025).

[Vin81] Bob Kahn Vint Cerf. *Transmission Control Protocol*. RFC 793. Sept. 1981. DOI: 10.17487/RFC0793.

[Vol+04] John Vollbrecht, James D. Carlson, Larry Blunk, Dr. Bernard D. Aboba, and Henrik Levkowetz. *Extensible Authentication Protocol (EAP)*. RFC 3748. June 2004. DOI: 10.17487/RFC3748.

[Vuč+20] Mališa Vučinić, Göran Selander, John Preuß Mattsson, and Dan Garcia-Carillo. *Requirements for a Lightweight AKE for OSCORE*. Internet-Draft draft-ietf-lake-reqs-04. Work in Progress. Internet Engineering Task Force, June 2020. 25 pp. URL: https://datatracker.ietf.org/doc/draft-ietf-lake-reqs/04/.

[YSB25] Awaneesh Kumar Yadav, Mohammad Shojafar, and An Braeken. "A Provably Secure Post-Quantum Based EDHOC Protocol". In: *22nd IEEE Consumer Communications & Networking Conference, CCNC 2025, Las Vegas, NV, USA, January 10-13, 2025*. IEEE, 2025, pp. 1–6. DOI: 10.1109/CCNC54725.2025.10976053.

[Zin+17] Jean Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. "HACL*: A Verified Modern Cryptographic Library". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*. Ed. by Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM, 2017, pp. 1789–1806. DOI: 10.1145/3133956.3134043.